Spring Break Cheat Sheets

Tier 1

Tier 1 - Won't Build

43 cards

1. AOP Starter Renamed to spring-boot-starter-aspectj
2. ApacheDS Embedded LDAP Support Removed
3. Spring Batch ChunkHandler Renamed; setJobLauncher Removed
4. Spring Batch JobBuilder(String) Constructor Removed
5. Spring Batch 6 Listener Support Classes Removed
6. Batch Listener Base Classes Removed
7. Spring Batch Core Package Relocations
8. BootstrapRegistry and EnvironmentPostProcessor Relocated
9. Classic Uber-Jar Loader Removed
10. Elasticsearch RestClient Renamed to Rest5Client
11. @EntityScan Relocated to persistence.autoconfigure
12. GraalVM 25 for Native Image Builds
13. Gradle 8.14+ / Gradle 9 Required
14. Hibernate EmptyInterceptor Removed
15. hibernate-jpamodelgen Renamed
16. Hibernate SelectionQuery.setOrder() Removed
17. Session.delete() Removed
18. Hibernate @Where and @OrderBy Removed
19. HttpHeaders No Longer Implements MultiValueMap
20. Jackson Class Renames
21. @JsonComponent and @JsonMixin Renamed to @JacksonComponent and @JacksonMixin
22. Jackson 3.0 Group ID Change
23. Kafka StreamsBuilderFactoryBeanCustomizer Removed
24. ListenableFuture Removed
25. Maven Plugin Version Alignment for AOT
26. @MockBean / @SpyBean Removed
27. OkHttp3 Support Removed
28. OpenSAML 4 Support Removed
29. PropertyMapper.alwaysApplyingWhenNonNull() Removed
30. @PropertyMapping Relocated to test.context
31. RestTemplate Auto-Configuration Removed
32. Security DSL Rewrite
33. SimpDestinationMessageMatcher Removed
34. Spring AMQP RabbitRetryTemplateCustomizer Removed
35. spring-jcl Removed
36. spring-retry Removed from BOM
37. Spring Security Access API Moved to spring-security-access
38. Test-Slice Annotation Starters Relocated
39. Testcontainers 2.0 Package Relocation

40. TestRestTemplate Package Relocated
41. Tomcat WAR Deployment Requires New Runtime Starter
42. Undertow Embedded Server Removed
43. webjars-locator-core Removed from BOM

AOP Starter Renamed to spring-boot-starter-aspectj

Tier 1 - Won't Build Impact: S

spring-boot-starter-aop is no longer in the Spring Boot 4.0 BOM. Replace it with spring-boot-starter-aspectj.

What You'll See

```
$ mvn validate
[ERROR] 'dependencies.dependency.version' for
  org.springframework.boot:spring-boot-starter-aop:jar is missing.
```

What Changed

`spring-boot-starter-aop` has been removed from the Spring Boot BOM. The replacement, `spring-boot-starter-aspectj`, provides the same AspectJ weaving capabilities.

Why

Spring AOP (interface-proxy and CGLIB-proxy based AOP) works without any starter. The old name implied you needed this starter for any AOP, when you only need it for AspectJ load-time or compile-time weaving. The rename makes the distinction clear.

The Fix

pom.xml dependency
Before

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

How To Fix

Replace the starter in pom.xml / build.gradle.

Find `spring-boot-starter-aop` in all build files and replace with `spring-boot-starter-aspectj`. No code changes needed: the classpath content is equivalent.

Scope Check

Search all `pom.xml` and `build.gradle` files for `spring-boot-starter-aop`. Multi-module projects may declare it in several places.

Watch Out

- If you only use Spring's proxy-based AOP (`@Aspect` beans, `@Transactional`, etc.) you may not need this starter at all. Proxy-based AOP is included in `spring-boot-starter` transitively. The aspectj starter is only required if you use AspectJ weaving agents.

Verify

mvn validate — no version is missing error for the AOP starter

Further Info

This is a T1B failure (artifact no longer in the BOM): the build fails at dependency resolution, before any source compiles.

Links

- spring-break module: aop-starter-rename
- Spring Boot 4.0 Migration Guide

ApacheDS Embedded LDAP Support Removed

Tier 1 - Won't Build Impact: M

ApacheDSContainer is removed in Spring Security 7.0. Use UnboundIdContainer instead.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/ApacheDsUsage.java:[3,57]
  error: package org.springframework.security.ldap.server does not contain
  ApacheDSContainer
```

What Changed

`org.springframework.security.ldap.server.ApacheDSContainer` has been removed from Spring Security. The embedded LDAP server capability is now provided exclusively through `org.springframework.security.ldap.server.UnboundIdContainer`.

Why

ApacheDS development had stalled and its Jakarta EE compatibility lagged. UnboundID is actively maintained and provides equivalent functionality for embedded LDAP testing and development use cases.

The Fix

pom.xml — swap the embedded LDAP dependency
Before

```
<dependency>
    <groupId>org.apache.directory.server</groupId>
    <artifactId>apacheds-server-jndi</artifactId>
    <scope>test</scope>
</dependency>
```

After

```
<dependency>
    <groupId>com.unboundid</groupId>
    <artifactId>unboundid-ldapsdk</artifactId>
```

```
    <scope>test</scope>
</dependency>
```

Java config — replace the container class
Before

```
import org.springframework.security.ldap.server.ApacheDSCont
// ...
new ApacheDSContainer("dc=springframework,dc=org", "classpat
```

After

```
import org.springframework.security.ldap.server.UnboundIdCon
// ...
new UnboundIdContainer("dc=springframework,dc=org", "classpa
```

How To Fix

Replace ApacheDSContainer with UnboundIdContainer.

Swap the dependency in your build file and update the import and instantiation. The constructor signature is the same: base DN and LDIF classpath location. The LDIF format is also compatible.

Use Spring Boot's embedded LDAP auto-configuration.

If you're only using embedded LDAP for tests, add `spring-boot-starter-ldap` and set `spring.ldap.embedded.base-dn` in test properties. Spring Boot will auto-configure UnboundID for you.

Scope Check

Search for `ApacheDSContainer` and `apacheds-server-jndi` (or other ApacheDS artifacts) across Java sources and build files.

Watch Out

- LDIF files written for ApacheDS should work with UnboundID, but verify any vendor-specific schema extensions you may have used.

Verify

mvn compile — no cannot find symbol for ApacheDSContainer

Further Info

Both containers were available in 3.5; only UnboundID survives in 4.0.

Links

- spring-break module: apacheds-ldap-removed
- Spring Security 7 LDAP migration

Spring Batch ChunkHandler Renamed; setJobLauncher Removed

Tier 1 - Won't Build Impact: S

ChunkHandler renamed to ChunkRequestHandler. JobStep.setJobLauncher replaced by setJobOperator. Both fail to compile on Boot 4.0.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[4,65]
  error: cannot find symbol
    symbol: class ChunkHandler
    location: package org.springframework.batch.integration.chunk
[ERROR] /src/main/java/com/example/BatchRenamingUsage.java:[18,19]
  error: cannot find symbol
    symbol:   method setJobLauncher(JobLauncher)
```

What Changed

`org.springframework.batch.integration.chunk.ChunkHandler` has been renamed to `ChunkRequestHandler`. `JobStep.setJobLauncher(JobLauncher)` has been replaced by `JobStep.setJobOperator(JobOperator)`.

Why

The rename of `ChunkHandler` clarifies its role in the request/response model of remote chunking. `setJobLauncher` was removed as part of the broader move away from `JobLauncher` toward `JobOperator` as the public API for job execution control.

The Fix

ChunkHandler rename

Before

```
import org.springframework.batch.integration.chunk.ChunkHand
// ...
ChunkHandler handler = ...;
```

After

```
import org.springframework.batch.integration.chunk.ChunkRequ
// ...
ChunkRequestHandler handler = ...;
```

JobStep launcher → operator
Before

```
jobStep.setJobLauncher(jobLauncher);
```

After

```
jobStep.setJobOperator(jobOperator);
```

How To Fix

Rename ChunkHandler to ChunkRequestHandler.

Update the import and all references. Only the name changed; the interface contract is the same.

Replace setJobLauncher with setJobOperator.

Inject a `JobOperator` bean instead of `JobLauncher` and pass it to `setJobOperator()`. `JobOperator` is available as a Spring-managed bean when Spring Batch autoconfiguration is active.

Scope Check

Search for `ChunkHandler` (not `ChunkRequestHandler`) and `setJobLauncher` across all Java/Kotlin sources. Affects remote chunking setups and custom job step configuration.

Watch Out

- `JobLauncher` itself still exists. Only its use in `JobStep` was removed; other usages are unaffected.

Verify

mvn compile — no cannot find symbol for ChunkHandler or setJobLauncher

Further Info

ChunkHandler belongs to the remote-chunking integration module. Both changes are mechanical renames.

Links

- spring-break module: batch-chunkhandler-renamed
- Spring Batch 6.0 Migration Guide

Spring Batch JobBuilder(String) Constructor Removed

Tier 1 - Won't Build Impact: S

JobBuilder(String) is removed. JobRepository is now mandatory at construction: new JobBuilder("name", jobRepository).

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/JobBuilderUsage.java:[9,33]
  error: no suitable constructor found for JobBuilder(java.lang.String)
```

What Changed

`new JobBuilder(String name)` and the subsequent `.repository(JobRepository)` method have been removed. `JobRepository` is now mandatory at construction time: `new JobBuilder(String name, JobRepository repository)`. The same applies to `StepBuilder` and other builder classes.

Why

Spring Batch 6.0 made `JobRepository` non-optional throughout the framework to ensure jobs are always backed by a persistent or in-memory store. Deferring the repository assignment was a source of subtle bugs where jobs ran without persistence.

The Fix

Before — repository set after construction
Before

```
new JobBuilder("myJob")
    .repository(jobRepository)
    .start(step)
    .build();
```

After — repository required at construction
Added

```
new JobBuilder("myJob", jobRepository)
    .start(step)
    .build();
```

@Bean method pattern
Before

```
@Bean
public Job myJob(Step step) {
    return new JobBuilder("myJob").start(step).build();
}
```

After

```
@Bean
public Job myJob(JobRepository jobRepository, Step step) {
    return new JobBuilder("myJob", jobRepository).start(step
}
```

How To Fix

Inject JobRepository and pass it to the constructor.

Add `JobRepository` as a parameter to your `@Bean` method (Spring will inject it) and pass it as the second argument to `new JobBuilder(...)`. The same change is needed for `StepBuilder` and any other builder that previously accepted only a name.

Scope Check

Search for `new JobBuilder(`, `new StepBuilder(`, and `.repository(` across all `@Configuration` classes that define Batch jobs and steps.

Watch Out

- The `.repository()` method is also gone, so adding the repository via the fluent chain won't compile either. Pass it in the constructor; there is no fallback.

Verify

mvn compile — no cannot find symbol for JobBuilder(String) constructor

Further Info

Spring Batch 6.0 makes the JobRepository compulsory framework-wide; it can no longer be deferred or omitted anywhere.

Links

- spring-break module: batch-job-builder-string-constructor
- Spring Batch 6.0 Migration Guide

Spring Batch 6 Listener Support Classes Removed

Tier 1 - Won't Build Impact: S

`JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport` are removed in Spring Batch 6.0. Code extending them fails to compile on Boot 4.0 with "cannot find symbol".

What You'll See

```
// Boot 3.5 (Batch 5.x): classes present, deprecated since 5.0; compiles fine.

// Boot 4.0 (Batch 6.0): classes removed; compilation fails.
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] MyJobListener.java:[4,47] cannot find symbol
    symbol: class JobExecutionListenerSupport
[INFO] BUILD FAILURE
```

What Changed

Spring Batch 6.0 (shipped with Spring Boot 4.0) removed three convenience base classes that had been deprecated since Batch 5.0: `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport`. These classes provided empty default implementations of listener methods.

Why

Java 8 interface default methods eliminate the need for adapter-style base classes. Batch 5.0 deprecated them with that rationale; Batch 6.0 follows through with removal. Implementing the listener interface directly is both more explicit and compatible with composition rather than inheritance.

The Fix

> Replace base class with direct interface implementation
> Before

```
public class MyJobListener extends JobExecutionListenerSuppo
    @Override
    public void afterJob(JobExecution jobExecution) {
        // only override what you need
```

```
    }
}
```

After

```
public class MyJobListener implements JobExecutionListener {
    @Override
    public void afterJob(JobExecution jobExecution) {
        // implement the interface directly
    }
}
```

How To Fix

Implement the listener interface directly.

Replace `extends JobExecutionListenerSupport` with `implements JobExecutionListener`. Do the same for `StepExecutionListenerSupport` → `StepExecutionListener` and `ChunkListenerSupport` → `ChunkListener`. Implement only the methods you use: the rest have default no-op implementations on the interface.

Scope Check

Search for `extends JobExecutionListenerSupport`, `extends StepExecutionListenerSupport`, and `extends ChunkListenerSupport` in your source. Also check Spring XML configuration files and any reflection-based lookups for these class names.

Watch Out

- The compiler only catches direct imports and `extends` clauses. Listeners referenced by class name in Spring XML config or loaded via reflection bypass the compiler and fail only when that code path runs. Search configuration files as well as source.

Verify

mvn clean compile — no "cannot find symbol" errors for JobExecutionListenerSupport, StepExecutionListenerSupport, or ChunkListenerSupport; listeners implement the interfaces directly

Further Info

Complements batch-listener-classes: both modules demonstrate the same underlying Batch 6.0 removal.

Links

- spring-break module: batch-job-serialisation
- Spring Batch 6.0 Migration Guide

Batch Listener Base Classes Removed

Tier 1 - Won't Build Impact: M ⚙ OpenRewrite

Spring Batch 6.0 removed the abstract listener base classes like JobExecutionListenerSupport and StepExecutionListenerSupport.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/batch/JobCompletionListener.java:[4,52]
  error: cannot find symbol
    symbol:   class JobExecutionListenerSupport
    location: package org.springframework.batch.core.listener
[ERROR] /src/main/java/com/example/batch/StepLoggingListener.java:[4,52]
  error: cannot find symbol
    symbol:   class StepExecutionListenerSupport
```

What Changed

Spring Batch 6.0 deleted the `*ListenerSupport` abstract classes: `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, `ChunkListenerSupport`, `ItemReadListenerSupport`, and others. Listeners must now implement the interfaces directly, which have had default methods since Batch 5.0.

Why

Java 8 default methods made the support classes unnecessary: the interfaces provide no-op defaults for every callback, so there's nothing left to inherit.

The Fix

Job listener
Before

```
import org.springframework.batch.core.listener.JobExecutionL

public class JobCompletionListener extends JobExecutionListe
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus
    }
}
```

After

```
import org.springframework.batch.core.JobExecutionListener;

public class JobCompletionListener implements JobExecutionLi
    @Override
    public void afterJob(JobExecution jobExecution) {
        log.info("Job completed: {}", jobExecution.getStatus
    }
}
```

Step listener
Before

```
import org.springframework.batch.core.listener.StepExecution

public class StepLogger extends StepExecutionListenerSupport
```

After

```
import org.springframework.batch.core.StepExecutionListener;

public class StepLogger implements StepExecutionListener {
```

How To Fix

Change extends to implements.

Replace `extends JobExecutionListenerSupport` with `implements JobExecutionListener`. Same for all other `*ListenerSupport` classes. The interfaces have default methods, so you only need to override the callbacks you use.

Use @BeforeJob / @AfterJob annotations.

Spring Batch also supports annotation-based listeners. Annotate methods with `@BeforeJob`, `@AfterJob`, `@BeforeStep`, `@AfterStep` etc. on any POJO; no interface needed.

Scope Check

Search for `ListenerSupport` and `extends.*ListenerSupport` across your batch job classes. Common hits include `JobExecutionListenerSupport`, `StepExecutionListenerSupport`, and `ChunkListenerSupport`.

Watch Out

- If your listener extends a support class and calls `super.afterJob()` or `super.beforeStep()`, remove those super calls. The interface default methods are no-ops, but calling `super` on an interface method looks wrong and may confuse static analysis tools.
- The `ItemWriter` and `ItemProcessor` interfaces also changed in Batch 6.0. If you're fixing listeners, audit your entire batch configuration while you're at it.

Verify

mvn compile and Batch job completes with listener callbacks firing

Further Info

The *ListenerSupport classes were deprecated in Spring Batch 5.0 when the interfaces gained default methods. See also: batch-schema-rename.

Links

- spring-break module: batch-job-serialisation
- Spring Batch 6 migration guide

Spring Batch Core Package Relocations

Tier 1 - Won't Build Impact: M

Core Spring Batch domain classes (Job, JobExecution, etc.) moved from org.springframework.batch.core to org.springframework.batch.core.job.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[3,44]
  error: package org.springframework.batch.core does not contain class Job
[ERROR] /src/main/java/com/example/BatchCoreUsage.java:[4,44]
  error: package org.springframework.batch.core does not contain class JobExecution
```

What Changed

Core domain classes have moved from `org.springframework.batch.core` to subpackages, primarily `org.springframework.batch.core.job`. Affected classes include `Job`, `JobExecution`, `JobInstance`, and `JobParameters` among others.

Why

Spring Batch 6.0 redesigned the domain model to give each concern its own subpackage and make the module boundaries clear.

The Fix

Core domain imports

Before

```
import org.springframework.batch.core.Job;
import org.springframework.batch.core.JobExecution;
import org.springframework.batch.core.JobInstance;
import org.springframework.batch.core.JobParameters;
```

After

```
import org.springframework.batch.core.job.Job;
import org.springframework.batch.core.job.JobExecution;
import org.springframework.batch.core.job.JobInstance;
import org.springframework.batch.core.job.JobParameters;
```

How To Fix

Update imports to include the job subpackage.

Add `.job` to the package path for the affected classes. This is a mechanical find-and-replace: `org.springframework.batch.core.Job` becomes `org.springframework.batch.core.job.Job`, and so on. Check the Spring Batch 6.0 Migration Guide for the complete list of moved classes.

Scope Check

Search for `import org.springframework.batch.core.` across all Java/Kotlin sources. Any import that resolves directly to a class in the root `core` package is likely affected.

Watch Out

- Not all classes in `org.springframework.batch.core` moved. Some remain at the root level. Verify each import against the 6.0 Javadoc rather than bulk-replacing all root-level imports.

Verify

mvn compile — no package does not exist errors for org.springframework.batch.core classes

Further Info

The classes are unchanged; only their packages moved. Expect a large batch of import fixes if your code uses Batch domain types directly.

Links

- spring-break module: batch-package-moves
- Spring Batch 6.0 Migration Guide

BootstrapRegistry and EnvironmentPostProcessor Relocated

Tier 1 - Won't Build Impact: S

BootstrapRegistry moved to org.springframework.boot.bootstrap and EnvironmentPostProcessor moved to org.springframework.boot. Both old import paths are gone.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/BootstrapRegistryUsage.java:[3,47]
  error: package org.springframework.boot does not contain BootstrapRegistry
[ERROR] /src/main/java/com/example/EnvironmentPostProcessorUsage.java:[3,43]
  error: package org.springframework.boot.env does not contain EnvironmentPostProces
```

What Changed

`BootstrapRegistry` and `ConfigurableBootstrapContext` moved from `org.springframework.boot` to `org.springframework.boot.bootstrap`. `EnvironmentPostProcessor` moved the other way, from `org.springframework.boot.env` to `org.springframework.boot`.

Why

Spring Boot 4.0 groups its infrastructure by concern: bootstrap lifecycle classes got their own subpackage, while EnvironmentPostProcessor, used across many contexts, moved to the root package for discoverability.

The Fix

BootstrapRegistry import
Before

```
import org.springframework.boot.BootstrapRegistry;
```

After

```
import org.springframework.boot.bootstrap.BootstrapRegistry;
```

ConfigurableBootstrapContext import

Before

```
import org.springframework.boot.ConfigurableBootstrapContext
```

After

```
import org.springframework.boot.bootstrap.ConfigurableBootst
```

EnvironmentPostProcessor import
Before

```
import org.springframework.boot.env.EnvironmentPostProcessor
```

After

```
import org.springframework.boot.EnvironmentPostProcessor;
```

How To Fix

Update imports.

Find all usages of the old package paths and replace with the new ones. Both changes are pure package moves: no method signatures or behaviour changed.

Search spring.factories / AutoConfiguration.imports.

If you register an `EnvironmentPostProcessor` in `META-INF/spring.factories` or `META-INF/spring/org.springframework.boot.env.EnvironmentPostProcessor.imports`, update the key to `org.springframework.boot.EnvironmentPostProcessor`.

Scope Check

Search for `org.springframework.boot.BootstrapRegistry`, `org.springframework.boot.ConfigurableBootstrapContext`, and `org.springframework.boot.env.EnvironmentPostProcessor` across all Java/Kotlin sources and `META-INF` registration files.

Watch Out

- The `spring.factories` key for `EnvironmentPostProcessor` must also be updated. With the old key your processor is skipped at startup: no error, your environment customisation just never runs.

Verify

mvn compile — no package does not exist errors for BootstrapRegistry or EnvironmentPostProcessor

Further Info

Affects code that customises early application startup or environment configuration.

Links

- spring-break module: bootstrap-registry-relocated
- Spring Boot 4.0 Migration Guide

Classic Uber-Jar Loader Removed

Tier 1 - Won't Build Impact: S

The `CLASSIC` uber-jar loader option is removed in Boot 4.0: the Maven/Gradle plugin fails the build if it is configured. Remove the setting.

What You'll See

```
<!-- pom.xml with CLASSIC loader -->
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <loaderImplementation>CLASSIC</loaderImplementation>
    </configuration>
</plugin>

// Boot 4.0 build error:
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin
...CLASSIC loader implementation is no longer supported
```

What Changed

The `CLASSIC` loader implementation, the original Spring Boot uber-jar format, was removed from the Maven and Gradle plugins. Any build file with `loaderImplementation=CLASSIC` fails at the packaging phase. The default nested-jar loader (introduced in Boot 3.2) is the only supported format.

Why

The classic loader was deprecated in Spring Boot 3.2 when the nested-jar loader arrived; Boot 4.0 removes it. The new loader offers better compatibility with AOT compilation, virtual threads, and GraalVM native image tooling.

The Fix

Maven — remove CLASSIC loader configuration
Before

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
```

```
            <loaderImplementation>CLASSIC</loaderImplementation>
        </configuration>
    </plugin>
```

After

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <!-- No loaderImplementation needed - default is the onl
</plugin>
```

Gradle — remove loaderImplementation property
Before

```
tasks.named("bootJar") {
    loaderImplementation = LoaderImplementation.CLASSIC
}
```

After

```
// No loaderImplementation configuration needed
```

How To Fix

Remove the CLASSIC loader configuration.

Delete `<loaderImplementation>CLASSIC</loaderImplementation>` from the Maven plugin configuration, or remove `loaderImplementation = LoaderImplementation.CLASSIC` from the Gradle boot jar task. The build then defaults to the modern nested-jar loader.

Scope Check

Search build files for `CLASSIC` and `loaderImplementation`. Check `pom.xml`, parent POMs, and any Gradle build scripts that configure the Spring Boot plugin.

Watch Out

- The new nested-jar loader changes the path structure inside the uber jar. Tools that inspect the jar directly (custom launch scripts, Docker entrypoints referencing internal paths) may need updates to reflect the `BOOT-INF/` layout rather than the classic flat layout.

Verify

Application jar is built successfully and starts correctly after removing CLASSIC loader configuration from the build file

Further Info

The plugin rejects the `CLASSIC` value with an explicit error rather than ignoring it, so the failure is loud and easy to find.

Links

- Spring Boot 4.0 Migration Guide

Elasticsearch RestClient Renamed to Rest5Client

Tier 1 - Won't Build Impact: S

Spring Boot 4.0 auto-configures Rest5Client instead of RestClient for Elasticsearch. Code injecting the old type won't compile.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/ElasticsearchUsage.java:[5,35]
  error: cannot find symbol
    symbol:   class RestClient
    location: package org.elasticsearch.client
```

What Changed

Spring Boot 4.0 auto-configures `org.elasticsearch.client.Rest5Client` instead of `org.elasticsearch.client.RestClient`. Components that inject `RestClient` directly will fail because the auto-configured bean is now of type `Rest5Client`.

Why

The Elasticsearch Java client library introduced `Rest5Client` as the successor to the original `RestClient`. Spring Boot 4.0 adopted the newer type to track the upstream library.

The Fix

Import
Before

```
import org.elasticsearch.client.RestClient;
```

After

```
import org.elasticsearch.client.Rest5Client;
```

Injection
Before

```
@Autowired
private RestClient restClient;
```

After

```
@Autowired
private Rest5Client restClient;
```

Customizer bean (if used)
Before

```
@Bean
public RestClientBuilderCustomizer customizer() { ... }
```

After

```
@Bean
public Rest5ClientBuilderCustomizer customizer() { ... }
```

How To Fix

Rename RestClient to Rest5Client.

Replace `RestClient` with `Rest5Client` in all injection points, customizer beans, and direct usages. The API is broadly compatible, but check the Elasticsearch client changelog for subtle differences.

Scope Check

Search for `org.elasticsearch.client.RestClient` and `RestClientBuilderCustomizer` in your Java sources. Any direct injection of the low-level client is affected.

Watch Out

- Do not confuse Elasticsearch's `RestClient` with Spring Framework's `org.springframework.web.client.RestClient`: they are different classes, and only the Elasticsearch one changed.

Verify

mvn compile — no cannot find symbol errors for RestClient auto-config injection

Further Info

The high-level RestHighLevelClient was already removed in Spring Boot 3.x; this is the next step in the same direction.

Links

- spring-break module: elasticsearch-rest5client
- Spring Boot 4.0 Migration Guide

@EntityScan Relocated to persistence.autoconfigure

Tier 1 - Won't Build Impact: S

@EntityScan moved from org.springframework.boot.autoconfigure.domain to org.springframework.boot.persistence.autoconfigure. Old import doesn't compile.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/EntityScanApp.java:[3,52]
  error: package org.springframework.boot.autoconfigure.domain does not exist
```

What Changed

`@EntityScan` moved from `org.springframework.boot.autoconfigure.domain` to `org.springframework.boot.persistence.autoconfigure`. The annotation's behaviour is unchanged; only the package differs.

Why

Spring Boot 4.0 extracted persistence-related auto-configuration into a dedicated `spring-boot-persistence` module. This gives persistence concerns a cleaner home and makes the module boundaries explicit.

The Fix

Import
Before

```
import org.springframework.boot.autoconfigure.domain.EntityS
```

After

```
import org.springframework.boot.persistence.autoconfigure.En
```

How To Fix

Update the import.

Replace `org.springframework.boot.autoconfigure.domain.EntityScan` with `org.springframework.boot.persistence.autoconfigure.EntityScan`. One import

change per file; no other modifications needed.

Scope Check

Search for `org.springframework.boot.autoconfigure.domain.EntityScan` across all Java/Kotlin sources. Typically appears on main application classes or dedicated JPA configuration classes.

Watch Out

- Other classes from `org.springframework.boot.autoconfigure.domain` may have moved too. If you import anything else from that package, check the Spring Boot 4.0 Migration Guide for the new location.

Verify

mvn compile — no package does not exist error for @EntityScan import

Further Info

Several JPA autoconfiguration classes moved together; EntityScan is the most commonly used of the set.

Links

- spring-break module: entityscan-relocated
- Spring Boot 4.0 Migration Guide

GraalVM 25 for Native Image Builds

Tier 1 - Won't Build Impact: M

Spring Boot 4.0 requires GraalVM 25 for native image compilation. Older GraalVM versions produce build failures or runtime crashes.

What You'll See

```
$ mvn -Pnative native:compile
[ERROR] Error: Unsupported native-image version: 22.3.5
[ERROR] Spring Boot 4.0 requires GraalVM 25.0 or later.
[ERROR] Please upgrade your GraalVM installation.
[ERROR]
[ERROR] -> [Help 1]
[ERROR] org.graalvm.buildtools:native-maven-plugin:0.10.6:compile failed
```

What Changed

Spring Boot 4.0 targets GraalVM 25 (based on JDK 25) for native image builds. The `native-maven-plugin` and `org.graalvm.buildtools.native` Gradle plugin must be updated to version 0.10.6+. Older GraalVM versions (22.x, 23.x) are no longer compatible with the reachability metadata shipped in starters.

Why

GraalVM 25 introduced a new metadata format and improved closed-world analysis. Spring's reachability metadata was rewritten to use these features, making it incompatible with older native-image compilers.

The Fix

pom.xml — native build tools plugin

Before

```
<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
    <version>0.9.28</version>
</plugin>
```

After

```
<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
    <version>0.10.6</version>
</plugin>
```

build.gradle — GraalVM plugin
Before

```
plugins {
    id 'org.graalvm.buildtools.native' version '0.9.28'
}
```

After

```
plugins {
    id 'org.graalvm.buildtools.native' version '0.10.6'
}
```

JAVA_HOME / GRAALVM_HOME
Before

```
export GRAALVM_HOME=/opt/graalvm-ce-java17-22.3.5
```

After

```
export GRAALVM_HOME=/opt/graalvm-jdk-25+35
```

How To Fix

Install GraalVM 25.

Download GraalVM 25 from graalvm.org and set `GRAALVM_HOME`. If using SDKMAN: `sdk install java 25-graal`.

Update native build tools.

Bump the `native-maven-plugin` or Gradle `org.graalvm.buildtools.native` plugin to 0.10.6+. The parent POM manages this if you inherit from `spring-boot-starter-parent`.

Scope Check

Search for `native-maven-plugin` and `org.graalvm.buildtools.native` in build files. Check CI pipelines for `GRAALVM_HOME` or `native-image` version references.

Watch Out

- GraalVM 25 is based on JDK 25, but the native image it produces still runs on any compatible OS. Don't confuse the build-time JDK requirement with the runtime requirement.
- If you use custom reachability metadata in `META-INF/native-image/`, verify it still works. The metadata format added new fields in GraalVM 25 and some older entries may be silently ignored.

Verify

native-image --version shows 25+ and mvn -Pnative package succeeds

Further Info

Applies only to native image builds via the spring-boot-starter-parent native profile; standard JVM deployments are unaffected.

Links

- Spring Boot 4.0 Migration Guide

Gradle 8.14+ / Gradle 9 Required

Tier 1 - Won't Build Impact: S

Spring Boot 4.0's Gradle plugin requires Gradle 8.14 or later. Older Gradle wrappers fail immediately at configuration time.

What You'll See

```
$ ./gradlew build
FAILURE: Build failed with an exception.
* Where:
Build file '/project/build.gradle' line: 3
* What went wrong:
An exception occurred applying plugin request [id: 'org.springframework.boot', versi
> Failed to apply plugin 'org.springframework.boot'.
   > Spring Boot plugin requires Gradle 8.14+. Found: 8.5.
```

What Changed

The Spring Boot Gradle plugin now requires Gradle 8.14 as a minimum. Gradle 9.0 is fully supported and recommended for new projects.

Why

Gradle 8.14 introduced configuration cache stability and improved dependency resolution APIs that the Spring Boot plugin now relies on. Supporting older Gradle versions held back build performance improvements.

The Fix

gradle/wrapper/gradle-wrapper.properties
Before

```
distributionUrl=https\://services.gradle.org/distributions/g
```

After

```
distributionUrl=https\://services.gradle.org/distributions/g
```

Alternative: jump to Gradle 9
Before

```
distributionUrl=https\://services.gradle.org/distributions/g
```

After

```
distributionUrl=https\://services.gradle.org/distributions/g
```

How To Fix

Update the Gradle wrapper.

Run `./gradlew wrapper --gradle-version 8.14` (or `9.0`) to update the wrapper in place. Commit the updated wrapper files.

Check plugin compatibility.

After upgrading Gradle, run a build to verify all other plugins are compatible. The Gradle upgrade guide lists deprecations to address.

Scope Check

Check `gradle/wrapper/gradle-wrapper.properties` for the current Gradle version. Also check CI caches and any shared Gradle distributions. Multi-module projects only need one wrapper update.

Watch Out

- If you're on Gradle 7.x, you can't jump straight to 8.14. Gradle deprecation cycles mean you need to fix warnings at each major version boundary. Plan for a staged upgrade: 7.x to 8.0, then 8.0 to 8.14+.
- The Gradle configuration cache is now stable and on by default in Gradle 9. If your build scripts use project-level state at execution time, you'll hit configuration cache errors after upgrading.

Verify

./gradlew --version shows 8.x+ and build succeeds

Further Info

Maven builds are unaffected.

Links

- Spring Boot 4.0 Migration Guide

Hibernate EmptyInterceptor Removed

Tier 1 - Won't Build Impact: S

Hibernate's EmptyInterceptor is gone. Implement Interceptor directly: it has default methods for everything you don't need to override.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/MyHibernateInterceptor.java:[5,43]
  error: cannot find symbol
    symbol: class EmptyInterceptor
    location: package org.hibernate
```

What Changed

`org.hibernate.EmptyInterceptor` has been removed. The `org.hibernate.Interceptor` interface now has default (no-op) implementations for all its methods, so the abstract base class serves no purpose.

Why

When Hibernate 6.0 added default methods to `Interceptor`, `EmptyInterceptor` became redundant. Hibernate 7.0 removed it as part of the overall API cleanup.

The Fix

Before — extending EmptyInterceptor
Before

```
import org.hibernate.EmptyInterceptor;

public class MyInterceptor extends EmptyInterceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
            Object[] state, String[] propertyNames, Type[] t
        // custom logic
        return false;
    }
}
```

After — implementing Interceptor directly

Added

```
import org.hibernate.Interceptor;

public class MyInterceptor implements Interceptor {
    @Override
    public boolean onLoad(Object entity, Object id,
            Object[] state, String[] propertyNames, Type[] t
        // custom logic
        return false;
    }
    // All other methods have default no-op implementations
}
```

How To Fix

Implement Interceptor directly.

Replace `extends EmptyInterceptor` with `implements Interceptor` and remove the `EmptyInterceptor` import. Override only the methods you actually use.

Scope Check

Search for `EmptyInterceptor` and `extends EmptyInterceptor` across all Java/Kotlin sources.

Watch Out

- Some method signatures on `Interceptor` changed in Hibernate 6.x (e.g., `Serializable id` became `Object id`). Verify your overridden signatures match exactly. A mismatch compiles as a new overload and Hibernate never calls it.

Verify

mvn compile — no cannot find symbol for EmptyInterceptor

Further Info

EmptyInterceptor was a convenience class giving no-op implementations of all Interceptor methods. Deprecated in Hibernate 6.0, removed in 7.0.

Links

- spring-break module: hibernate-empty-interceptor-removed
- Hibernate 7.0 Migration Guide

hibernate-jpamodelgen Renamed

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

Hibernate renamed its annotation processor artifact from hibernate-jpamodelgen to hibernate-processor. Metamodel generation breaks.

What You'll See

```
$ mvn compile
[WARNING] The POM for org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0 is missing,
  information available
[ERROR] Failed to execute goal on project my-service: Could not resolve dependencies
  Could not find artifact org.hibernate.orm:hibernate-jpamodelgen:jar:7.0.0
  in central (https://repo.maven.apache.org/maven2)
```

What Changed

Hibernate 7.0 (shipped with Spring Boot 4.0) renamed the annotation processor artifact from `org.hibernate.orm:hibernate-jpamodelgen` to `org.hibernate.orm:hibernate-processor`.

Why

The processor now validates HQL/JPQL queries at compile time and generates type-safe query methods as well as the JPA static metamodel. The new name reflects the broader scope.

The Fix

pom.xml — annotation processor dependency

Before

```
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
    <scope>provided</scope>
</dependency>
```

After

```
<dependency>
    <groupId>org.hibernate.orm</groupId>
```

```
    <artifactId>hibernate-processor</artifactId>
    <scope>provided</scope>
</dependency>
```

build.gradle — annotation processor
Before

```
annotationProcessor 'org.hibernate.orm:hibernate-jpamodelgen
```

After

```
annotationProcessor 'org.hibernate.orm:hibernate-processor'
```

maven-compiler-plugin annotationProcessorPaths
Before

```
<path>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
</path>
```

After

```
<path>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-processor</artifactId>
</path>
```

How To Fix

Rename the artifact.

Replace `hibernate-jpamodelgen` with `hibernate-processor` in your POM or Gradle build file. The Spring Boot BOM manages the version: change only the artifact ID.

Check compiler plugin configuration.

If you declared the processor in `maven-compiler-plugin`'s `annotationProcessorPaths`, update it there too.

Scope Check

Search for `hibernate-jpamodelgen` in all POM and Gradle files. Also check `annotationProcessorPaths` in `maven-compiler-plugin` configuration.

Watch Out

- The new `hibernate-processor` validates your HQL queries at compile time by default. Invalid queries that previously failed only at runtime now fail the build. That is a feature, but it can surface hidden bugs.
- If your IDE configures annotation processors separately (e.g., IntelliJ's settings), update that too. Otherwise the metamodel generates in Maven but not in the IDE.

Verify

mvn compile uses hibernate-processor and metamodel generates

Further Info

Driven by Hibernate 7.0. See also: cascade-save-update, session-delete-removed.

Links

- Spring-Break Demo
- Hibernate 7 migration guide

Hibernate SelectionQuery.setOrder() Removed

Tier 1 - Won't Build Impact: S

Hibernate's incubating SelectionQuery.setOrder() was removed in 7.0. Use an ORDER BY clause in HQL instead.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/HibernateQueryUsage.java:[14,15]
  error: cannot find symbol
    symbol:   method setOrder(java.util.List)
    location: interface SelectionQuery<Product>
```

What Changed

The `@Incubating` `setOrder(List<Order>)` method on `SelectionQuery` (and `Query`) was removed in Hibernate 7.0. It had been added in Hibernate 6.x as part of Jakarta Data integration work but was never promoted to stable API.

Why

The method was always experimental. Hibernate 7.0 completed the Jakarta Data integration under a more stable API surface and removed the incubating intermediate.

The Fix

> Programmatic order via removed setOrder()
> Before
>
> ```
> SelectionQuery<Product> query =
>     session.createSelectionQuery("from Product", Product.cla
> query.setOrder(List.of(Order.asc(Product.class, "name")));
> ```
>
> Fix: inline ORDER BY in HQL
> Added
>
> ```
> SelectionQuery<Product> query =
>     session.createSelectionQuery("from Product order by name
> ```

How To Fix

Move ordering into the HQL query string.

Add an `ORDER BY` clause directly to the HQL/JPQL string. This is the most portable approach and works on both 3.5 and 4.0.

Use CriteriaQuery with Order.

For fully programmatic ordering, use the JPA Criteria API: `criteriaQuery.orderBy(builder.asc(root.get("name")))`.

Scope Check

Search for `.setOrder(` on query objects across all repository and data-access classes. Rare outside code written to try the Hibernate 6.x incubating API.

Watch Out

- `org.hibernate.query.Order` (the parameter type) is also affected. If you imported it, that import can be removed once you stop using `setOrder`.

Verify

mvn compile — no cannot find symbol for setOrder on SelectionQuery

Further Info

setOrder() only existed during the Hibernate 6.x window. Code that adopted it must move to HQL ORDER BY or the SelectionSpecification API.

Links

- spring-break module: hibernate-query-setorder-removed
- Hibernate 7.0 Migration Guide

Session.delete() Removed

Tier 1 - Won't Build Impact: M ⚙ OpenRewrite

Hibernate 7.0 removed Session.delete(): use Session.remove() instead. Any direct Hibernate Session usage for deleting entities won't compile.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/repo/CustomRepoImpl.java:[28,16]
  error: cannot find symbol
    symbol:   method delete(com.example.model.Order)
    location: variable session of type org.hibernate.Session
```

What Changed

Hibernate 7.0 removed the legacy `Session.delete()` method. The JPA-standard `EntityManager.remove()` (which maps to `Session.remove()`) is the only way to delete entities. `Session.save()`, `Session.update()`, and `Session.saveOrUpdate()` were also removed.

Why

These methods were Hibernate-proprietary duplicates of the JPA standard API. Having two ways to do the same thing caused confusion about cascade behaviour and flush timing. The JPA methods have clearer semantics.

The Fix

Deleting an entity
Before

```
session.delete(order);
```

After

```
session.remove(order);
```

Saving an entity
Before

```
session.save(newOrder);
```

After

```
session.persist(newOrder);
```

Updating an entity
Before

```
session.update(existingOrder);
```

After

```
session.merge(existingOrder);
```

Save-or-update pattern
Before

```
session.saveOrUpdate(order);
```

After

```
session.merge(order);
```

How To Fix

Replace with JPA standard methods.

`delete()` becomes `remove()`, `save()` becomes `persist()`, `update()` becomes `merge()`, `saveOrUpdate()` becomes `merge()`.

Prefer EntityManager over Session.

If you're unwrapping the `Session` just to call these methods, consider using `EntityManager` directly. It has the same `persist()`, `merge()`, and `remove()` methods.

Scope Check

Search for `session.delete(`, `session.save(`, `session.update(`, and `session.saveOrUpdate(`. Also search for `Session.delete` in any utility or base repository classes.

Watch Out

- `merge()` returns the managed instance; `saveOrUpdate()` did not. If your code used the original reference after the call, switch to the instance `merge()` returns.
- `remove()` requires a managed entity. If you called `delete()` on detached entities, `merge()` or re-fetch them first. Otherwise you get an `IllegalArgumentException` at runtime.

Verify

mvn compile — no Session.delete() symbol errors

Further Info

Driven by Hibernate 7.0, upstream of Spring Boot 4.0. The methods were deprecated in Hibernate 6.0. See also: cascade-save-update, hibernate-dialect-removal.

Links

- spring-break module: hibernate-session-delete
- Hibernate 7 migration guide

Hibernate @Where and @OrderBy Removed

Tier 1 - Won't Build Impact: S

Hibernate's @Where and @OrderBy are removed. Replace with @SQLRestriction and @SQLOrder.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/User.java:[5,39]
  error: cannot find symbol
    symbol: class Where
    location: package org.hibernate.annotations
[ERROR] /src/main/java/com/example/User.java:[6,39]
  error: cannot find symbol
    symbol: class OrderBy
    location: package org.hibernate.annotations
```

What Changed

`@org.hibernate.annotations.Where` and `@org.hibernate.annotations.OrderBy` have been removed. The replacements are `@org.hibernate.annotations.SQLRestriction` and `@org.hibernate.annotations.SQLOrder`, which take the same SQL fragment arguments.

Why

The new annotation names make it explicit that the argument is a raw SQL fragment, not a JPQL or HQL expression. This avoids confusion with the JPA `@OrderBy` (which takes attribute names) and with Hibernate's HQL-level features.

The Fix

Entity annotations
Before

```
import org.hibernate.annotations.Where;
import org.hibernate.annotations.OrderBy;

@OneToMany
@Where(clause = "deleted = false")
@OrderBy(clause = "name desc")
private List<Post> posts;
```

After

```
import org.hibernate.annotations.SQLRestriction;
import org.hibernate.annotations.SQLOrder;

@OneToMany
@SQLRestriction("deleted = false")
@SQLOrder("name desc")
private List<Post> posts;
```

How To Fix

Replace @Where with @SQLRestriction.

Change the annotation name and attribute: `@Where(clause = "...")` becomes `@SQLRestriction("...")`. The SQL fragment is the same.

Replace @OrderBy with @SQLOrder.

Change the annotation name and attribute: `@OrderBy(clause = "...")` becomes `@SQLOrder("...")`.

Scope Check

Search for `@Where` and `@OrderBy` from the `org.hibernate.annotations` package across all entity classes and embeddables. Also check `@WhereJoinTable` if used.

Watch Out

- Only Hibernate's `@OrderBy` needs replacing. JPA's `@jakarta.persistence.OrderBy` takes attribute names, not SQL, and is unaffected.

Verify

mvn compile — no cannot find symbol for @Where or @OrderBy from Hibernate

Further Info

@SQLRestriction and @SQLOrder arrived in Hibernate 6.3; the old annotations were deprecated then and removed in 7.0.

Links

- spring-break module: hibernate-where-orderby-removed

- Hibernate 7.0 Migration Guide

HttpHeaders No Longer Implements MultiValueMap

Tier 1 - Won't Build Impact: M

HttpHeaders no longer implements MultiValueMap. Code that treats headers as a general-purpose map fails to compile.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[8,24]
  error: incompatible types: HttpHeaders cannot be converted to MultiValueMap
[ERROR] /src/main/java/com/example/HttpHeadersUsage.java:[12,16]
  error: cannot find symbol
    symbol: method containsKey(String)
```

What Changed

`HttpHeaders` no longer implements `MultiValueMap<String, String>` or `Map<String, List<String>>`. Map-style methods like `containsKey()`, `keySet()`, and `entrySet()` are gone from the type. Header-specific methods replace them.

Why

Treating HTTP headers as a generic map encouraged misuse and blocked performance work. The dedicated `HttpHeaders` API is more expressive and frees the backing store from the Map contract.

The Fix

Replace Map method calls with HttpHeaders equivalents

Before

```
headers.containsKey("Content-Type")
```

After

```
headers.containsHeader("Content-Type")
```

Before

```
Set<String> names = headers.keySet();
```

After

```
Set<String> names = headers.headerNames();
```

Before

```
Set<Map.Entry<String, List<String>>> entries = headers.entry
```

After

```
Set<Map.Entry<String, List<String>>> entries = headers.heade
```

Passing HttpHeaders where MultiValueMap is expected
Before

```
void process(MultiValueMap<String, String> map) { ... }
process(headers);
```

After

```
void process(HttpHeaders headers) { ... }
// or, if the method signature can't change:
process(headers.toMultiValueMap());
```

How To Fix

Replace Map methods with HttpHeaders methods.

`containsKey(k)` → `containsHeader(k)`, `keySet()` → `headerNames()`, `entrySet()` → `headerSet()`. These replacements compile on both 3.5 and 4.0.

For method signatures that accept MultiValueMap.

If you have a utility method that accepts `MultiValueMap<String, String>` and you pass `HttpHeaders` into it, either change the parameter type to `HttpHeaders`, or call `headers.toMultiValueMap()` at the call site.

Scope Check

Search for `MultiValueMap` usages near `HttpHeaders` across your codebase. Also search for `.containsKey(`, `.keySet()`, `.entrySet()` called on variables of type `HttpHeaders`.

Watch Out

- Code in Spring interceptors, filters, and custom converters that inspects headers using Map idioms is the most common source of these failures. Check `HandlerInterceptor`, `ClientHttpRequestInterceptor`, and `ResponseBodyAdvice` implementations.
- `toMultiValueMap()` returns a copy, not a live view. Mutations to the returned map do not affect the original `HttpHeaders`.

Verify

mvn compile — no incompatible types or cannot find symbol errors on HttpHeaders usage

Further Info

Driven by Spring Framework 7.0. HttpHeaders had been a MultiValueMap subtype since early in Spring's history.

Links

- spring-break module: httpheaders-multivaluemap
- Spring Framework 7.0 Release Notes

Jackson Class Renames

Tier 1 - Won't Build Impact: M ⚙ OpenRewrite

Jackson 3.0 renamed core classes like JsonSerializer, JsonDeserializer, and SerializerProvider. Every custom serialiser is now broken.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/DateSerializer.java:[4,44]
  error: cannot find symbol
    symbol:   class JsonSerializer
    location: package tools.jackson.databind
[ERROR] /src/main/java/com/example/DateSerializer.java:[12,47]
  error: cannot find symbol
    symbol:   class SerializerProvider
    location: package tools.jackson.databind
```

What Changed

Jackson 3.0 renamed several core classes: `JsonSerializer` became `ValueSerializer`, `JsonDeserializer` became `ValueDeserializer`, `SerializerProvider` became `SerializationContext`, `DeserializationContext` kept its name but moved packages. Method signatures on these classes also changed.

Why

The "Json" prefix was misleading: Jackson handles YAML, XML, CBOR, and other formats, so serialisers work with values, not specifically JSON. The rename also aligned the class hierarchy with the new streaming API.

The Fix

Custom serializer class

Before

```
import com.fasterxml.jackson.databind.JsonSerializer;
import com.fasterxml.jackson.databind.SerializerProvider;

public class DateSerializer extends JsonSerializer<LocalDate
    @Override
```

```
    public void serialize(LocalDate value, JsonGenerator gen
            SerializerProvider provider) throws IOException
```

After

```
import tools.jackson.databind.ser.ValueSerializer;
import tools.jackson.databind.SerializationContext;

public class DateSerializer extends ValueSerializer<LocalDat
    @Override
    public void serialize(LocalDate value, JsonGenerator gen
            SerializationContext ctxt) throws IOException {
```

Custom deserializer class
Before

```
import com.fasterxml.jackson.databind.JsonDeserializer;

public class DateDeserializer extends JsonDeserializer<Local
```

After

```
import tools.jackson.databind.deser.ValueDeserializer;

public class DateDeserializer extends ValueDeserializer<Loca
```

Exception handling
Before

```
import com.fasterxml.jackson.databind.JsonMappingException;
```

After

```
import tools.jackson.databind.DatabindException;
```

How To Fix

OpenRewrite (recommended).

Run the Jackson 3 type changes recipe. It handles class renames, method signature updates, and import changes in one pass.

Manual migration.

Replace class names and imports file by file. The full mapping is in the Jackson 3 migration guide. Don't forget to update method parameter types in overridden methods.

Scope Check

Search for `extends JsonSerializer`, `extends JsonDeserializer`, `SerializerProvider`, and `JsonMappingException`. Every custom serialiser, deserialiser, and exception handler needs updating.

Watch Out

- The method signatures changed too. `serialize()` now takes a `SerializationContext` instead of `SerializerProvider`. Renaming the class but keeping the old parameter type gives an abstract method error instead of a clean override.
- Annotations like `@JsonSerialize(using = ...)` still work but must reference the new class names. A custom serialiser registered via annotation will fail at runtime if it extends the wrong base class.

Verify

mvn compile — no JsonSerializer/SerializerProvider errors

Further Info

Driven by Jackson 3.0, upstream of Spring Boot 4.0, and finalised in Jackson 3.0-rc1. Affects spring-boot-starter-json consumers with custom serialisers. See also: jackson-group-id, jackson-exception-hierarchy.

Links

- spring-break module: jackson-class-renames
- Jackson 3 migration guide

@JsonComponent and @JsonMixin Renamed to @JacksonComponent and @JacksonMixin

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

`@JsonComponent` and `@JsonMixin` are removed in Boot 4.0 and fail to compile. Replace with `@JacksonComponent` and `@JacksonMixin`.

What You'll See

```
@JsonComponent
public class MoneySerializer extends JsonSerializer<Money> { ... }

// Boot 4.0 compile error:
error: cannot find symbol
  symbol: @JsonComponent
  location: class MoneySerializer
```

What Changed

Spring Boot's custom serialiser registration annotations were renamed to align with Jackson 3's branding. `@JsonComponent` becomes `@JacksonComponent` and `@JsonMixin` becomes `@JacksonMixin`. The functionality is identical: only the annotation names changed.

Why

Jackson 3 rebranded from the `com.fasterxml.jackson` group ID to `tools.jackson`. Spring Boot's annotation names were updated to match, making it clear they belong to the Jackson 3 ecosystem rather than the legacy Jackson 2 stack.

The Fix

Custom serializer

Before

```
import org.springframework.boot.jackson.JsonComponent;

@JsonComponent
public class MoneySerializer extends JsonSerializer<Money> {
```

After

```
import org.springframework.boot.jackson.JacksonComponent;

@JacksonComponent
public class MoneySerializer extends JsonSerializer<Money> {
```

Mixin
Before

```
import org.springframework.boot.jackson.JsonMixin;

@JsonMixin(Money.class)
public abstract class MoneyMixin { ... }
```

After

```
import org.springframework.boot.jackson.JacksonMixin;

@JacksonMixin(Money.class)
public abstract class MoneyMixin { ... }
```

How To Fix

Rename the annotations.

Replace all `@JsonComponent` with `@JacksonComponent` and all `@JsonMixin` with `@JacksonMixin`. Update the imports accordingly.

Scope Check

Grep for `@JsonComponent` and `@JsonMixin` across your source tree. Also search for `import org.springframework.boot.jackson.JsonComponent` and `import org.springframework.boot.jackson.JsonMixin`.

Watch Out

- Do not confuse Spring Boot's `@JacksonComponent` with Jackson's own `@JsonSerialize` / `@JsonDeserialize` annotations. Spring Boot's annotation auto-registers the serialiser with the `ObjectMapper` bean; Jackson's own annotations require explicit wiring.

Verify

Custom serializers and mixins annotated with the new names register correctly with the auto-configured ObjectMapper

Further Info

@JsonComponent and @JsonMixin were Spring Boot-specific annotations that registered custom serialisers, deserialisers, and mixins with the auto-configured ObjectMapper.

Links

- spring-break module: jackson-component-rename
- Spring Boot 4.0 Migration Guide

Jackson 3.0 Group ID Change

Tier 1 - Won't Build Impact: L ⚙ OpenRewrite

Jackson 3.0 moved from com.fasterxml.jackson to tools.jackson. Every Maven coordinate and every import path is now wrong.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/JacksonConfig.java:[3,32]
  package com.fasterxml.jackson.databind does not exist
[ERROR] /src/main/java/com/example/JacksonConfig.java:[8,5]
  cannot find symbol
    symbol: class ObjectMapper
```

What Changed

Jackson 3.0 moved from the `com.fasterxml.jackson` Maven group to `tools.jackson`. Spring Boot 4.0's managed dependencies point to Jackson 3.0, so the old group ID no longer resolves. Every import statement using `com.fasterxml.jackson` breaks.

Why

Jackson's maintainers separated the project identity from the original FasterXML organisation, giving a clean break to drop deprecated APIs and ship Jackson 3.0 as a proper major release.

The Fix

pom.xml — explicit Jackson dependency

Before

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>tools.jackson.core</groupId>
```

```
    <artifactId>jackson-databind</artifactId>
</dependency>
```

Java imports — ObjectMapper and core types
Before

```
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.core.JsonProcessingException;
```

After

```
import tools.jackson.databind.ObjectMapper;
import tools.jackson.core.JacksonException;
```

How To Fix

OpenRewrite (recommended).

Run the Jackson 3 migration recipe. Handles group ID changes and import updates across the entire codebase in one pass.

Manual: update Maven coordinates first.

Replace `com.fasterxml.jackson` with `tools.jackson` in all POM or Gradle files, then fix imports file by file.

Scope Check

Search your codebase for `com.fasterxml.jackson`. Every hit is a coordinate or import that needs updating. If you use `spring-boot-starter-json` or `spring-boot-starter-web`, Jackson arrives transitively. You still need to update explicit dependency declarations and all source-level imports.

Watch Out

- The class renames (JsonSerializer → ValueSerializer, SerializerProvider → SerializationContext) arrive in the same Jackson 3.0 upgrade. Fixing only the group ID leaves broken class references. See jackson-class-renames for the full list.
- Jackson exceptions (`JsonMappingException`, `JsonProcessingException`) are `Serializable`. If you've persisted these anywhere (dead letter queues, error logs), those serialised streams contain the old class names and won't deserialise correctly after migration.

Verify

mvn compile succeeds with no com.fasterxml errors

Further Info

Driven by Jackson 3.0, upstream of Spring Boot 4.0. The FasterXML to tools.jackson rename was announced in 2023 and finalised in Jackson 3.0-rc1. See also: jackson-class-renames, jackson-exception-hierarchy.

Links

- spring-break module: jackson-group-id
- Jackson 3 migration guide

Kafka StreamsBuilderFactoryBeanCustomizer Removed

Tier 1 - Won't Build Impact: S

Spring Boot's StreamsBuilderFactoryBeanCustomizer is gone. Replace it with Spring Kafka's StreamsBuilderFactoryBeanConfigurer.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/KafkaCustomizerUsage.java:[3,60]
  error: package org.springframework.boot.autoconfigure.kafka does not contain
  StreamsBuilderFactoryBeanCustomizer
```

What Changed

`org.springframework.boot.autoconfigure.kafka.StreamsBuilderFactoryBeanCustomizer` has been removed. The replacement is `org.springframework.kafka.config.StreamsBuilderFactoryBeanConfigurer` from Spring Kafka itself.

Why

Spring Boot's interface was a thin duplicate of one Spring Kafka already provides. Removing it cuts an unnecessary indirection.

The Fix

Import
Before

```
import org.springframework.boot.autoconfigure.kafka.StreamsB
```

After

```
import org.springframework.kafka.config.StreamsBuilderFactor
```

Bean method signature
Before

```
@Bean
public StreamsBuilderFactoryBeanCustomizer customizer() { ..
```

After

```
@Bean
public StreamsBuilderFactoryBeanConfigurer configurer() { ..
```

implements clause
Before

```
public class MyCustomizer implements StreamsBuilderFactoryBe
```

After

```
public class MyCustomizer implements StreamsBuilderFactoryBe
```

How To Fix

Switch to StreamsBuilderFactoryBeanConfigurer.

Replace the import and the implements / return type declaration. The interface method signature is compatible: the method you override is `configure(StreamsBuilderFactoryBean)` in both cases.

Scope Check

Search for `StreamsBuilderFactoryBeanCustomizer` in all Java/Kotlin sources. Only Kafka Streams applications are affected.

Watch Out

- The method name on `StreamsBuilderFactoryBeanConfigurer` is `configure`, same as the old `customize`. Verify the exact method name in the Spring Kafka version you're using: it differs between minor versions.

Verify

mvn compile — no cannot find symbol for StreamsBuilderFactoryBeanCustomizer

Further Info

The removal is part of shrinking Boot's Kafka auto-configuration surface; the upstream Spring Kafka interface is the long-term home.

Links

- spring-break module: kafka-streams-customizer-removed
- Spring Boot 4.0 Migration Guide

ListenableFuture Removed

Tier 1 - Won't Build Impact: M

Spring Framework removed ListenableFuture entirely. All async code using it must switch to CompletableFuture.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/service/AsyncService.java:[4,49]
  error: cannot find symbol
    symbol:   class ListenableFuture
    location: package org.springframework.util.concurrent
[ERROR] /src/main/java/com/example/service/AsyncService.java:[5,49]
  error: cannot find symbol
    symbol:   class ListenableFutureCallback
    location: package org.springframework.util.concurrent
```

What Changed

`ListenableFuture`, `ListenableFutureCallback`, `ListenableFutureTask`, and related classes in `org.springframework.util.concurrent` were deleted from Spring Framework 7.0. All Spring APIs that previously returned `ListenableFuture` now return `java.util.concurrent.CompletableFuture`.

Why

`ListenableFuture` predated Java 8. Once `CompletableFuture` arrived in the JDK, maintaining a parallel abstraction added complexity with no benefit.

The Fix

Async method return type

Before

```
import org.springframework.util.concurrent.ListenableFuture;

@Async
public ListenableFuture<String> fetchData() {
    return AsyncResult.forValue(doWork());
}
```

After

```
import java.util.concurrent.CompletableFuture;

@Async
public CompletableFuture<String> fetchData() {
    return CompletableFuture.completedFuture(doWork());
}
```

Callback-based usage
Before

```
ListenableFuture<String> future = asyncService.fetchData();
future.addCallback(
    result -> log.info("Success: {}", result),
    ex -> log.error("Failed", ex)
);
```

After

```
CompletableFuture<String> future = asyncService.fetchData();
future.whenComplete((result, ex) -> {
    if (ex != null) log.error("Failed", ex);
    else log.info("Success: {}", result);
});
```

How To Fix

Replace with CompletableFuture.

Change return types from `ListenableFuture<T>` to `CompletableFuture<T>`. Replace `AsyncResult.forValue()` with `CompletableFuture.completedFuture()`. Replace `addCallback()` with `whenComplete()` or `thenAccept()`.

Use OpenRewrite.

The `org.openrewrite.java.spring.framework.MigrateSpringAssert` recipe set includes `ListenableFuture` migration.

Scope Check

Search for `ListenableFuture`, `ListenableFutureCallback`, and `AsyncResult`. Also check for `addCallback(` calls: these usage sites need rewriting even if the declaration is in a library.

Watch Out

- `AsyncResult` was also removed. If you used `new AsyncResult<>(value)` or `AsyncResult.forValue()`, both must change to `CompletableFuture.completedFuture()`.
- If you used `ListenableFuture` in a messaging listener (e.g., Spring Kafka's `KafkaTemplate.send()`), the return type changed upstream too. Check your Kafka and AMQP send-and-confirm patterns.

Verify

mvn compile — no ListenableFuture symbol errors

Further Info

Deprecated in Spring Framework 6.0, removed in 7.0 (tracked in spring-framework#33808). Spring Kafka and Spring AMQP return types that previously used ListenableFuture changed too.

Links

- Spring-Break Demo
- Spring Framework ListenableFuture removal

Maven Plugin Version Alignment for AOT

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

The Spring Boot Maven plugin's AOT processing goals changed in 4.0. Mismatched plugin versions or stale AOT configuration breaks the build.

What You'll See

```
$ mvn spring-boot:process-aot
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin:4.0
  on project my-service: Execution default of goal
  org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot failed:
  An API incompatibility was encountered while executing
  org.springframework.boot:spring-boot-maven-plugin:4.0.0:process-aot:
  java.lang.NoSuchMethodError:
    'void org.springframework.aot.generate.GenerationContext.<init>(java.lang.ClassL
```

What Changed

The `spring-boot-maven-plugin` must match the Spring Boot version exactly. In 4.0 the AOT processing API changed: the `process-aot` and `process-test-aot` goals use new generation context methods. A plugin version mismatch causes `NoSuchMethodError` at build time.

Why

AOT processing was refactored for GraalVM 25 compatibility and to support the new Spring Framework 7.0 code generation model. The plugin version must be kept in lock-step with the framework.

The Fix

pom.xml — plugin version

Before

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>3.4.1</version>
</plugin>
```

After

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <version>4.0.0</version>
</plugin>
```

pom.xml — remove explicit AOT execution if using parent POM
Before

```
<execution>
    <id>process-aot</id>
    <goals><goal>process-aot</goal></goals>
</execution>
```

After

```
<!-- AOT goals are now configured automatically by the paren
```

How To Fix

Align the plugin version with Spring Boot.

If you inherit from `spring-boot-starter-parent`, the plugin version is managed automatically: remove any explicit `<version>` tag. If you use a BOM without the parent POM, set the plugin version to `4.0.0` explicitly.

Remove manual AOT execution blocks.

The parent POM now binds the AOT goals automatically. Keeping an explicit `<execution>` block can cause the goal to run twice or with stale configuration.

Scope Check

Search for `spring-boot-maven-plugin` across all POM files. Check for explicit `<version>` tags and manual AOT `<execution>` blocks. Multi-module projects need every module's POM checked.

Watch Out

- If you're not using native images, AOT processing is still enabled by default in 4.0 for startup optimisation. You may hit this error even if you never intentionally configured AOT.
- Parent POM version and plugin version must match. If your `<parent>` points to 4.0.0 but a plugin override still says 3.x, the build will fail with cryptic reflection errors.

Verify

mvn spring-boot:aot-generate runs without plugin errors

Further Info

AOT processing runs by default in Boot 4.0 even for non-native builds. See also: graalvm-25.

Links

- Spring Boot 4.0 Migration Guide

@MockBean / @SpyBean Removed

Tier 1 - Won't Build Impact: M ⚙ OpenRewrite

Spring Boot 4.0 removed the @MockBean and @SpyBean annotations. Test sources that import them fail to compile. Replace with @MockitoBean and @MockitoSpyBean.

What You'll See

```
$ mvn clean test
[ERROR] COMPILATION ERROR :
[ERROR] OrderServiceTest.java:[3,48] package
  org.springframework.boot.test.mock.mockito does not exist
[ERROR] OrderServiceTest.java:[18,6] cannot find symbol
    symbol: class MockBean
[INFO] BUILD FAILURE
---
The build stops at test-compile. No tests run.
```

What Changed

The `@MockBean` and `@SpyBean` annotations from `org.springframework.boot.test.mock.mockito` were deprecated in Spring Boot 3.4 and removed in 4.0. They are replaced by `@MockitoBean` and `@MockitoSpyBean` from `org.springframework.test.context.bean.override.mockito`, which live in Spring Framework 7 itself rather than Spring Boot.

Why

The new annotations live in Spring Framework's test context, so they work in plain Framework tests as well as Boot tests. Their cleaner bean override mechanism also avoids some of the context caching issues that plagued `@MockBean`.

The Fix

Import change

Before

```
import org.springframework.boot.test.mock.mockito.MockBean;
```

After

```
import org.springframework.test.context.bean.override.mockit
```

Before

```
import org.springframework.boot.test.mock.mockito.SpyBean;
```

After

```
import org.springframework.test.context.bean.override.mockit
```

Annotation usage
Before

```
@MockBean
private OrderRepository orderRepository;

@SpyBean
private NotificationService notificationService;
```

After

```
@MockitoBean
private OrderRepository orderRepository;

@MockitoSpyBean
private NotificationService notificationService;
```

How To Fix

Find-and-replace annotations.

Replace `@MockBean` with `@MockitoBean` and `@SpyBean` with `@MockitoSpyBean`. Update the imports from `org.springframework.boot.test.mock.mockito` to `org.springframework.test.context.bean.override.mockito`.

OpenRewrite recipe.

The Spring Boot 4.0 OpenRewrite migration recipe handles this rename automatically across the entire test codebase.

Check for @MockBean attributes.

If you used `@MockBean(name = "...")` or `@MockBean(classes = ...)`, verify the equivalent attributes exist on `@MockitoBean`. The attribute names may differ slightly.

Scope Check

Search for `@MockBean` and `@SpyBean` across `src/test`. Every test class using these annotations needs updating. That can be dozens or hundreds of test files. Count them before starting.

Watch Out

- If you upgrade without running `mvn clean`, stale test classes compiled against 3.5 can still load and run. Spring no longer recognises the old annotations, so the mock fields silently stay `null` and the first symptom is a `NullPointerException` mid-test, which looks like a runtime injection failure. Run a clean build and the real compile failure appears.
- The `@MockitoBean` reset behaviour between tests may differ from `@MockBean`. If you relied on mocks being reset between test methods, verify that behaviour is preserved.
- The new annotations are in the Spring Framework package, not Spring Boot. If you have test utility classes that reference the old annotation by fully qualified name (e.g., in custom annotations or reflection), those references need updating too.
- IDE templates and test generators may still produce `@MockBean`. Update your IDE templates after migrating.

Verify

mvn clean test — test sources compile; no "cannot find symbol" errors for @MockBean or @SpyBean

Further Info

The whole org.springframework.boot.test.mock.mockito package is gone, so any other import from it fails too.

Links

- spring-break module: mockbean-removed
- Testing changes in Spring Boot 4.0
- Spring Boot 4.0 Migration Guide

OkHttp3 Support Removed

Tier 1 - Won't Build Impact: M

Spring Boot 4.0 dropped the OkHttp3 client factory and BOM entry. Projects using OkHttp3RestClient or the managed dependency won't compile.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/config/RestConfig.java:[5,48]
  package org.springframework.http.client.OkHttp3ClientHttpRequestFactory does not e
[ERROR] /src/main/java/com/example/config/RestConfig.java:[14,16]
  cannot find symbol
    symbol:   class OkHttp3ClientHttpRequestFactory
    location: class com.example.config.RestConfig
```

What Changed

The `OkHttp3ClientHttpRequestFactory` class was removed from Spring Framework 7.0. Spring Boot 4.0 also removed `com.squareup.okhttp3` from its managed dependency BOM. Any code creating a `RestClient` or `RestTemplate` with the OkHttp3 factory won't compile.

Why

The JDK's built-in `java.net.http.HttpClient` (since Java 11) and the Apache HttpClient 5 cover the same use cases. Maintaining a third HTTP client integration added complexity without clear benefit.

The Fix

RestClient configuration
Before

```
import org.springframework.http.client.OkHttp3ClientHttpRequ

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new OkHttp3ClientHttpRequestFactory(
        .build();
}
```

After

```
import org.springframework.http.client.JdkClientHttpRequestF

@Bean
public RestClient restClient() {
    return RestClient.builder()
        .requestFactory(new JdkClientHttpRequestFactory())
        .build();
}
```

pom.xml — remove OkHttp dependency
Before

```
<dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>okhttp</artifactId>
</dependency>
```

After

```
<!-- JdkClientHttpRequestFactory uses java.net.http — no ext
```

How To Fix

Switch to JdkClientHttpRequestFactory (simplest).

Replace `OkHttp3ClientHttpRequestFactory` with `JdkClientHttpRequestFactory`. It uses the JDK's built-in HTTP client and requires no additional dependencies.

Switch to Apache HttpClient 5.

If you need connection pooling or advanced proxy configuration, use `HttpComponentsClientHttpRequestFactory` with Apache HttpClient 5. Add `org.apache.httpcomponents.client5:httpclient5` to your POM.

Scope Check

Search for `OkHttp3` and `com.squareup.okhttp3` across your codebase and build files. Check both production code and test configurations: OkHttp's `MockWebServer` is also affected.

Watch Out

- OkHttp interceptors for logging, retries, or authentication have no direct equivalent in the JDK client. Reimplement that logic with Spring's `ClientHttpRequestInterceptor`.
- OkHttp's `MockWebServer` is popular in tests. If you only use OkHttp for testing, consider switching to `MockRestServiceServer` or WireMock instead.

Verify

mvn dependency:tree shows no okhttp3 and build compiles

Further Info

OkHttp itself still works as a plain library; what vanished is Spring's factory integration and the managed BOM version.

Links

- Spring-Break Demo
- Spring Boot 4.0 Migration Guide

OpenSAML 4 Support Removed

Tier 1 - Won't Build Impact: L

Spring Security 7 dropped OpenSAML 4 support and requires OpenSAML 5. Code referencing OpenSaml4AuthenticationProvider and related classes fails to compile on Boot 4.0.

What You'll See

```
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] SamlConfig.java:[4,63] cannot find symbol
    symbol: class OpenSaml4AuthenticationProvider
[INFO] BUILD FAILURE
---
The build stops at compile. The application never starts.
```

What Changed

Spring Security 7 removed the OpenSAML 4 compatibility classes and requires OpenSAML 5. The SAML support classes (`OpenSaml4AuthenticationProvider`, `OpenSaml4AuthenticationRequestResolver`, etc.) were replaced with OpenSAML 5 equivalents. OpenSAML 5 changed package structures and removed the marshaller/unmarshaller split.

Why

OpenSAML 4 is no longer maintained. OpenSAML 5 brought significant API improvements, including a simplified object-provider model and better XML security defaults. Maintaining dual support across two major OpenSAML versions was unsustainable.

The Fix

Maven dependency
Before

```
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-api</artifactId>
    <version>4.3.2</version>
</dependency>
<dependency>
```

```
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-impl</artifactId>
    <version>4.3.2</version>
</dependency>
```

After

```
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-api</artifactId>
    <version>5.1.2</version>
</dependency>
<dependency>
    <groupId>org.opensaml</groupId>
    <artifactId>opensaml-saml-impl</artifactId>
    <version>5.1.2</version>
</dependency>
```

Java config — authentication provider

Before

```
import org.springframework.security.saml2.provider.service.a
    .OpenSaml4AuthenticationProvider;

OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
```

After

```
import org.springframework.security.saml2.provider.service.a
    .OpenSaml5AuthenticationProvider;

OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
```

Java config — custom response validator

Before

```
OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml4AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

After

```
OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml5AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

How To Fix

Upgrade OpenSAML dependencies to 5.x.

Update your OpenSAML dependencies to version 5.1.x. If you use Spring Boot's dependency management, the managed version will already be OpenSAML 5 in Spring Boot 4.0. Remove any explicit version overrides pinning to 4.x.

Rename Spring Security SAML classes.

Replace all references to `OpenSaml4*` classes with their `OpenSaml5*` equivalents. The main classes are `OpenSaml5AuthenticationProvider` and `OpenSaml5AuthenticationRequestResolver`.

Review custom SAML processing code.

If you directly use OpenSAML APIs (marshallers, unmarshallers, builders), review the OpenSAML 5 migration guide. The marshaller/unmarshaller split was removed and the object provider model changed.

Scope Check

Search for `opensaml` in your Maven/Gradle files, and for `OpenSaml4` in Java code. If your application uses SAML 2.0 login (`saml2Login()` in your security config), you are affected. If you only use OAuth 2.0 / OIDC, this card does not apply.

Watch Out

- OpenSAML 5 requires Java 17+ and has different XML processing requirements. Verify your XML security libraries are compatible.
- If you have custom `Saml2AuthenticationRequestContext` processing or assertion decryption, those APIs may have changed signatures in the OpenSAML 5 upgrade.
- The Shibboleth Maven repository is still required for OpenSAML 5 artifacts. Make sure your build still has that repository configured.

Verify

mvn clean compile — no "cannot find symbol" errors for OpenSaml4* classes; SAML login flow completes with OpenSAML 5 libraries

Further Info

The change comes from Spring Security 7.0, which Spring Boot 4.0 ships. Affects spring-boot-starter-security consumers using SAML 2.0 login.

Links

- Spring-Break Demo
- Spring Security 7 migration
- Spring Boot 4.0 Migration Guide

PropertyMapper.alwaysApplyingWhenNonNull() Removed

Tier 1 - Won't Build Impact: S

PropertyMapper.alwaysApplyingWhenNonNull() is removed because skipping null values is now the default. Just delete the call.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/PropertyMapperUsage.java:[8,47]
  error: cannot find symbol
    symbol:   method alwaysApplyingWhenNonNull()
    location: class PropertyMapper
```

What Changed

`PropertyMapper.alwaysApplyingWhenNonNull()` is gone. Its behaviour, skipping mappings when the source value is `null`, is now the default for `PropertyMapper`, so calling it was a no-op from 4.0 onward.

Why

The non-null default was the right behaviour for virtually all use cases. Requiring an explicit opt-in call was boilerplate with no practical reason to exist. Making it the default and removing the method simplifies the API.

The Fix

Remove the method call — the behaviour is now default

Before

```
PropertyMapper map = PropertyMapper.get().alwaysApplyingWhen
```

After

```
PropertyMapper map = PropertyMapper.get();
```

How To Fix

Delete the alwaysApplyingWhenNonNull() call.

Remove `.alwaysApplyingWhenNonNull()` from the chain. The mapper behaves identically on Boot 4.0 without it. If you explicitly want to map null values, use `.always()` on individual mappings.

Scope Check

Search for `alwaysApplyingWhenNonNull` across all Java/Kotlin sources. This typically appears in autoconfiguration classes or custom configuration adapters that mirror Spring Boot's internal style.

Watch Out

- If you were relying on the absence of `alwaysApplyingWhenNonNull()` to intentionally map null values through, that behaviour has also changed: nulls are now skipped by default. Add `.always()` on the specific mappings that need to pass nulls.

Verify

mvn compile — no cannot find symbol for alwaysApplyingWhenNonNull

Further Info

PropertyMapper is an internal Spring Boot utility, used mostly in auto-configuration classes to map configuration properties onto builder objects.

Links

- spring-break module: propertymapper-alwaysapplyingnonnull
- Spring Boot 4.0 Migration Guide

@PropertyMapping Relocated to test.context

Tier 1 - Won't Build Impact: S

@PropertyMapping moved from org.springframework.boot.test.autoconfigure.properties to org.springframework.boot.test.context. Old import doesn't compile.

What You'll See

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/CustomTestAnnotation.java:[3,62]
  error: package org.springframework.boot.test.autoconfigure.properties does not exi
```

What Changed

`@PropertyMapping` moved from `org.springframework.boot.test.autoconfigure.properties` to `org.springframework.boot.test.context`. Behaviour is unchanged.

Why

Spring Boot 4.0 consolidated test-context infrastructure into a more coherent package hierarchy. The separate `test.autoconfigure.properties` package was collapsed into `test.context`.

The Fix

Import on custom test annotation
Before

```
import org.springframework.boot.test.autoconfigure.propertie
```

After

```
import org.springframework.boot.test.context.PropertyMapping
```

How To Fix

Update the import.

Replace `org.springframework.boot.test.autoconfigure.properties.PropertyMapping` with `org.springframework.boot.test.context.PropertyMapping`. This typically appears

only in custom meta-annotations for test slices.

Scope Check

Search for `@PropertyMapping` and the old import path in your test source tree. Only affects teams that have written custom test slice annotations; most applications won't have this at all.

Watch Out

- If you have a shared test-utilities library that defines custom slices, the fix must be applied there rather than in the consuming projects. A single unfixed library breaks all consumers.

Verify

mvn test-compile — no package does not exist error for @PropertyMapping import

Further Info

@PropertyMapping is used when authoring custom test slice annotations. It binds annotation attributes to Spring Boot auto-configuration properties. Several other test-autoconfigure classes moved in the same cleanup.

Links

- spring-break module: propertymapping-relocated
- Spring Boot 4.0 Migration Guide

RestTemplate Auto-Configuration Removed

Tier 1 - Won't Build Impact: S

RestTemplateBuilder moved to the new spring-boot-restclient module. Imports from the old org.springframework.boot.web.client package no longer compile.

What You'll See

```
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] OrderClient.java:[3,42] package
  org.springframework.boot.web.client does not exist
[ERROR] OrderClient.java:[12,25] cannot find symbol
    symbol: class RestTemplateBuilder
[INFO] BUILD FAILURE
---
The build stops at compile. The application never starts.
```

What Changed

Spring Boot 4.0 removed the `org.springframework.boot.web.client` package. `RestTemplateBuilder` and its support classes relocated to the new `spring-boot-restclient` module, which covers both `RestClient` and `RestTemplate`.

Why

`RestTemplate` has been in maintenance mode since Spring 5, and Spring Framework 6.1 introduced `RestClient` as its fluent replacement. Removing the auto-configuration pushes migration to `RestClient` or `WebClient`.

The Fix

Injecting the HTTP client — service class
Before

```java
@Service
public class OrderClient {
    private final RestTemplate restTemplate;

    public OrderClient(RestTemplateBuilder builder) {
        this.restTemplate = builder
            .rootUri("https://api.example.com")
```

```
            .build();
    }

    public Order getOrder(Long id) {
        return restTemplate.getForObject(
            "/orders/{id}", Order.class, id);
    }
}
```

After

```
@Service
public class OrderClient {
    private final RestClient restClient;

    public OrderClient(RestClient.Builder builder) {
        this.restClient = builder
            .baseUrl("https://api.example.com")
            .build();
    }

    public Order getOrder(Long id) {
        return restClient.get()
            .uri("/orders/{id}", id)
            .retrieve()
            .body(Order.class);
    }
}
```

Test — using @RestClientTest

Before

```
@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
    private OrderClient client;
```

After

```
@RestClientTest(OrderClient.class)
class OrderClientTest {
    @Autowired
    private MockRestServiceServer server;
    @Autowired
```

```
    private OrderClient client;
    // Works the same — @RestClientTest supports RestClient
```

How To Fix

Migrate to RestClient (recommended).

Replace `RestTemplate` usage with `RestClient`. Inject `RestClient.Builder` (auto-configured by Spring Boot 4) instead of `RestTemplateBuilder`. The API is fluent and supports the same customizers.

Update the RestTemplateBuilder import.

If migration is too large to do now, keep `RestTemplate`: add the `spring-boot-restclient` module (or its starter) to your build and update the `RestTemplateBuilder` import. This restores the old behaviour while you plan the migration.

Use WebClient for reactive code.

Reactive applications should use `WebClient`, which remains auto-configured in Spring Boot 4.0.

Scope Check

Search for `RestTemplate`, `RestTemplateBuilder`, and `RestTemplateCustomizer` across your codebase. Every import from the old package is a compile failure. Also check test classes that use `TestRestTemplate`: its auto-configuration may also be affected.

Watch Out

- `TestRestTemplate` is a separate concern. Check whether it is still auto-configured in your test slices or if you need to provide it manually.
- Third-party libraries that inject `RestTemplateBuilder` (e.g., Spring Cloud OpenFeign, some tracing libraries) may also break. Check transitive dependencies.
- The `RestClient` API is different from `RestTemplate`. Methods like `getForObject()` and `postForEntity()` become chained `.get().uri().retrieve().body()` calls. Budget time for the API rewrite as well as the bean swap.

Verify

mvn clean compile — no "package org.springframework.boot.web.client does not exist" errors; HTTP calls succeed

Further Info

Part of Spring Boot 4.0's wider modularisation of HTTP client support. See also: okhttp3-removed.

Links

- Spring-Break Demo
- Testing changes in Spring Boot 4.0
- Spring Boot 4.0 Migration Guide

Security DSL Rewrite

Tier 1 - Won't Build Impact: L ⚙ OpenRewrite

Spring Security removed authorizeRequests(), antMatchers(), and the .and() chaining method. The HTTP security DSL needs a full rewrite.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/SecurityConfig.java:[22,14]
  error: cannot find symbol
    symbol:   method authorizeRequests()
    location: variable http of type HttpSecurity
[ERROR] /src/main/java/com/example/SecurityConfig.java:[23,18]
  error: cannot find symbol
    symbol:   method antMatchers(java.lang.String)
[ERROR] /src/main/java/com/example/SecurityConfig.java:[28,10]
  error: cannot find symbol
    symbol:   method and()
```

What Changed

Spring Security 7.0 (shipped with Boot 4.0) removed the deprecated `authorizeRequests()` method: use `authorizeHttpRequests()` instead. `antMatchers()`, `mvcMatchers()`, and `regexMatchers()` are replaced by `requestMatchers()`. The `.and()` chaining method is gone; the lambda DSL takes its place.

Why

The lambda DSL is more readable and avoids a class of configuration bugs where `.and()` chains silently reset the security context. The new `requestMatchers()` API auto-detects whether to use Ant or MVC matching based on your classpath.

The Fix

Security filter chain — before
Before

```
http
    .authorizeRequests()
        .antMatchers("/public/**").permitAll()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
```

```
            .and()
        .formLogin()
            .loginPage("/login")
            .and()
        .logout()
            .logoutSuccessUrl("/");
```

After

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/public/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin(form -> form
        .loginPage("/login")
    )
    .logout(logout -> logout
        .logoutSuccessUrl("/")
    );
```

CSRF configuration
Before

```
http.csrf().disable();
```

After

```
http.csrf(csrf -> csrf.disable());
```

How To Fix

Rewrite to lambda DSL.

Replace every `.and()` chain with a lambda-based configuration block. Each security concern (authorization, form login, logout, CSRF) gets its own lambda. See the Spring Security 7 migration guide.

Replace matchers.

Change `antMatchers()` and `mvcMatchers()` to `requestMatchers()`. The new method auto-selects the matching strategy. `regexMatchers()` becomes `requestMatchers(new RegexRequestMatcher(...))`.

Scope Check

Search for `authorizeRequests()`, `antMatchers()`, `mvcMatchers()`, `.and()` in security config classes, and `.csrf().disable()`. Every `SecurityFilterChain` bean needs reviewing.

Watch Out

- The `.and()` removal is the most disruptive part. A long chain with 5+ `.and()` calls means restructuring the whole method. Review the result carefully: it's easy to nest a lambda inside the wrong block.
- `requestMatchers()` auto-detects MVC vs Ant matching. If you have Spring MVC on the classpath, it uses MVC matching (which is stricter). If this changes behaviour, use `AntPathRequestMatcher` explicitly.

Verify

App starts and security filter chain initialises without errors

Further Info

Driven by Spring Security 7.0. The old DSL was deprecated in Security 5.7/6.0; the lambda DSL has been the recommended replacement since Security 5.2. Affects every spring-boot-starter-security consumer. See also: websecurity-adapter-removed, auth-default-deny.

Links

- spring-break module: security-removed-apis
- Spring Security 7 migration guide

SimpDestinationMessageMatcher Removed

Tier 1 - Won't Build Impact: M

SimpDestinationMessageMatcher is gone in Spring Security 7.0. WebSocket security now uses the AuthorizationManager pattern.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/SimpDestUsage.java:[3,66]
  error: package org.springframework.security.messaging.util.matcher
  does not contain SimpDestinationMessageMatcher
```

What Changed

`org.springframework.security.messaging.util.matcher.SimpDestinationMessageMatcher` has been removed. Spring Security 7.0 replaces the old `AbstractSecurityWebSocketMessageBrokerConfigurer` / matcher-based approach with `MessageMatcherDelegatingAuthorizationManager`.

Why

The old configurer and matcher classes were deprecated in Security 6.x. 7.0 removed them, completing the move to AuthorizationManager.

The Fix

Old — matcher-based WebSocket security config
Before

```
import org.springframework.security.messaging.util.matcher.S
// ...
new SimpDestinationMessageMatcher("/topic/**");
```

New — AuthorizationManager-based config
Added

```
import org.springframework.security.messaging.access.interce
// ...
MessageMatcherDelegatingAuthorizationManager.builder()
    .simpDestMatchers("/topic/**").authenticated()
```

```
        .anyMessage().denyAll()
        .build();
```

How To Fix

Migrate to MessageMatcherDelegatingAuthorizationManager.

Replace any use of `SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer` with a `MessageMatcherDelegatingAuthorizationManager` bean registered on your message broker. See the Spring Security 7 messaging migration guide for the full configuration pattern.

Scope Check

Search for `SimpDestinationMessageMatcher` and `AbstractSecurityWebSocketMessageBrokerConfigurer`. Both are removed. Only applications using STOMP over WebSocket with Spring Security are affected.

Watch Out

- The new AuthorizationManager must be wired into the message broker configuration explicitly. There is no auto-migration path: the configuration structure changes substantially.

Verify

mvn compile — no cannot find symbol for SimpDestinationMessageMatcher

Further Info

Part of Spring Security 7.0's WebSocket security overhaul: the same AuthorizationManager pattern now applies across HTTP, method, and message security.

Links

- spring-break module: simpdest-message-matcher-removed
- Spring Security 7 messaging migration

Spring AMQP RabbitRetryTemplateCustomizer Removed

Tier 1 - Won't Build Impact: S

Boot 4.0 removed `RabbitRetryTemplateCustomizer`. Every bean implementing it fails to compile; publisher and consumer retry now have separate customizer interfaces.

What You'll See

```
@Bean
public RabbitRetryTemplateCustomizer retryCustomizer() {
    return (target, retryTemplate) -> { ... };
}

// Boot 4.0 compile error:
error: cannot find symbol
  symbol: class RabbitRetryTemplateCustomizer
```

What Changed

Spring AMQP 4.0 dropped its dependency on Spring Retry and now uses Spring Framework's retry abstraction. `RabbitRetryTemplateCustomizer` is replaced by `RabbitTemplateRetrySettingsCustomizer` (publisher-side) and `RabbitListenerRetrySettingsCustomizer` (consumer-side).

Why

Spring Boot 4.0 dropped Spring Retry across the board in favour of Spring Framework's native retry support. Spring AMQP followed the same path.

The Fix

Publisher-side retry customizer
Before

```
import org.springframework.amqp.rabbit.retry.RabbitRetryTemp

@Bean
public RabbitRetryTemplateCustomizer retryCustomizer() {
    return (target, retryTemplate) -> {
        retryTemplate.setRetryPolicy(...);
    };
}
```

After

```
import org.springframework.boot.autoconfigure.amqp.RabbitTem

@Bean
public RabbitTemplateRetrySettingsCustomizer retryCustomizer
    return settings -> {
        settings.setMaxAttempts(3);
    };
}
```

Consumer-side retry customizer
Before

```
// Previously both publisher and consumer shared RabbitRetry
```

After

```
import org.springframework.boot.autoconfigure.amqp.RabbitLis

@Bean
public RabbitListenerRetrySettingsCustomizer listenerRetryCu
    return settings -> {
        settings.setMaxAttempts(5);
    };
}
```

How To Fix

Replace with the split customizer interfaces.

Replace each `RabbitRetryTemplateCustomizer` bean with the publisher or consumer variant, whichever side it configured. The new interfaces set retry settings directly rather than manipulating a `RetryTemplate`.

Scope Check

Grep for `RabbitRetryTemplateCustomizer` across your source tree. Also check `import org.springframework.amqp.rabbit.retry` imports.

Watch Out

- The new customizer interfaces accept a settings object rather than a `RetryTemplate`. Retry policy logic expressed as `RetryTemplate` configuration must be re-expressed

using the properties on the settings object.

Verify

RabbitTemplate and RabbitListener retry settings are configured via the new customizer interfaces without compile errors

Further Info

One of several retry-related breaks in Boot 4.0. See also: spring-retry-removed, retry-semantics-change.

Links

- Spring Boot 4.0 Migration Guide

spring-jcl Removed

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

Spring Framework replaced its spring-jcl logging bridge with a direct dependency on Commons Logging 1.3.0. Manual spring-jcl exclusions break.

What You'll See

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies:
  The following artifacts could not be resolved:
    org.springframework:spring-jcl:jar:7.0.0 (not found in central)
```

What Changed

The `org.springframework:spring-jcl` module was removed in Spring Framework 7.0. Spring now depends directly on `commons-logging:commons-logging:1.3.0`, which gained native SLF4J and `java.util.logging` bridge support. Any POM that references `spring-jcl`, or excludes `commons-logging` in its favour, breaks the build.

Why

Commons Logging 1.3 added the same SLF4J bridge that spring-jcl provided, making Spring's forked module redundant.

The Fix

pom.xml — remove spring-jcl exclusion/inclusion
Before

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <exclusions>
        <exclusion>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <!-- Commons Logging 1.3.0 is pulled in transitively - n
</dependency>
```

pom.xml — remove explicit spring-jcl dependency
Before

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jcl</artifactId>
</dependency>
```

After

```
<!-- spring-jcl no longer exists. Commons Logging 1.3.0 brid
```

How To Fix

Remove spring-jcl references.

Delete any explicit `spring-jcl` dependency declarations and any `commons-logging` exclusions on Spring modules. Commons Logging 1.3.0 bridges to SLF4J out of the box.

Remove jcl-over-slf4j.

If you had `org.slf4j:jcl-over-slf4j` on the classpath, remove it. Commons Logging 1.3.0 provides the same bridge natively. Having both will cause a classpath conflict.

Scope Check

Search for `spring-jcl`, `jcl-over-slf4j`, and exclusions of `commons-logging` in all POM and Gradle files. All three are leftovers of the old logging bridge dance.

Watch Out

- The classic "commons-logging exclusion + jcl-over-slf4j" pattern every Spring project used for a decade is now harmful: it excludes the Commons Logging 1.3.0 that Spring needs, then substitutes an older SLF4J bridge that may conflict. Remove both.
- If you use a BOM or parent POM that applies global exclusions on commons-logging (common in large enterprises), that exclusion now breaks Spring. Update the corporate parent POM.

Verify

mvn compile — no spring-jcl dependency errors

Further Info

A rare migration win: a decade of logging-bridge boilerplate can come out of your POMs.

Links

- Spring-Break Demo
- Spring Boot 4.0 Migration Guide

spring-retry Removed from BOM

Tier 1 - Won't Build Impact: S

Spring Boot 4.0 removed spring-retry from its managed dependency BOM. Declare an explicit version or the build fails.

What You'll See

```
$ mvn compile
[ERROR] 'dependencies.dependency.version' for org.springframework.retry:spring-retry
  is missing. @ line 42, column 17
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies: Failed to collect dependencies at
  org.springframework.retry:spring-retry:jar:RELEASE (no version)
```

What Changed

`org.springframework.retry:spring-retry` is no longer version-managed by the Spring Boot BOM. Any declaration without an explicit `<version>` tag fails because Maven/Gradle can't resolve it.

Why

Spring Retry's release cadence diverged from Spring Boot's. Managing it in the BOM implied a level of compatibility testing that wasn't happening. Removing it from the BOM lets teams pin the version they actually test against.

The Fix

pom.xml — add explicit version

Before

```
<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.retry</groupId>
    <artifactId>spring-retry</artifactId>
```

```
    <version>2.0.11</version>
</dependency>
```

build.gradle — add explicit version
Before

```
implementation 'org.springframework.retry:spring-retry'
```

After

```
implementation 'org.springframework.retry:spring-retry:2.0.1
```

How To Fix

Add an explicit version.

Add `<version>2.0.11</version>` (or the latest compatible release) to your spring-retry dependency declaration. Check Maven Central for the latest version.

Import Spring Retry's own BOM.

If multiple modules use spring-retry, add its BOM to your `<dependencyManagement>` section so versions are managed in one place.

Scope Check

Search for `spring-retry` in POM and Gradle files. Any declaration without an explicit version will fail. Also check for `@Retryable` and `@EnableRetry` annotations: these confirm the library is actually in use.

Watch Out

- If you use `spring-cloud-starter-circuitbreaker-resilience4j`, it may have pulled in spring-retry transitively. With the BOM removal, the transitive version is no longer aligned. Pin it explicitly to avoid classpath surprises.
- Spring Retry 2.x requires Spring Framework 6.0+. Since Boot 4.0 ships Framework 7.0, use Retry 2.0.x: older 1.x versions are incompatible.

Verify

mvn dependency:tree shows aligned Spring Retry version and retry works

Further Info

The library itself is unchanged and still maintained; only Boot's version management went away. See also: retry-semantics, retryable-transaction-order.

Links

- spring-break module: spring-retry-removed
- Spring Boot 4.0 Migration Guide

Spring Security Access API Moved to spring-security-access

Tier 1 - Won't Build Impact: M

AccessDecisionManager and related legacy authorisation classes moved to a separate spring-security-access module. Add it or migrate to AuthorizationManager.

What You'll See

```
$ mvn compile
[ERROR] /src/main/java/com/example/AccessApiUsage.java:[3,48]
  error: package org.springframework.security.access does not exist
```

What Changed

`AccessDecisionManager`, `AccessDecisionVoter`, `@EnableGlobalMethodSecurity`, and related legacy authorisation classes have been moved from `spring-security-core` to a new standalone `spring-security-access` artifact.

Why

Spring Security 7.0 makes `AuthorizationManager` the sole supported authorisation API. The legacy Access API survives for migration, isolated in its own module so finished migrations don't carry the dead weight.

The Fix

pom.xml — add legacy module if needed
Added

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-access</artifactId>
</dependency>
```

Preferred: migrate to AuthorizationManager
Before

```
import org.springframework.security.access.AccessDecisionMan
// implements AccessDecisionManager
```

After

```
import org.springframework.security.authorization.Authorizat
// implements AuthorizationManager<RequestAuthorizationConte
```

How To Fix

Add spring-security-access (short-term fix).

Add `spring-security-access` as an explicit dependency. This restores compilation without any logic changes and buys time to do the full migration.

Migrate to AuthorizationManager (recommended).

Replace `AccessDecisionManager` / `AccessDecisionVoter` implementations with `AuthorizationManager`. Replace `@EnableGlobalMethodSecurity` with `@EnableMethodSecurity`. The Spring Security migration guide has step-by-step instructions.

Scope Check

Search for `AccessDecisionManager`, `AccessDecisionVoter`, `@EnableGlobalMethodSecurity`, and `org.springframework.security.access` imports across all sources.

Watch Out

- `@EnableGlobalMethodSecurity` is also in the legacy module. If you use method security, you need either the legacy module or to migrate to `@EnableMethodSecurity` before removing it.

Verify

mvn compile — no cannot find symbol for AccessDecisionManager or related classes

Further Info

Driven by Spring Security 7.0; the spring.io announcement in the footer covers the full background.

Links

- spring-break module: spring-security-access-relocated

- Spring Security access API moves

Test-Slice Annotation Starters Relocated

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

Spring Boot 4.0 relocated the test-slice annotations. The old packages (e.g. org.springframework.boot.test.autoconfigure.orm.jpa) no longer exist, so imports of @DataJpaTest, @WebMvcTest, and friends fail to compile.

What You'll See

```
$ mvn clean test
[ERROR] COMPILATION ERROR :
[ERROR] UserRepositoryTest.java:[5,50] package
  org.springframework.boot.test.autoconfigure.orm.jpa does not exist
[ERROR] UserRepositoryTest.java:[12,2] cannot find symbol
    symbol: class DataJpaTest
[INFO] BUILD FAILURE
---
The build stops at test-compile. No tests run.
```

What Changed

Spring Boot 4.0 reorganised the test auto-configuration packages. The old test-slice packages, such as `org.springframework.boot.test.autoconfigure.orm.jpa`, no longer exist, so existing imports of `@DataJpaTest`, `@WebMvcTest`, and the other slice annotations fail to compile. The annotations now live in the new per-technology test modules.

Why

The test auto-configuration was restructured to support the new bean override mechanism and to align the package structure with Spring Framework 7's test context changes.

The Fix

Custom test-slice annotation
Before

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
```

```
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
}
```

After

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@BootstrapWith(SpringBootTestContextBootstrapper.class)
@OverrideAutoConfiguration(enabled = false)
@TypeExcludeFilters(CustomSliceTypeExcludeFilter.class)
@AutoConfigureCache
@AutoConfigureCustomService
@ImportAutoConfiguration
public @interface CustomServiceTest {
    // Annotation structure unchanged, but verify your
    // TypeExcludeFilter extends the correct base class
    // and spring.factories / AutoConfiguration.imports are
}
```

META-INF/spring.factories — test slice registration
Before

```
org.springframework.boot.test.autoconfigure.web.servlet.Auto
  com.example.test.CustomWebMvcAutoConfiguration
```

After

```
# Moved to META-INF/spring/AutoConfiguration.imports
# or updated key in spring.factories
org.springframework.boot.autoconfigure.AutoConfiguration.imp
  com.example.test.CustomWebMvcAutoConfiguration
```

How To Fix

Update the test-slice imports.

Every test importing a slice annotation from the old `org.springframework.boot.test.autoconfigure` packages needs the import updated to the annotation's new home in the per-technology test modules. The Spring Boot 4.0 OpenRewrite recipe handles this across the whole test codebase; your IDE can re-resolve individual imports.

Migrate spring.factories to AutoConfiguration.imports.

Move test auto-configuration entries from `META-INF/spring.factories` to `META-INF/spring/AutoConfiguration.imports`. This was deprecated in Boot 3.x and is now enforced.

Update custom test-slice annotations.

If you wrote custom test-slice annotations (like `@CustomServiceTest`), verify that the `TypeExcludeFilter`, bootstrapper, and auto-configuration imports still resolve. Update fully qualified class references to match the new package locations, then run the full suite to confirm each slice still loads a thin context.

Scope Check

Search test sources for imports from `org.springframework.boot.test.autoconfigure`: every one of them fails to compile on 4.0. Also search for custom annotations that use `@TypeExcludeFilters`, `@OverrideAutoConfiguration`, or `@ImportAutoConfiguration`, and for `spring.factories` entries under test source sets.

Watch Out

- Once the imports compile again, check what context each slice loads. A misconfigured slice can silently load a full application context instead of a thin one, making tests pass but run much slower and test less precisely.
- Libraries that provide custom test slices (Spring Cloud, Spring Modulith) need to be updated too. Check their Spring Boot 4.0 compatibility versions.
- If your test suite uses `spring.factories` for test auto-configuration and you haven't migrated to `AutoConfiguration.imports`, those configurations are ignored.

Verify

mvn clean test — test sources compile with the new imports and all @DataJpaTest/@WebMvcTest slices load correctly

Further Info

Part of Spring Boot 4.0's modularisation: test auto-configuration moved out of the monolithic spring-boot-test-autoconfigure module into per-technology test modules (spring-boot-data-jpa-test, spring-boot-webmvc-test, and so on), and the annotation packages moved with it.

Links

- Spring-Break Demo
- Testing changes in Spring Boot 4.0

- Spring Boot 4.0 Migration Guide

Testcontainers 2.0 Package Relocation

Tier 1 - Won't Build Impact: S ⚙ OpenRewrite

Testcontainers 2.0 moved database and middleware containers into module-specific sub-packages (e.g. org.testcontainers.postgresql). The Maven group ID stays org.testcontainers.

What You'll See

```
$ mvn compile
[ERROR] /src/test/java/com/example/IntegrationTest.java:[4,52]
  error: cannot find symbol
    symbol:   class PostgreSQLContainer
    location: package org.testcontainers.containers
```

What Changed

Testcontainers 2.0 resolved JPMS split-package problems by moving specialised containers into module-specific packages. `PostgreSQLContainer` moved from `org.testcontainers.containers` to `org.testcontainers.postgresql`. Similar moves apply to all database and middleware modules (Kafka, Redis, RabbitMQ, etc.). `GenericContainer` remains in `org.testcontainers.containers`. The Maven group ID (`org.testcontainers`) is unchanged.

Why

JPMS forbids multiple JARs contributing to the same package (split packages). In Testcontainers 1.x every module dumped its containers into `org.testcontainers.containers`, making modular JARs impossible. The 2.0 sub-package split is a prerequisite for Java 17+ module support.

The Fix

> pom.xml — BOM: group ID stays org.testcontainers; remove explicit version if using Boot parent
> Before
>
> ```
> <dependency>
>     <groupId>org.testcontainers</groupId>
>     <artifactId>testcontainers-bom</artifactId>
>     <version>1.19.8</version>
>     <type>pom</type>
> ```

```
    <scope>import</scope>
</dependency>
```

After

```
<!-- Managed by Spring Boot 4.0 parent POM - remove explicit
```

Java imports — specialised containers move to module sub-packages
Before

```
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.containers.KafkaContainer;
```

After

```
import org.testcontainers.postgresql.PostgreSQLContainer;
import org.testcontainers.kafka.KafkaContainer;
```

GenericContainer stays put — no change needed
Before

```
import org.testcontainers.containers.GenericContainer;
```

After

```
import org.testcontainers.containers.GenericContainer;  // u
```

How To Fix

Update Java imports for specialised containers.

For each database or middleware container, change the import from `org.testcontainers.containers.XxxContainer` to the module-specific sub-package, e.g. `org.testcontainers.postgresql.PostgreSQLContainer`, `org.testcontainers.kafka.KafkaContainer`, `org.testcontainers.redis.RedisContainer`. Check the Testcontainers 2.0 migration guide for the full mapping.

Remove explicit BOM version if using Spring Boot parent.

Spring Boot 4.0's parent POM manages the Testcontainers 2.0 version. If you previously imported the Testcontainers BOM explicitly with a 1.x version, remove the version override to avoid conflicts.

Scope Check

Search for `org.testcontainers.containers` in Java/Kotlin test sources. Any import of a specialised container (PostgreSQL, MySQL, Kafka, Redis, etc.) from that package needs updating to its new module sub-package. `GenericContainer` does not need changing.

Watch Out

- The Maven group ID is still `org.testcontainers`; do not change it to `com.testcontainers`. Only the Java package paths inside the JARs changed.
- Spring Boot's `@ServiceConnection` annotation resolves container types by class. If you reference the old `org.testcontainers.containers.PostgreSQLContainer` import in a `@ServiceConnection` field, updating the import is sufficient; no annotation change needed.
- If you use Testcontainers in a shared test library, update that module first. A mix of 1.x and 2.x imports on the classpath will cause `ClassNotFoundException` at runtime.

Verify

mvn test — Testcontainers start and tests pass

Further Info

Driven by Testcontainers 2.0, the version Spring Boot 4.0 now manages.

Links

- spring-break module: testcontainers-class-relocation
- Testcontainers 2.0 migration guide

TestRestTemplate Package Relocated

Tier 1 - Won't Build Impact: S

TestRestTemplate moved to org.springframework.boot.resttestclient. Every test that imports the old package fails to compile.

What You'll See

```
$ mvn test-compile
[ERROR] /src/test/java/com/example/MyControllerTest.java:[3,56]
  error: package org.springframework.boot.test.web.client does not exist
[ERROR] /src/test/java/com/example/MyControllerTest.java:[15,5]
  error: cannot find symbol
    symbol: class TestRestTemplate
```

What Changed

`TestRestTemplate` was removed from `org.springframework.boot.test.web.client` and relocated to `org.springframework.boot.resttestclient`.

Why

Spring Boot 4.0 modularised its testing support. Separating the REST test clients into their own module shrinks the core test-autoconfigure footprint.

The Fix

Test class import

Before

```
import org.springframework.boot.test.web.client.TestRestTemp
```

After

```
import org.springframework.boot.resttestclient.TestRestTempl
```

How To Fix

Update the import.

Replace `org.springframework.boot.test.web.client.TestRestTemplate` with `org.springframework.boot.resttestclient.TestRestTemplate`. The class itself is functionally equivalent.

Consider RestTestClient.

The new `RestTestClient` provides a more modern, fluent API for REST testing and is the preferred approach in Boot 4.0. If you are refactoring tests anyway, this is a good time to migrate.

Scope Check

Search all test sources for `org.springframework.boot.test.web.client.TestRestTemplate` and for `@Autowired TestRestTemplate` or `@Autowired private TestRestTemplate`. Every hit needs the import updated.

Watch Out

- Custom configuration classes that create a `TestRestTemplate` bean (for authentication headers, say) need the import updated too, alongside the test classes that inject it.

Verify

mvn test-compile — no package does not exist for org.springframework.boot.test.web.client

Further Info

Part of Spring Boot 4.0's testing overhaul: the class now ships in the spring-boot-test-rest-client module rather than spring-boot-test-autoconfigure. See also: springboottest-no-mockmvc.

Links

- spring-break module: testrest-template-removed
- Spring Boot 4.0 Migration Guide

Tomcat WAR Deployment Requires New Runtime Starter

Tier 1 - Won't Build Impact: S

Boot 4.0 adds `spring-boot-starter-tomcat-runtime` for WAR deployment. The old provided-scope `spring-boot-starter-tomcat` pattern may conflict with the external container's classpath.

What You'll See

```
// Deploying WAR to external Tomcat with Boot 3.5 dependency pattern:
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
    <scope>provided</scope>
</dependency>

// Boot 4.0: may result in:
java.lang.ClassNotFoundException: jakarta.servlet.FilterRegistration
// Or: version conflicts between embedded and container Tomcat classes
```

What Changed

A new `spring-boot-starter-tomcat-runtime` artifact covers WAR deployment. It provides the runtime bridge classes needed to run a Boot application inside an external Tomcat, without the full embedded Tomcat server dependencies. `spring-boot-starter-tomcat` is now exclusively for embedded use.

Why

Embedded Tomcat and external container are distinct deployment models with different runtime dependencies. The old trick of marking `spring-boot-starter-tomcat` as `provided` blurred that line and was error-prone.

The Fix

pom.xml — WAR deployment dependency update

Before

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat</artifactId>
```

```
    <scope>provided</scope>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat-runtime</artifact
    <scope>provided</scope>
</dependency>
```

How To Fix

Replace spring-boot-starter-tomcat (provided) with spring-boot-starter-tomcat-runtime.

In your WAR project's `pom.xml`, replace the provided-scope `spring-boot-starter-tomcat` dependency with `spring-boot-starter-tomcat-runtime`. Keep the `provided` scope: the external container supplies Tomcat at runtime.

Scope Check

Search `pom.xml` for `spring-boot-starter-tomcat` with `scope=provided`. Also check `SpringBootServletInitializer` subclasses: any WAR project using that class is affected.

Watch Out

- If your project also runs as an executable JAR (dual deployment mode), keep `spring-boot-starter-tomcat` for the embedded server but add `spring-boot-starter-tomcat-runtime` as provided.

Verify

WAR file deploys to external Tomcat without ClassNotFoundException or dependency conflicts after swapping to spring-boot-starter-tomcat-runtime

Further Info

The Boot 3.5 pattern, `spring-boot-starter-tomcat` at `provided` scope, can still work on 4.0. Mixing it with the new split risks classpath conflicts or `ClassNotFoundException` at deployment time.

Links

- Spring Boot 4.0 Migration Guide

Undertow Embedded Server Removed

Tier 1 - Won't Build Impact: M

Spring Boot 4.0 dropped Undertow as an embedded server option. Projects depending on spring-boot-starter-undertow won't resolve.

What You'll See

```
$ mvn compile
[ERROR] Failed to execute goal on project my-service:
  Could not resolve dependencies for project com.example:my-service:jar:1.0.0:
  The following artifacts could not be resolved:
    org.springframework.boot:spring-boot-starter-undertow:jar:4.0.0 (not found)
```

What Changed

The `spring-boot-starter-undertow` module was removed from Spring Boot 4.0. Undertow is no longer a supported embedded servlet container. The supported options are Tomcat (default), Jetty, and Netty (for reactive).

Why

Undertow's development slowed and its Jakarta EE 11 support lagged behind Tomcat and Jetty. Maintaining a third servlet container integration stopped being worth the support burden.

The Fix

pom.xml — remove Undertow starter
Before

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifact
        </exclusion>
    </exclusions>
</dependency>
<dependency>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-undertow</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- Uses Tomcat by default. For Jetty, add spring-boot-star
     and exclude spring-boot-starter-tomcat -->
```

application.yml — Undertow-specific config
Before

```
server:
  undertow:
    io-threads: 8
    worker-threads: 64
    buffer-size: 1024
```

After

```
server:
  tomcat:
    threads:
      max: 200
      min-spare: 10
```

How To Fix

Switch to Tomcat (default).

Remove the `spring-boot-starter-undertow` dependency and the Tomcat exclusion. Tomcat is the default: `spring-boot-starter-web` alone is enough.

Switch to Jetty.

If you need a non-Tomcat server, use `spring-boot-starter-jetty` instead. Exclude the Tomcat starter and add the Jetty starter.

Scope Check

Search for `spring-boot-starter-undertow` in POM and Gradle files. Also search for `server.undertow` in application properties/YAML files: those properties won't cause a build failure but will be silently ignored.

Watch Out

- Undertow-specific configuration properties under `server.undertow.*` have no Tomcat or Jetty equivalent. Review your `application.yml` and translate settings like `io-threads`, `worker-threads`, and `buffer-size` to the new server's configuration model.
- If you chose Undertow for its non-blocking I/O on the servlet side, consider switching to Spring WebFlux with Netty instead of forcing the same architecture onto Tomcat.

Verify

App starts on Tomcat/Jetty with no Undertow class errors

Further Info

One of the few 4.0 breaks with no like-for-like replacement: you must change servers.

Links

- spring-break module: undertow-removed
- Spring Boot 4.0 Migration Guide

webjars-locator-core Removed from BOM

Tier 1 - Won't Build Impact: S

Spring Boot 4.0 removes `webjars-locator-core` from the BOM. Version-agnostic WebJar URLs (`/webjars/jquery/jquery.min.js`) stop resolving. Switch to `webjars-locator-lite`.

What You'll See

```
// pom.xml — dependency with no version (relies on BOM)
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>webjars-locator-core</artifactId>
</dependency>

// Boot 3.5: BOM provides version 0.59 — resolves fine.

// Boot 4.0: BOM no longer manages webjars-locator-core.
[ERROR] 'dependencies.dependency.version' for
  org.webjars:webjars-locator-core:jar is missing.
[ERROR] BUILD FAILURE

// Or if version is pinned in pom.xml — compiles but WebJarAssetLocator
// is no longer registered by Boot's auto-configuration:
java.lang.ClassNotFoundException: org.webjars.WebJarAssetLocator
```

What Changed

`org.webjars:webjars-locator-core` was the original WebJar asset locator, using full classpath scanning to find versioned WebJar paths. Spring Boot 4.0 removes it from the BOM and its auto-configuration wiring, replacing it with `org.webjars:webjars-locator-lite`, which uses a pre-generated manifest file (`webjars-requirejs.js` or `META-INF/resources/webjars/`) to locate assets without scanning.

Why

The classpath scanning approach in `webjars-locator-core` added startup overhead proportional to the number of JARs on the classpath and was incompatible with GraalVM native image compilation. The lite variant eliminates both problems: manifest-based lookup is O(1) and works in native images.

The Fix

pom.xml — replace locator dependency

Before

```
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>webjars-locator-core</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>webjars-locator-lite</artifactId>
</dependency>
```

Code using WebJarAssetLocator directly
Before

```
import org.webjars.WebJarAssetLocator;
WebJarAssetLocator locator = new WebJarAssetLocator();
String path = locator.getFullPath("jquery", "jquery.min.js")
```

After

```
import org.webjars.lite.WebJarAssetLocator;
WebJarAssetLocator locator = new WebJarAssetLocator();
String path = locator.getFullPath("jquery", "jquery.min.js")
```

How To Fix

Replace webjars-locator-core with webjars-locator-lite.

Update `pom.xml` or `build.gradle` to declare `org.webjars:webjars-locator-lite`. The API is compatible for common use cases: Spring MVC's WebJar resource handler works with both.

Update direct WebJarAssetLocator imports.

If your code imports `org.webjars.WebJarAssetLocator` directly, change the import to the `org.webjars.lite` package. The method signatures for `getFullPath()` are compatible.

Scope Check

Search for `webjars-locator-core` in `pom.xml` and `build.gradle` files. Also grep your Java source for `import org.webjars.WebJarAssetLocator`. Check Thymeleaf templates and JSPs for version-agnostic WebJar URLs

(`th:href="@{/webjars/bootstrap/css/bootstrap.min.css}"`): these require a locator to resolve and will 404 if no locator is present.

Watch Out

- If you pin an explicit version of `webjars-locator-core` in your pom, the build succeeds, but Boot 4.0 no longer wires the auto-configuration. WebJar URLs that worked without a version segment silently return 404 instead of failing at startup.
- `webjars-locator-lite` requires that WebJar dependencies include a `webjars-requirejs.js` manifest. Older WebJar artifacts may not include this file. Check the WebJar version you are using is compatible with the lite locator.

Verify

WebJar assets resolve correctly via the webjars-locator-lite replacement

Further Info

Both locators come from the WebJars project itself; Spring Boot switched which one it manages and wires.

Links

- spring-break module: webjars-locator-core-removed
- Spring Boot 4.0 Migration Guide
- webjars-locator-lite on GitHub

Tier 2

Tier 2 - Won't Start

23 cards

1. Actuator Endpoint @Nullable: org.springframework.lang Replaced by JSpecify
2. AspectJ Weaving Required for @Observed
3. Spring Batch Now Uses In-Memory Job Repository by Default
4. Spring Batch 6 Schema Sequence Rename
5. Controller & View Spans Disabled by Default
6. Deprecated RestTemplateBuilder APIs Removed
7. CascadeType.SAVE_UPDATE Removed in Hibernate 7
8. Version-Specific Hibernate Dialects Removed
9. Hibernate Untyped Join Query Rejected at Runtime
10. write-dates-as-timestamps Property Breaks Startup
11. JacksonException No Longer Extends IOException
12. javax.annotation Removed
13. javax.inject Removed
14. MockitoTestExecutionListener Removed
15. Modular Auto-Configuration
16. OAuth 2.0 Password Grant Completely Removed
17. OTLP/HTTP Now Primary Export Protocol
18. PKCE Mandatory for Confidential OAuth Clients
19. Spring Pulsar Reactive Auto-Configuration Removed
20. Spring Boot Spock Integration Removed
21. Spring Session Hazelcast Auto-Configuration Removed
22. Spring Session MongoDB Auto-Configuration Removed
23. @SpringBootTest No Longer Auto-Configures MockMvc

Actuator Endpoint @Nullable: org.springframework.lang Replaced by JSpecify

Tier 2 - Won't Start Impact: S

The migration guide says actuator endpoint parameters need org.jspecify.annotations.Nullable, but the deprecated org.springframework.lang.Nullable still works. Migrate as hygiene, not as a fix.

What You'll See

```
// No error observed. On 4.0.0 and 4.0.7, an endpoint parameter
// annotated with @org.springframework.lang.Nullable is still treated
```

```
// as optional: GET without the parameter returns 200 (verified via MVC).
// The migration guide describes a break that does not reproduce.
```

What Changed

The Spring Boot 4.0 Migration Guide states that `org.springframework.lang.Nullable` no longer marks Actuator `@ReadOperation` / `@WriteOperation` parameters as optional, and that the supported annotation is now `org.jspecify.annotations.Nullable` (on the classpath transitively via Spring Framework 7.0). In practice the old annotation is still honoured on 4.0.0 and 4.0.7.

Why

Spring Framework 7.0 migrated its codebase to JSpecify null-safety annotations and Spring Boot 4.0 followed. The `org.springframework.lang` annotations remain for source compatibility but are no longer the active standard.

The Fix

Actuator endpoint parameter
Before

```
import org.springframework.lang.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}
```

After

```
import org.jspecify.annotations.Nullable;

@Endpoint(id = "my-endpoint")
public class MyEndpoint {
    @ReadOperation
    public String get(@Nullable String name) { ... }
}
```

How To Fix

Replace org.springframework.lang.Nullable with org.jspecify.annotations.Nullable.

Update the import on all Actuator endpoint parameters. Only the package changes; the annotation name and logic stay the same. JSpecify is already on the classpath via Spring Framework 7.0.

Scope Check

Search for `@Nullable` in classes annotated with `@Endpoint`, `@WebEndpoint`, or `@JmxEndpoint`. The issue is specific to Actuator endpoint parameter binding.

Watch Out

- The old annotation works today only because it is meta-annotated with JSpecify's `@Nullable`. Do not rely on that: it is deprecated, and a later release could stop honouring it. Migrate now while it is a mechanical import swap.

Verify

No observable break: an endpoint parameter annotated with the old @Nullable still returns 200 without the parameter on 4.0.0 and 4.0.7. Migrating to JSpecify is hygiene, not a fix

Further Info

This is one piece of a wider null-safety overhaul, described in the Spring Boot 4.0 blog post on null-safe applications.

Links

- Migration guide: removed support for org.springframework.lang.Nullable
- Null-safe applications with Spring Boot 4
- Spring Boot 4.0 Migration Guide

AspectJ Weaving Required for @Observed

Tier 2 - Won't Start Impact: S

Spring Boot 4.0 requires AspectJ-based weaving for the @Observed annotation to produce spans and metrics. Without the aspect configured, @Observed does nothing.

What You'll See

```
# No startup error. Detected when @Observed methods
# produce no observations:

@Observed(name = "order.process")
public Order processOrder(OrderRequest request) { ... }

# Expected in metrics:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"name":"order.process","measurements":[{"statistic":"COUNT","value":42.0}]}

# Actual after upgrade:
$ curl -s http://localhost:8080/actuator/metrics/order.process
{"status":404,"error":"Not Found"}

# No metric registered, no spans created. The method runs
# normally; observability is gone.
```

What Changed

In Spring Boot 3.x, `@Observed` worked via Spring AOP proxies and auto-configuration registered the `ObservedAspect` bean. In Spring Boot 4.0 that auto-configuration requires explicit AspectJ weaving support. Without the AspectJ weaving dependency and configuration, `@Observed` is ignored.

Why

The change gives `@Observed` full AspectJ semantics: it now works on non-proxied beans, self-invocations, and private methods, the places where the proxy-based approach failed without warning.

The Fix

> Maven dependency — add AspectJ weaver
> Before

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing</artifactId>
</dependency>
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
</dependency>
```

Configuration — register ObservedAspect if not auto-configured
Before

```
// ObservedAspect was auto-configured in Boot 3.x
```

After

```
@Configuration
public class ObservabilityConfig {
    @Bean
    public ObservedAspect observedAspect(ObservationRegistry
        return new ObservedAspect(registry);
    }
}
```

application.properties — enable AspectJ auto-proxy
Before

```
# proxy-based AOP was sufficient
```

After

```
spring.aop.auto=true
```

How To Fix

Add AspectJ weaver dependency.

Add `org.aspectj:aspectjweaver` to your dependencies. Spring Boot's dependency management provides the version.

Register ObservedAspect bean explicitly.

If the auto-configuration does not pick up the aspect, register an `ObservedAspect` bean manually in a `@Configuration` class. Pass the `ObservationRegistry` to its constructor.

Verify AOP is enabled.

Ensure `spring.aop.auto=true` (the default) and that you have not disabled AspectJ auto-proxying. Check for `@EnableAspectJAutoProxy` if you have custom AOP configuration.

Scope Check

Search for `@Observed` annotations across your codebase. Every `@Observed` method is broken until AspectJ weaving is configured. Also check whether dashboards or alerts depend on metrics or traces it produces.

Watch Out

- There is no error and no warning. The application starts, methods execute, and no observations (metrics or traces) are recorded. You discover it when monitoring dashboards show gaps.
- If you use `@Observed` on `@Repository` or `@Service` beans that are JDK dynamic proxies, ensure AspectJ weaving mode is compatible. CGLIB proxies are generally safer.
- Existing `@Timed` and `@Counted` annotations from Micrometer are separate from `@Observed` and may have their own configuration requirements. Check both.
- Load-time weaving (`-javaagent:aspectjweaver.jar`) gives the broadest coverage but requires JVM argument changes. Compile-time weaving requires build plugin changes. Choose based on your deployment model.

Verify

Custom @Observed aspects fire and metrics appear in /actuator/metrics

Further Info

Part of the broader Micrometer Observation API refactoring in Spring Framework 7.0. See also: controller-spans-disabled, observability-dashboard-gaps.

Links

- Spring-Break Demo

- Spring Boot 4.0 Migration Guide

Spring Batch Now Uses In-Memory Job Repository by Default

Tier 2 - Won't Start Impact: S

Spring Batch defaults to an in-memory job repository in Boot 4.0, so `BATCH_JOB_EXECUTION` and related tables are no longer created. Add `spring-boot-starter-batch-jdbc` to restore database persistence.

What You'll See

```
// Boot 3.5: BATCH_JOB_EXECUTION table created, job persisted.

// Boot 4.0 with only spring-boot-starter-batch:
// No schema created. Query against batch tables fails:
org.springframework.jdbc.BadSqlGrammarException:
  PreparedStatementCallback; bad SQL grammar
  [SELECT JOB_INSTANCE_ID, JOB_NAME from BATCH_JOB_INSTANCE ...];
  Table "BATCH_JOB_INSTANCE" not found

// Or job restart logic ignores execution history.
```

What Changed

Spring Boot 4.0 introduced `spring-boot-starter-batch-jdbc` as the explicit opt-in for database-backed job execution persistence. `spring-boot-starter-batch` now configures an in-memory `JobRepository` backed by a `ConcurrentHashMap`. The Batch schema DDL is no longer applied automatically unless the JDBC starter is on the classpath.

Why

Many Batch use cases (short-lived jobs, unit tests, development) do not need database persistence, yet the old default forced every Batch application to carry a datasource and manage schema migrations. The in-memory default is the right starting point; persistence is an explicit opt-in.

The Fix

pom.xml — add the JDBC starter to restore database persistence
Before

```
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-batch</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-batch-jdbc</artifactId>
</dependency>
```

How To Fix

Add spring-boot-starter-batch-jdbc for database-backed persistence.

If your application relies on job execution history, restartability, or the Batch admin tables, add `spring-boot-starter-batch-jdbc` alongside `spring-boot-starter-batch`. This restores the Boot 3.5 behaviour.

Embrace the in-memory default if persistence isn't needed.

For stateless or ephemeral jobs (one-shot CLI jobs, test fixtures, data imports) the in-memory repository is correct and removes the need for a datasource. Just remove any schema initialisation configuration.

Scope Check

Check for code that queries Batch tables directly via JDBC, uses `JobExplorer` to inspect execution history, or relies on Batch restart semantics (same job parameters, incomplete executions). All of these require the JDBC starter.

Watch Out

- This interacts with the `batch-schema-change` break: Boot 4.0 also renamed the schema sequence from `BATCH_STEP_EXECUTION_SEQ` to `BATCH_STEP_EXECUTION_SEQUENCE`. If you add `spring-boot-starter-batch-jdbc`, make sure your schema migration handles both changes.

Verify

Batch jobs persist execution history to the database when spring-boot-starter-batch-jdbc is declared

Further Info

In Boot 3.5, spring-boot-starter-batch with a JDBC datasource created the Batch schema and persisted job executions automatically. Code that relies on the Batch tables existing (auditing, restart logic, admin UI) breaks at runtime.

Links

- spring-break module: batch-in-memory-default
- Spring Boot 4.0 Migration Guide

Spring Batch 6 Schema Sequence Rename

Tier 2 - Won't Start Impact: S ⚙ OpenRewrite

Spring Batch 6 renamed its database sequences (e.g., BATCH_JOB_SEQ became BATCH_JOB_INSTANCE_SEQ). Existing databases using the old names fail when Batch tries to generate IDs.

What You'll See

```
org.springframework.dao.InvalidDataAccessResourceUsageException:
  could not fetch next sequence value
...
Caused by: java.sql.SQLSyntaxErrorException:
  sequence "BATCH_JOB_INSTANCE_SEQ" does not exist
 at org.postgresql.core.v3.QueryExecutorImpl.receiveErrorResponse(...)
 ...
org.springframework.batch.core.launch.JobLaunchException:
  Could not launch job 'importCustomersJob'.
  Failed to create new JobInstance for job [importCustomersJob]
```

What Changed

Spring Batch 6 renamed the database sequences used for generating IDs: `BATCH_JOB_SEQ` became `BATCH_JOB_INSTANCE_SEQ`, `BATCH_JOB_EXECUTION_SEQ` stayed the same, and `BATCH_STEP_EXECUTION_SEQ` stayed the same. The metadata table structures also changed. Applications with existing Batch schema from Spring Batch 5 will fail when the framework tries to use the new sequence names.

Why

The rename aligns sequence names with the tables they serve, making the schema self-documenting. The old generic names like `BATCH_JOB_SEQ` were ambiguous about which table's IDs they generated.

The Fix

SQL migration script — PostgreSQL

Before

```
-- Old sequences from Spring Batch 5
```

After

```
-- Rename sequences to match Spring Batch 6 expectations
ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SE
```

SQL migration script — MySQL
Before

```
-- Old sequences from Spring Batch 5
```

After

```
-- MySQL uses tables for sequence emulation
RENAME TABLE BATCH_JOB_SEQ TO BATCH_JOB_INSTANCE_SEQ;
```

application.properties — opt into old names temporarily
Before

```
# no Batch schema configuration
```

After

```
# Point Batch to legacy sequence names while you plan migrat
spring.batch.jdbc.isolation-level-for-create=ISOLATION_DEFAU
```

How To Fix

Run a schema migration script.

Create a Flyway or Liquibase migration that renames the sequences. For PostgreSQL: `ALTER SEQUENCE BATCH_JOB_SEQ RENAME TO BATCH_JOB_INSTANCE_SEQ;`. For MySQL, rename the emulation tables. Apply this before deploying the upgraded application.

Let Spring Batch recreate the schema.

If your Batch job data is disposable (e.g., dev/staging), set `spring.batch.jdbc.initialize-schema=always` to let Spring Batch drop and recreate the metadata tables with the new names. Do not use this in production.

Use the migration SQL from Spring Batch.

The Spring Batch 6 migration guide provides official SQL scripts for each database platform. Use those as the basis for your migration rather than writing them by hand.

Scope Check

Check if your application uses `spring-boot-starter-batch` and has a persistent job repository (not in-memory). If you see `BATCH_JOB_INSTANCE`, `BATCH_JOB_EXECUTION`, or `BATCH_STEP_EXECUTION` tables in your database, you need this migration. In-memory job repositories (MapJobRepositoryFactoryBean) are not affected.

Watch Out

- Sequences are only part of the schema change. Column types and table structures also changed in Spring Batch 6. Check the official migration SQL scripts for the full set of changes.
- If you run multiple application versions against the same database during a rolling deploy, the old version will break when the sequences are renamed. Plan a maintenance window or use blue-green deployment.
- H2 in-memory databases used in tests auto-create the schema fresh each run, so tests may pass while production fails. Test against a real database with existing Batch tables.

Verify

Spring Batch job launches with no BATCH_ table not found errors

Further Info

The sequence rename was announced in the Batch 6.0 milestone notes. See also: batch-listener-classes.

Links

- spring-break module: batch-schema-change
- Spring Batch 6 migration guide
- Spring Boot 4.0 Migration Guide

Controller & View Spans Disabled by Default

Tier 2 - Won't Start Impact: S

Spring Boot 4.0 disables controller and view rendering spans by default in Micrometer tracing. Dashboards, alerts, and SLOs built on those spans go dark.

What You'll See

```
# No startup error. Detected when spans are missing
# in your observability platform:

$ curl -s http://localhost:16686/api/traces?service=my-app | jq '.data[0].spans[].op
"HTTP GET /api/orders"
# Previously also showed:
# "OrderController#getOrders"
# "orders :: view rendering"
# These spans are now GONE.

# Grafana alert fires:
[FIRING] No controller-level spans for service my-app in last 15m
# Or SLO dashboard shows 0% data for controller latency percentiles.
```

What Changed

Spring Boot 4.0 changed the default for `management.observations.http.server.requests.name` and disabled the creation of child spans for Spring MVC controller method invocations and view rendering. Previously, Micrometer Tracing created spans for both the HTTP request and the controller method execution. Now only the HTTP server request span is created by default.

Why

The controller and view spans added overhead and noise for applications that did not need sub-request granularity. Disabling them by default reduces trace volume and performance impact. Applications that need them can re-enable them explicitly.

The Fix

application.properties — re-enable controller spans
Before

```
# default: controller spans were enabled
```

After

```
management.observations.http.server.requests.controller.enab
management.observations.http.server.requests.view.enabled=tr
```

application.yml equivalent
Before

```
# default: controller spans were enabled
```

After

```
management:
  observations:
    http:
      server:
        requests:
          controller:
            enabled: true
          view:
            enabled: true
```

How To Fix

Re-enable controller spans.

Add `management.observations.http.server.requests.controller.enabled=true` to your `application.properties`. Do the same for `view.enabled` if you need view rendering spans. This restores the Spring Boot 3.x behaviour.

Update dashboards and alerts.

If you decide to accept the new default (no controller spans), update your Grafana dashboards, alert rules, and SLO definitions to use the HTTP server request span instead of controller-level spans. The HTTP span still contains the handler method in its attributes.

Use @Observed for targeted spans.

Instead of blanket controller spans, annotate specific controller methods with `@Observed` to get spans only where you need them. This is more efficient than re-enabling all controller spans globally.

Scope Check

Check your observability dashboards and alerts for references to controller-level span names. Search for span names like `controller.method.name`, `HandlerMethod`, or view

rendering span names in your Grafana/Datadog/New Relic configuration. If you only use HTTP-level metrics (`http.server.requests`), you are not affected.

Watch Out

- The application gives no errors and no warnings. You discover the change when dashboards go blank or missing-data alerts fire.
- If you use distributed tracing to debug latency between the HTTP layer and the controller, the missing span makes it harder to pinpoint where time is spent inside the server.
- Micrometer's `@Timed` annotation on controller methods still works for metrics. Only tracing spans are affected, not timer metrics.

Verify

Trace dashboard shows controller-level spans after re-enabling

Further Info

Part of Spring Boot 4.0's Micrometer Tracing defaults overhaul. See also: observability-dashboard-gaps, aspectj-observed.

Links

- Spring Boot 4.0 Migration Guide

Deprecated RestTemplateBuilder APIs Removed

Tier 2 - Won't Start Impact: S

Deprecated `RestTemplateBuilder` methods are removed in Boot 4.0: code that compiled with warnings on 3.5 no longer compiles. Migrate to `RestClient`.

What You'll See

```
// Calling a deprecated builder method
RestTemplate rt = builder
    .additionalMessageConverters(new MappingJackson2HttpMessageConverter())
    .build();

// Boot 3.5: compiles with deprecation warning, runs fine.

// Boot 4.0: compile error.
error: cannot find symbol
  symbol: method additionalMessageConverters(MappingJackson2HttpMessageConverter)
```

What Changed

Spring Boot 3.x deprecated several `RestTemplateBuilder` methods to signal the migration path to `RestClient` (introduced in Spring Framework 6.1). Boot 4.0 removes these deprecated methods entirely, following the standard deprecation-then-removal cycle.

Why

`RestTemplate` is in maintenance mode. The modern replacement is `RestClient`, which offers a fluent, functional API and is the recommended HTTP client for new development in Spring Framework 6.x+. Removing the deprecated builder methods accelerates the migration.

The Fix

> Migrate from RestTemplate to RestClient
> Before
>
> ```
> @Bean
> public RestTemplate restTemplate(RestTemplateBuilder builder
>     return builder
>         .additionalMessageConverters(new MappingJackson2Http
> ```

```
        .build();
}
```

After

```
@Bean
public RestClient restClient() {
    return RestClient.builder()
        .messageConverters(converters ->
            converters.add(new MappingJackson2HttpMessageCon
        .build();
}
```

How To Fix

Migrate to RestClient.

`RestClient` is the modern replacement. It offers a similar fluent builder API and is available from Spring Framework 6.1 onward (Spring Boot 3.2+). Migrating removes the deprecated call and future-proofs the HTTP client code.

If RestTemplate must be kept, remove the deprecated builder calls.

Construct a `RestTemplate` directly (via constructor or with a plain `RestTemplateBuilder.build()`) and configure message converters via `restTemplate.getMessageConverters().add(...)` after construction.

Scope Check

Run `mvn compile` with `-Xlint:deprecation` on Boot 3.5 to find every deprecated API call before upgrading to 4.0. Pay particular attention to `RestTemplateBuilder` method chains: each deprecated call is a potential compile error on 4.0.

Watch Out

- RestTemplate auto-configuration is also removed in Boot 4.0, a separate break from deprecated method removal. If you rely on an auto-configured `RestTemplate` bean rather than declaring one, see the `resttemplate-autoconfig` card.

Verify

Code using deprecated RestTemplateBuilder methods fails to compile on Boot 4.0

Further Info

Affected methods include additionalMessageConverters(), messageConverters(), and requestFactory(). See also: resttemplate-autoconfig.

Links

- spring-break module: deprecated-classes-removed
- Spring Boot 4.0 Migration Guide
- RestClient migration guide

CascadeType.SAVE_UPDATE Removed in Hibernate 7

Tier 2 - Won't Start Impact: M ⚙ OpenRewrite

Hibernate 7 removed the proprietary CascadeType.SAVE_UPDATE. Entities using this cascade type fail at startup or throw MappingException when the SessionFactory is built.

What You'll See

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory'
...
Caused by: org.hibernate.MappingException:
  Unrecognized legacy cascade style: save-update
 at org.hibernate.boot.model.internal.EntityBinder.bindEntityAnnotation(...)
 at org.hibernate.boot.model.internal.AnnotationBinder.bindClass(...)
 ...
APPLICATION FAILED TO START
---
Description:
Failed to initialize JPA EntityManagerFactory:
Unrecognized legacy cascade style: save-update
```

What Changed

`org.hibernate.annotations.CascadeType.SAVE_UPDATE` was a Hibernate-specific extension to JPA cascade types. It triggered cascade behaviour on Hibernate's proprietary `Session.saveOrUpdate()` method. Hibernate 7 removed both `saveOrUpdate()` and the corresponding cascade type. Code using the Hibernate-specific `@Cascade` annotation with `SAVE_UPDATE` breaks at entity mapping time.

Why

The `saveOrUpdate()` method was deprecated in Hibernate 6 as part of aligning with the JPA `EntityManager` API. Hibernate 7 completed the removal. The standard JPA `CascadeType.PERSIST` and `CascadeType.MERGE` cover the same use cases without vendor lock-in.

The Fix

> Using Hibernate @Cascade annotation
> Before

```
import org.hibernate.annotations.Cascade;
import org.hibernate.annotations.CascadeType;

@OneToMany(mappedBy = "order")
@Cascade(CascadeType.SAVE_UPDATE)
private List<OrderItem> items;
```

After

```
@OneToMany(mappedBy = "order", cascade = {
    javax.persistence.CascadeType.PERSIST,
    javax.persistence.CascadeType.MERGE
})
private List<OrderItem> items;
```

Using JPA cascade with Hibernate extras
Before

```
@OneToMany(mappedBy = "parent")
@Cascade({CascadeType.SAVE_UPDATE, CascadeType.DELETE})
private Set<Child> children;
```

After

```
@OneToMany(mappedBy = "parent", cascade = {
    CascadeType.PERSIST, CascadeType.MERGE, CascadeType.REMO
})
private Set<Child> children;
```

How To Fix

Replace with JPA standard cascades.

Swap `CascadeType.SAVE_UPDATE` for the combination of `CascadeType.PERSIST` and `CascadeType.MERGE`. These two JPA-standard cascades cover the same persist-or-update behaviour that `SAVE_UPDATE` provided.

Remove @Cascade if redundant.

If your entity already has `cascade = CascadeType.ALL` on the JPA annotation, the Hibernate `@Cascade` is redundant. Delete it entirely.

Scope Check

Search for `CascadeType.SAVE_UPDATE` and `@Cascade` (the Hibernate annotation, not JPA). Every hit is a mapping that will fail at startup. Also search for `session.saveOrUpdate(`; that method is gone too.

Watch Out

- `CascadeType.PERSIST` alone does not cover the "update existing detached entity" case. You need both `PERSIST` and `MERGE` to match the old `SAVE_UPDATE` behaviour.
- If you were using `Session.saveOrUpdate()` directly in DAO code, that method is also removed. Replace with `entityManager.merge()` or `entityManager.persist()`.
- Some code generators and legacy mapping tools emit `SAVE_UPDATE` by default. Re-run generation after upgrading.

Verify

mvn compile — no CascadeType.SAVE_UPDATE errors

Further Info

Arrives via spring-boot-starter-data-jpa, which pulls in Hibernate 7.0. See also: session-delete-removed, hibernate-dialect-removal.

Links

- spring-break module: hibernate-cascade-removal
- Hibernate 7 migration guide
- Spring Boot 4.0 Migration Guide

Version-Specific Hibernate Dialects Removed

Tier 2 - Won't Start Impact: S ⚙ OpenRewrite

Hibernate 7 removed version-specific dialect classes like MySQL57Dialect and PostgreSQL95Dialect. Applications that explicitly configure a dialect fail at startup with ClassNotFoundException.

What You'll See

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'entityManagerFactory'
...
Caused by: java.lang.ClassNotFoundException:
  org.hibernate.dialect.MySQL57Dialect
 at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.ja
 at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders
 ...
APPLICATION FAILED TO START
---
Description:
Failed to configure a DataSource: Hibernate dialect
'org.hibernate.dialect.MySQL57Dialect' could not be found.
```

What Changed

Hibernate 7 removed all version-specific dialect classes (`MySQL57Dialect`, `MySQL8Dialect`, `PostgreSQL95Dialect`, `PostgreSQL10Dialect`, `Oracle12cDialect`, `SQLServer2016Dialect`, etc.). Only the base dialect classes remain (`MySQLDialect`, `PostgreSQLDialect`, `OracleDialect`, `SQLServerDialect`). Hibernate now auto-detects the database version from the JDBC connection and configures features accordingly.

Why

A dialect class per database version created a combinatorial explosion. Runtime detection removes the maintenance burden and the risk of choosing the wrong class.

The Fix

application.properties
Before

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialec
```

After

```
# Remove explicit dialect - Hibernate auto-detects from JDBC
```

application.properties — if you must be explicit
Before

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialec
```

After

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialec
```

persistence.xml
Before

```
<property name="hibernate.dialect" value="org.hibernate.dial
```

After

```
<property name="hibernate.dialect" value="org.hibernate.dial
```

How To Fix

Remove explicit dialect (recommended).

Delete the `spring.jpa.properties.hibernate.dialect` property entirely. Hibernate 7 detects the correct dialect from the JDBC connection metadata.

Switch to base dialect class.

If you need an explicit dialect for testing or CI (e.g., H2 in-memory), use the unversioned base class: `MySQLDialect`, `PostgreSQLDialect`, `OracleDialect`, or `SQLServerDialect`.

Scope Check

Search for `hibernate.dialect` in `application.properties`, `application.yml`, `persistence.xml`, and any programmatic `LocalContainerEntityManagerFactoryBean` configuration. Also check test configurations and Docker Compose profiles.

Watch Out

- If you use Testcontainers or H2 for tests, those test configs often set an explicit dialect. Check `src/test/resources` too.

- Some connection pools and frameworks set the dialect programmatically. Search Java code for `setProperty("hibernate.dialect"` too.
- The auto-detection requires an active JDBC connection at startup. If your application uses lazy datasource initialization, you may still need an explicit base dialect class.

Verify

App starts with no DialectResolutionInfoMissingException

Further Info

Driven by Hibernate 7.0, upstream of Spring Boot 4.0. The classes were deprecated in Hibernate 6.0. Affects spring-boot-starter-data-jpa consumers. See also: hibernate-date-types, cascade-save-update.

Links

- spring-break module: hibernate-dialect-removal
- Hibernate 7 migration guide
- Spring Boot 4.0 Migration Guide

Hibernate Untyped Join Query Rejected at Runtime

Tier 2 - Won't Start Impact: S

Hibernate 7 rejects untyped join queries with no explicit SELECT clause. A query that returned Object[] on Boot 3.5 throws SemanticException at runtime on Boot 4.0.

What You'll See

```
// Query without explicit result type
session.createQuery("from Product p join p.category c").getResultList();

// Boot 3.5 (Hibernate 6.x): runs fine, returns List<Object[]>

// Boot 4.0 (Hibernate 7.x): throws at runtime
org.hibernate.query.SemanticException:
  Query has no explicit SELECT and multiple query roots are defined.
  An explicit SELECT clause is required when multiple query roots exist
  or when the query contains a JOIN.
```

What Changed

Hibernate 7.0 tightened validation of HQL queries. A query like `from Product p join p.category c` has two implicit query roots (Product and the joined Category). Without an explicit `SELECT` clause or result type, the return type is ambiguous. Hibernate 6.x resolved this as `Object[]`; Hibernate 7.0 rejects it at parse time with a `SemanticException`.

Why

Silent return of `Object[]` hides intent and causes surprising `ClassCastException` at the point of use rather than at the query. A hard error forces explicit, clearer queries.

The Fix

Add an explicit SELECT and result type
Before

```
session.createQuery("from Product p join p.category c")
       .getResultList();
```

After

```
session.createQuery(
    "SELECT p FROM Product p JOIN p.category c",
```

```
    Product.class
).getResultList();
```

Or use a typed tuple query
Before

```
session.createQuery("from Product p join p.category c")
       .getResultList();
```

After

```
session.createQuery(
    "SELECT p, c FROM Product p JOIN p.category c",
    Object[].class
).getResultList();
```

How To Fix

Add explicit SELECT clause and result type (recommended).

All `Session.createQuery(String)` calls with join queries must include an explicit `SELECT` clause. Pass the expected result class as the second argument: `createQuery(hql, Entity.class)`.

Search for bare 'from' queries.

Grep for `createQuery("from` and `createQuery('from` in your codebase. Any query that starts with `from` and contains a join is at risk.

Scope Check

Search for `createQuery(` calls. JPQL queries via `EntityManager.createQuery(String, Class)` are also affected if they omit the `SELECT` clause with joins.

Watch Out

- The query compiles and runs on Boot 3.5 without any warning. The failure only surfaces on Boot 4.0 at the first call site, not at startup. Tests that exercise query paths are the only way to catch this before production.
- Simple queries without joins are unaffected: `from Product p` with a single root remains valid. Only multi-root or joined queries trigger the error.

Verify

HQL join queries without explicit SELECT or result type execute without SemanticException

Further Info

Introduced in Hibernate 7.0, upstream of Spring Boot 4.0. Affects any application using Session.createQuery(String) with join queries and no result type.

Links

- spring-break module: hibernate-query-type-required
- Hibernate 7.0 Migration Guide
- Spring Boot 4.0 Migration Guide

write-dates-as-timestamps Property Breaks Startup

Tier 2 - Won't Start Impact: M

The `spring.jackson.serialization.write-dates-as-timestamps` property no longer binds in Boot 4.0: Jackson 3 moved the feature out of `SerializationFeature`. One leftover config line stops your application from starting.

What You'll See

```
# application.properties (worked on Boot 3.5)
spring.jackson.serialization.write-dates-as-timestamps=true

// Spring Boot 4.0 — application fails to start:
org.springframework.boot.context.properties.bind.BindException:
  Failed to bind properties under 'spring.jackson.serialization' to
  java.util.Map<tools.jackson.databind.SerializationFeature, Boolean>
Caused by: java.lang.IllegalArgumentException: No enum constant
  tools.jackson.databind.SerializationFeature.write-dates-as-timestamps
```

What Changed

Jackson 3 reorganised its feature flags: date/time toggles such as WRITE_DATES_AS_TIMESTAMPS are no longer constants on `SerializationFeature`. Boot 4.0 still binds `spring.jackson.serialization.*` keys against that enum, so the old property value fails enum conversion and property binding aborts context startup.

Why

The old `SerializationFeature` enum values don't map one-to-one to Jackson 3's new configuration API; the date/time features moved to their own feature set. Spring Boot's property binding follows the new enum, so values naming removed constants fail fast instead of being remapped.

The Fix

application.properties — old (silently ignored)
Before

```
spring.jackson.serialization.write-dates-as-timestamps=true
```

After

```
# Removed - configure via ObjectMapper customiser instead
```

ObjectMapper customiser replacement
Before

```
# (relied on spring.jackson.serialization.write-dates-as-tim
```

After

```
@Bean
public Jackson2ObjectMapperBuilderCustomizer timestampDates(
    return builder -> builder.featuresToEnable(
        SerializationFeature.WRITE_DATES_AS_TIMESTAMPS
    );
}
```

Alternative: per-field annotation
Before

```
private Instant timestamp;
```

After

```
@JsonFormat(shape = JsonFormat.Shape.NUMBER)
private Instant timestamp;
```

How To Fix

Register an ObjectMapper customiser bean.

Create a `Jackson2ObjectMapperBuilderCustomizer` that explicitly enables `WRITE_DATES_AS_TIMESTAMPS`. This replaces the dead properties-based config.

Migrate clients to ISO-8601 (recommended).

If you can coordinate with API consumers, drop the timestamp format entirely and adopt ISO-8601 strings. Remove the old property and let Jackson 3's new default take effect.

Scope Check

Search your `application.properties` and `application.yml` files for any `spring.jackson.serialization` or `spring.jackson.deserialization` entries. Every one of those feature toggles is potentially dead in Boot 4.0.

Watch Out

- This compounds with the jackson-date-format change. If you set `write-dates-as-timestamps=true` specifically to keep numeric timestamps, you now have two problems: the default flipped, and the property that pinned the old behaviour stops your app from starting.
- Any `spring.jackson.serialization.` *or* *`spring.jackson.deserialization.`* value naming an enum constant that Jackson 3 moved or removed fails startup the same way.

Verify

Application fails to start: BindException on spring.jackson.serialization, "No enum constant tools.jackson.databind.SerializationFeature.write-dates-as-timestamps"

Further Info

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. Verified 15 July 2026 on 4.0.7. Compounds with jackson-date-format. See also: jackson-date-format, jackson-property-inclusion.

Links

- spring-break module: jackson-dates-timestamps
- Spring Boot 4.0 Migration Guide
- Jackson 3 in Spring (blog)

JacksonException No Longer Extends IOException

Tier 2 - Won't Start Impact: M ⚙ OpenRewrite

Jackson 3.0 changed the exception hierarchy so JacksonException extends RuntimeException instead of IOException. Existing catch blocks silently miss Jackson errors, and throws clauses become lies.

What You'll See

```
com.example.OrderController: Unhandled exception in /api/orders
tools.jackson.core.exc.StreamReadException: Unexpected character ('x' (code 120)):
  expected a valid value (JSON String, Number, Array, Object or token)
 at [Source: (String)"x]"; line: 1, column: 2]
 at tools.jackson.core.json.JsonParser._reportUnexpectedChar(JsonParser.java:...)
 ...
Caused by: tools.jackson.core.JacksonException (not an IOException!)
---
HTTP 500 Internal Server Error
{"timestamp":"2026-04-15T10:22:03","status":500,
 "error":"Internal Server Error","path":"/api/orders"}
```

What Changed

In Jackson 2.x, `JacksonException` extended `java.io.IOException`, so any `catch (IOException e)` block would also catch malformed-JSON errors. In Jackson 3.0, `JacksonException` extends `RuntimeException`. Catch blocks for `IOException` no longer intercept Jackson parsing failures, and methods that declared `throws IOException` for Jackson work now throw unchecked exceptions instead.

Why

The original hierarchy was a design mistake: it forced checked-exception handling even for in-memory parsing, which involves no I/O. Making `JacksonException` unchecked aligns with modern Java conventions and removes the boilerplate try/catch.

The Fix

> Controller error handling — before
> Before

```java
try {
    Order order = objectMapper.readValue(body, Order.class);
} catch (IOException e) {
```

```
    return ResponseEntity.badRequest().body("Invalid JSON");
}
```

After

```
try {
    Order order = objectMapper.readValue(body, Order.class);
} catch (JacksonException e) {
    return ResponseEntity.badRequest().body("Invalid JSON");
}
```

Spring @ExceptionHandler — before
Before

```
@ExceptionHandler(IOException.class)
public ResponseEntity<String> handleJsonError(IOException ex
```

After

```
@ExceptionHandler(JacksonException.class)
public ResponseEntity<String> handleJsonError(JacksonExcepti
```

Method signature cleanup
Before

```
public Order parseOrder(String json) throws IOException {
```

After

```
public Order parseOrder(String json) {
```

How To Fix

Search for catch blocks.

Grep for `catch (IOException` and `catch (JsonProcessingException`. Replace with `catch (JacksonException` where the intent is to handle Jackson parse failures. Keep `IOException` catches for genuine I/O work (file reads, sockets).

Update @ExceptionHandler advice.

If your `@RestControllerAdvice` catches `IOException` to return 400 on bad JSON, add a separate handler for `JacksonException`. Otherwise those errors now surface as unhandled 500s.

Clean up throws clauses.

Methods that declared `throws IOException` only for Jackson can drop the clause entirely. The compiler won't complain, but the stale declaration misleads readers.

Scope Check

Search for `catch (IOException`, `catch (JsonProcessingException`, and `throws IOException` near any `ObjectMapper` usage. Every hit where the catch was meant to intercept Jackson errors now misses at runtime. REST controllers, message listeners, and integration test assertions are the most common locations.

Watch Out

- The code compiles cleanly because `JacksonException` is now unchecked. You only discover the problem when bad input arrives, the catch block no longer fires, and clients get 500 errors instead of 400s.
- A broad `catch (Exception` still catches Jackson errors, but loses the ability to return a domain-appropriate error message. Narrow your catch clauses.
- Spring's built-in `HttpMessageNotReadableException` still wraps Jackson errors for `@RequestBody` parsing. The risk is in manual `ObjectMapper` calls inside your own code.

Verify

mvn compile — no JsonMappingException symbol errors

Further Info

Driven by Jackson 3.0, upstream of Spring Boot 4.0. The change was announced in 2022 and finalised in Jackson 3.0-rc1. See also: jackson-group-id, jackson-class-renames.

Links

- spring-break module: jackson-exception-hierarchy
- Jackson 3 migration guide
- Spring Boot 4.0 Migration Guide

javax.annotation Removed

Tier 2 - Won't Start Impact: S ⚙ OpenRewrite

Spring Framework 7.0 stopped processing javax.annotation lifecycle annotations. @PostConstruct and @PreDestroy methods silently never fire. The code compiles and nothing warns you.

What You'll See

```
// No compile error — javax.annotation-api is still downloadable.
// Failure is silent at runtime:

@PostConstruct
public void init() {
    this.ready = true;  // never called on Boot 4.0
}

// Later in a test or endpoint:
java.lang.NullPointerException: Cannot invoke method because 'this.cache' is null
// or simply: assertions on initialisation state fail unexpectedly
```

What Changed

The `javax.annotation:javax.annotation-api` dependency is no longer on the Spring Boot classpath. Annotations `@javax.annotation.PostConstruct`, `@javax.annotation.PreDestroy`, and `@javax.annotation.Resource` are unavailable. The Jakarta equivalent is `jakarta.annotation:jakarta.annotation-api`, managed by the Boot BOM. See also: javax-inject-removed.

Why

The Jakarta EE transition moved all `javax.` *APIs to* `jakarta.`. Spring Framework 7.0 completed this transition by dropping the legacy `javax.annotation` artifact.

The Fix

Java imports

Before

```
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;
```

After

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
```

pom.xml — if you had an explicit dependency
Before

```
<dependency>
    <groupId>javax.annotation</groupId>
    <artifactId>javax.annotation-api</artifactId>
    <version>1.3.2</version>
</dependency>
```

After

```
<dependency>
    <groupId>jakarta.annotation</groupId>
    <artifactId>jakarta.annotation-api</artifactId>
</dependency>
```

How To Fix

Switch to jakarta.annotation.

Replace `javax.annotation` imports with `jakarta.annotation`. The annotations are identical: only the package name changed. The `jakarta.annotation-api` artifact is managed by the Spring Boot BOM so no version is needed.

Remove explicit dependency version.

If you declared `javax.annotation-api` explicitly in your POM, replace it with `jakarta.annotation-api` and omit the version.

Scope Check

Search for `javax.annotation` in all Java/Kotlin sources and build files. Pay particular attention to `@PostConstruct`: it appears in almost every service class with initialisation logic. Also check for `javax.annotation:javax.annotation-api` in POM files and Gradle build scripts.

Watch Out

- `@PostConstruct` is skipped if the import is wrong: no error at startup, the method simply never runs. If your cache, connection pool, or scheduled task isn't initialising, check the import first.

- `@Resource` (JSR-250 named injection) also lives in `javax.annotation`. If you use it for JNDI lookups or named bean injection, that import needs updating too.

Verify

App starts and @PostConstruct methods fire — confirm via log output or assert that initialisation state is set after context loads

Further Info

Driven by the Jakarta EE transition completed in Spring Framework 7.0. The javax.annotation:javax.annotation-api artifact still exists in Maven Central and still compiles. The break is at runtime, and @Resource injection stops working the same way.

Links

- spring-break module: javax-annotation-removed
- Spring Boot 4.0 Migration Guide

javax.inject Removed

Tier 2 - Won't Start Impact: S ⚙ OpenRewrite

Spring Framework 7.0 stopped processing javax.inject annotations. @Inject fields silently stay null. @Named beans may not be registered. The code compiles, the app starts, and nothing warns you.

What You'll See

```
// No compile error — javax.inject:javax.inject still downloadable.
// Failure is silent at runtime:

@Named("orderService")
public class OrderService {
    @Inject
    private OrderRepository repo;  // null on Boot 4.0 — never injected
}

// At call time:
java.lang.NullPointerException: Cannot invoke
  "com.example.OrderRepository.findById(Long)"
  because "this.repo" is null
```

What Changed

The `javax.inject:javax.inject` dependency is no longer on the Spring Boot classpath. Annotations like `@javax.inject.Inject`, `@javax.inject.Named`, and `@javax.inject.Singleton` are unavailable. The Jakarta equivalent is `jakarta.inject:jakarta.inject-api`.

Why

The Jakarta EE transition moved all `javax.` *APIs to* `jakarta.`. Spring Framework 7.0 completed this transition by dropping support for the legacy `javax.inject` artifact.

The Fix

Java imports

Before

```
import javax.inject.Inject;
import javax.inject.Named;
```

After

```
import jakarta.inject.Inject;
import jakarta.inject.Named;
```

pom.xml — if you had an explicit dependency
Before

```
<dependency>
    <groupId>javax.inject</groupId>
    <artifactId>javax.inject</artifactId>
    <version>1</version>
</dependency>
```

After

```
<dependency>
    <groupId>jakarta.inject</groupId>
    <artifactId>jakarta.inject-api</artifactId>
</dependency>
```

Or just use Spring annotations
Before

```
@Named("orderService")
public class OrderService {
    @Inject
    private OrderRepository repo;
```

After

```
@Service("orderService")
public class OrderService {
    @Autowired
    private OrderRepository repo;
```

How To Fix

Switch to jakarta.inject.

Replace `javax.inject` imports with `jakarta.inject`. The annotations are identical: only the package name changed. The `jakarta.inject-api` artifact is managed by the Spring Boot BOM.

Switch to Spring annotations (recommended).

Replace `@Inject` with `@Autowired` and `@Named` with `@Component` / `@Service` / `@Qualifier`. This removes the dependency on the inject API entirely.

Scope Check

Search for `javax.inject` in all Java/Kotlin sources and build files. Every `import javax.inject` statement needs updating. Also check for `javax.inject:javax.inject` in POM files.

Watch Out

- If you're using `@Inject` for CDI compatibility (e.g., shared code between Spring and a Jakarta EE server), switch to `jakarta.inject`: it works in both environments. Don't switch to Spring-specific annotations if portability matters.
- Constructor injection with `@Inject` on a single-constructor class can simply remove the annotation entirely. Spring auto-detects single-constructor injection since 4.3.

Verify

App starts and @Inject fields are non-null — confirm via a test that autowires a bean registered with @Named

Further Info

Driven by the Jakarta EE transition completed in Spring Framework 7.0. The javax.inject:javax.inject:1 artifact still exists in Maven Central and still compiles. The break is at runtime, and the first NullPointerException at call time is your only clue.

Links

- spring-break module: javax-inject-removed
- Spring Boot 4.0 Migration Guide

MockitoTestExecutionListener Removed

Tier 2 - Won't Start Impact: S

Spring Boot 4.0 removes MockitoTestExecutionListener. @Mock fields in @SpringBootTest tests stay null and throw NullPointerException unless @ExtendWith(MockitoExtension.class) is added.

What You'll See

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;

    @Test
    void test() {
        // Boot 3.5: myService is initialised → passes
        // Boot 4.0: myService is null → NullPointerException
        when(myService.greet()).thenReturn("Hello");
    }
}

java.lang.NullPointerException: Cannot invoke method on null object
  at com.example.MyTest.test(MyTest.java:15)
```

What Changed

`MockitoTestExecutionListener` was a Spring Boot-specific bridge that triggered Mockito's annotation processing inside Spring's test lifecycle. Spring Boot 4.0 removed this listener, aligning with the standard JUnit 5 approach of using `@ExtendWith(MockitoExtension.class)` or `MockitoAnnotations.openMocks(this)` in a `@BeforeEach` setup method.

Why

The listener duplicated Mockito's own JUnit 5 extension. Removing it pushes tests toward the idiomatic Mockito + JUnit 5 pattern that works in any JUnit 5 project.

The Fix

Add MockitoExtension alongside @SpringBootTest
Before

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;
}
```

After

```
@SpringBootTest
@ExtendWith(MockitoExtension.class)
class MyTest {
    @Mock
    private MyService myService;
}
```

Or switch to @MockitoBean (Boot 4.0 replacement for @MockBean)
Before

```
@SpringBootTest
class MyTest {
    @Mock
    private MyService myService;
}
```

After

```
@SpringBootTest
class MyTest {
    @MockitoBean
    private MyService myService;  // registered in the Sprin
}
```

How To Fix

Add @ExtendWith(MockitoExtension.class) to the test class.

This is the standard JUnit 5 way to enable Mockito annotation processing. It works alongside `@SpringBootTest` and needs no new dependency: Mockito's JUnit 5 extension is already on the test classpath via `spring-boot-starter-test`.

Use @MockitoBean instead of @Mock (if the mock belongs in the context).

If the mock is being injected into a Spring bean under test, use `@MockitoBean` (Boot 4.0's replacement for `@MockBean`). This registers the mock in the Spring application context rather than just the test instance.

Scope Check

Search for `@Mock`, `@Spy`, and `@Captor` annotations in test classes that also use `@SpringBootTest` or any Spring test slice annotation (`@WebMvcTest`, `@DataJpaTest`, etc.). Each such class needs `@ExtendWith(MockitoExtension.class)` added.

Watch Out

- If a test class extends another that has `@Mock` fields, the parent class also needs the extension. Inheritance doesn't propagate the `@ExtendWith` from child to parent field initialisation.
- Tests that used `@MockBean` (Boot 3.5) are affected by a separate change: `@MockBean` itself is removed in Boot 4.0 in favour of `@MockitoBean`. See the `mockbean-removed` card.

Verify

@Mock fields in @SpringBootTest classes are initialised correctly

Further Info

The listener called MockitoAnnotations.openMocks() on each test instance, initialising @Mock, @Spy, and @Captor fields before each test. Mockito's own JUnit 5 extension has done the same job since Mockito 3.x.

Links

- spring-break module: mockito-test-execution-listener
- Spring Boot 4.0 Migration Guide

Modular Auto-Configuration

Tier 2 - Won't Start Impact: M

Boot 4.0 split auto-configuration into per-feature modules. Free beans (ObjectMapper, Validator) vanish without their starter. The app starts; NoSuchBeanDefinitionException hits at first use.

What You'll See

```
// App compiles and starts. Failure at first injection or endpoint call:

org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'com.fasterxml.jackson.databind.ObjectMapper' available

// Or from a @RestController trying to serialise a response:
org.springframework.http.converter.HttpMessageNotWritableException:
  No converter for [class com.example.OrderDto]
```

What Changed

The `spring-boot-autoconfigure` jar bundled auto-configuration for every Spring Boot feature. Boot 4.0 splits it into focused modules, one per feature area, and each activates only when its corresponding starter is explicitly present. Adding `spring-boot-starter-web` no longer implicitly enables Jackson, validation, or actuator auto-configuration.

Why

The monolithic jar meant any starter pulled in auto-configuration for hundreds of features the application never used. The split cuts startup time and memory, and makes dependencies explicit: you only pay for what you declare.

The Fix

pom.xml — add starters that were previously implicit
Before

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

After

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<!-- Now required explicitly — no longer implicit via web st
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-json</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

How To Fix

Audit and add missing starters.

Run `mvn dependency:tree` and compare what was transitive on Boot 3.5 with what your Boot 4.0 build pulls in. Add an explicit `spring-boot-starter-*` dependency for every feature your application uses: `spring-boot-starter-json` for Jackson, `spring-boot-starter-validation` for Bean Validation, `spring-boot-starter-actuator` for Micrometer metrics.

Watch @RestController endpoints.

If a controller returns an object that Jackson used to serialise automatically, the endpoint now returns `406 Not Acceptable` or throws `HttpMessageNotWritableException` rather than an obvious startup error. Add `spring-boot-starter-json` to restore the message converter.

Scope Check

Run the app and scan for `NoSuchBeanDefinitionException` in startup logs. Also grep your code for `@Autowired ObjectMapper`, `@Autowired Validator`, and similar injections that relied on implicit auto-configuration. Check `@RestController` methods that return non-String types: these rely on Jackson message converters.

Watch Out

- The context loads fine; the missing bean surfaces at first injection or when a converter is needed. Endpoint smoke tests find the gaps fastest.

- `spring-boot-starter-web` still brings Jackson onto the *classpath* as a transitive compile dependency: the classes are present but never configured into beans. The app compiles with no import errors and no signal that anything is wrong.
- Each application hits a different subset of missing beans depending on the features it uses. There is no single fix: audit, using the Migration Guide's starter mapping table as a checklist.

Verify

App starts and ObjectMapper, Validator, and other auto-configured beans are present — confirm with a test that autowires each one

Further Info

In Boot 3.x every starter dragged in the entire auto-configuration bundle, so features like Jackson, Bean Validation, and Micrometer configured themselves without a declared dependency.

Links

- spring-break module: modular-starters
- Spring Boot 4.0 Migration Guide
- Modularizing Spring Boot (Spring Blog)

OAuth 2.0 Password Grant Completely Removed

Tier 2 - Won't Start Impact: L

Spring Security 7 removed the OAuth 2.0 Resource Owner Password Credentials grant. Apps that authenticate with it fail at startup or return 401s at runtime.

What You'll See

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'securityFilterChain'
...
Caused by: java.lang.IllegalArgumentException:
  The grant type 'password' is not supported.
  Supported grant types are: [authorization_code, client_credentials,
  refresh_token, urn:ietf:params:oauth:grant-type:device_code,
  urn:ietf:params:oauth:grant-type:token-exchange]
 at org.springframework.security.oauth2.client.registration.ClientRegistration$Build
---
APPLICATION FAILED TO START
```

What Changed

The password grant, deprecated in Spring Security 5.x, is gone in Spring Security 7 (shipped with Spring Boot 4.0). The `ClientRegistration` builder rejects `authorization-grant-type: password`, and the `ResourceOwnerPasswordResourceDetails` class no longer exists.

Why

The password grant hands user credentials directly to the client application, defeating the point of OAuth delegation. The OAuth 2.1 specification (RFC 9700) formally removed it; Spring Security followed.

The Fix

application.yml — client registration
Before

```yaml
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
```

```
            authorization-grant-type: password
            client-id: my-client
            client-secret: secret
        provider:
          my-api:
            token-uri: https://auth.example.com/oauth/token
```

After

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: authorization_code
            client-id: my-client
            client-secret: secret
            scope: openid,profile
            redirect-uri: "{baseUrl}/login/oauth2/code/{regi
        provider:
          my-api:
            issuer-uri: https://auth.example.com
```

Java configuration — programmatic registration

Before

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.PASSWORD)
    .tokenUri("https://auth.example.com/oauth/token")
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

After

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZA
    .authorizationUri("https://auth.example.com/authorize")
    .tokenUri("https://auth.example.com/oauth/token")
    .redirectUri("{baseUrl}/login/oauth2/code/{registrationI
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

How To Fix

Migrate to Authorization Code + PKCE.

Replace the password grant with Authorization Code flow. For browser-based apps, use Authorization Code with PKCE. This is the recommended approach and requires changes to both the client application and possibly the authorization server configuration.

Use Client Credentials for service-to-service.

If the password grant was used for machine-to-machine communication (no actual user), switch to the Client Credentials grant. This does not require user interaction.

Custom token exchange as a last resort.

If you must support legacy systems that send username/password, implement a custom token exchange endpoint on your authorization server that accepts credentials and issues tokens. This keeps the anti-pattern in one controlled place while you plan a proper migration.

Scope Check

Search for `authorization-grant-type: password` in YAML/properties files and `AuthorizationGrantType.PASSWORD` in Java code. Also search for `ResourceOwnerPasswordResourceDetails` which was the OAuth 2.0 password grant helper class. Any hit means that authentication flow needs redesigning.

Watch Out

- The grant is gone from the OAuth 2.1 specification itself, so your authorization server (Keycloak, Auth0, Okta) may drop support too. Check both sides.
- Mobile apps that used password grant to avoid browser redirects should migrate to Authorization Code with PKCE, which modern mobile OAuth SDKs support natively.
- Integration tests that authenticate by posting username/password to the token endpoint will break. Update test helpers to use a test-specific grant flow or mock the security context directly.

Verify

Token endpoint returns 200 for auth code flow (not password grant)

Further Info

Removal tracked in spring-security#6003. Affects spring-boot-starter-oauth2-client consumers. See also: pkce-mandatory.

Links

- spring-break module: oauth-password-grant-removed
- OAuth Password Grant removal issue
- Spring Security 7 migration
- Spring Boot 4.0 Migration Guide

OTLP/HTTP Now Primary Export Protocol

Tier 2 - Won't Start Impact: S

Spring Boot 4.0 changed the default OTLP transport from gRPC to HTTP/protobuf. Exports aimed at gRPC-only collectors get connection refused or vanish without an error.

What You'll See

```
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans. Server responded with HTTP status 404.
  url=http://otel-collector:4318/v1/traces
---
# Or if collector only exposes gRPC port 4317:
WARN [otel.exporter] io.opentelemetry.exporter.otlp.http.OtlpHttpSpanExporter:
  Failed to export spans.
  java.net.ConnectException: Connection refused
  target=http://otel-collector:4318/v1/traces
---
# Metrics and traces stop arriving in your backend.
# Application continues running but observability data is lost.
```

What Changed

Spring Boot 4.0 changed the default OTLP export transport from gRPC (port 4317) to HTTP/protobuf (port 4318). The default endpoint changed from `http://localhost:4317` to `http://localhost:4318/v1/traces` (and `/v1/metrics`, `/v1/logs`). Applications that relied on the default gRPC transport now send data to the wrong port and protocol.

Why

HTTP/protobuf is more universally supported, works through proxies and load balancers without special gRPC configuration, and does not require the gRPC dependency. The OpenTelemetry specification recommends HTTP/protobuf as the default transport.

The Fix

application.properties — keep using gRPC
Before

```
# default was gRPC on port 4317
```

After

```
management.otlp.tracing.transport=grpc
management.otlp.tracing.endpoint=http://otel-collector:4317
management.otlp.metrics.export.transport=grpc
management.otlp.metrics.export.endpoint=http://otel-collecto
```

application.properties — accept new HTTP default
Before

```
management.otlp.tracing.endpoint=http://otel-collector:4317
```

After

```
management.otlp.tracing.endpoint=http://otel-collector:4318/
```

docker-compose.yml — expose HTTP port on collector
Before

```
otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"   # gRPC
```

After

```
otel-collector:
  image: otel/opentelemetry-collector:latest
  ports:
    - "4317:4317"   # gRPC
    - "4318:4318"   # HTTP/protobuf
```

How To Fix

Accept HTTP/protobuf (recommended).

Update your OTLP endpoint to use port 4318 and the HTTP path format. Ensure your collector exposes the HTTP receiver. Most modern collectors (OpenTelemetry Collector, Grafana Agent, Datadog Agent) support both protocols.

Explicitly set gRPC transport.

If your collector only supports gRPC, add `management.otlp.tracing.transport=grpc` and set the endpoint to the gRPC port. This preserves the old behaviour.

Update collector configuration.

If using the OpenTelemetry Collector, ensure both the `otlp/grpc` and `otlp/http` receivers are enabled in your collector config. This lets you support both old and new applications during migration.

Scope Check

Check your `application.properties` for `management.otlp` settings. If you have explicit endpoint configuration pointing to port 4317, the application will try HTTP on a gRPC port. If you have no explicit configuration, the default changed from gRPC/4317 to HTTP/4318.

Watch Out

- The application starts and runs normally. You only notice when traces and metrics stop appearing in your observability backend. Add health checks for telemetry export.
- If you use a service mesh or network policy that only allows port 4317, you need to open port 4318 as well.
- Some managed observability platforms (Datadog, New Relic) have different endpoints for HTTP vs gRPC OTLP ingestion. Check your provider's documentation for the correct HTTP endpoint URL.
- The gRPC dependency (`io.grpc:grpc-netty-shaded`) may no longer be pulled in transitively. If you switch back to gRPC, you may need to add it explicitly.

Verify

OTLP exporter sends traces to collector (check collector logs)

Further Info

Affects spring-boot-starter-actuator consumers using OTLP tracing or metrics export.

Links

- Spring Boot 4.0 Migration Guide

PKCE Mandatory for Confidential OAuth Clients

Tier 2 - Won't Start Impact: M

Spring Security 7 enforces PKCE for all OAuth 2.0 clients, including confidential clients. Authorization requests without a code_challenge are rejected by the authorization server.

What You'll See

```
[oauth2-client] o.s.s.oauth2.client.web.OAuth2LoginAuthenticationFilter:
  Authentication request failed:
org.springframework.security.oauth2.core.OAuth2AuthenticationException:
  [invalid_request] PKCE code_challenge is required for this client.
 at org.springframework.security.oauth2.client.oidc.authentication
  .OidcAuthorizationCodeAuthenticationProvider.authenticate(...)
---
HTTP 302 → /login?error=invalid_request
---
Browser shows: "OAuth2 Error: [invalid_request]
PKCE code_challenge is required for this client."
```

What Changed

Spring Security 7 now sends PKCE parameters (`code_challenge` and `code_challenge_method`) on every Authorization Code request, for both public and confidential clients. If the authorization server requires PKCE but the client was not sending it (pre-upgrade behaviour for confidential clients), or if the authorization server rejects unexpected PKCE parameters, the flow breaks.

Why

OAuth 2.1 (RFC 9700) mandates PKCE for all clients, public and confidential alike, because PKCE prevents authorization code interception attacks regardless of client type. Spring Security aligned with that requirement.

The Fix

> application.yml — no changes needed on client side
> Before
>
> ```yaml
> spring:
>   security:
>     oauth2:
>       client:
> ```

```
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile
```

After

```
# Client config stays the same - Spring Security 7 adds PKCE
# But your authorization server MUST accept PKCE parameters.
spring:
  security:
    oauth2:
      client:
        registration:
          my-app:
            authorization-grant-type: authorization_code
            client-id: my-confidential-client
            client-secret: ${CLIENT_SECRET}
            scope: openid,profile
```

Keycloak realm config — enable PKCE for confidential client
Before

```
"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": ""
```

After

```
"clientId": "my-confidential-client",
"publicClient": false,
"pkceCodeChallengeMethod": "S256"
```

Disable PKCE enforcement if auth server can't support it (temporary)
Before

```
// default PKCE behaviour
```

After

```
@Bean
public OAuth2AuthorizationRequestResolver pkceResolver(
        ClientRegistrationRepository repo) {
```

```
    DefaultOAuth2AuthorizationRequestResolver resolver =
        new DefaultOAuth2AuthorizationRequestResolver(
            repo, "/oauth2/authorization");
    // Temporary: remove when auth server supports PKCE
    resolver.setAuthorizationRequestCustomizer(
        OAuth2AuthorizationRequestCustomizers.withoutPkce())
    return resolver;
}
```

How To Fix

Enable PKCE on your authorization server.

Configure your authorization server (Keycloak, Auth0, Okta, Azure AD) to accept PKCE `code_challenge` parameters for confidential clients. Most modern providers support this; it may need enabling per client. Use `S256` as the challenge method.

Verify no auth server rejects extra parameters.

Some older or custom authorization servers reject unknown parameters. If yours rejects `code_challenge`, you need to update the auth server first, or temporarily disable PKCE enforcement on the Spring Security side using a custom resolver.

Disable PKCE temporarily (not recommended).

If you cannot update the authorization server immediately, register a custom `OAuth2AuthorizationRequestResolver` that strips PKCE parameters. Treat this as technical debt and plan the auth server update.

Scope Check

Check every OAuth 2.0 client registration in your application. For each one, verify that the corresponding authorization server accepts PKCE parameters. If you use Spring Authorization Server, it already supports PKCE. If you use a third-party provider, check their documentation.

Watch Out

- The failure often appears only in the browser redirect flow. Automated tests that mock the OAuth exchange may not trigger the error. Test with a real authorization server.
- If you use Spring Authorization Server as your auth server and it is also being upgraded, both sides get PKCE support automatically. The risk is when the client (Spring Boot 4) and the auth server (third-party or older version) are upgraded independently.
- Azure AD and some ADFS versions may need tenant-level configuration changes to allow PKCE for confidential clients. Check your identity provider's documentation.

Verify

OAuth2 login completes with PKCE challenge in the auth request

Further Info

PKCE enforcement was tracked in spring-security#16391. Affects spring-boot-starter-oauth2-client consumers using the authorization_code grant. See also: oauth-password-grant.

Links

- spring-break module: pkce-mandatory
- PKCE enforcement issue
- Spring Security 7 migration
- Spring Boot 4.0 Migration Guide

Spring Pulsar Reactive Auto-Configuration Removed

Tier 2 - Won't Start Impact: M

Spring Boot 4.0 dropped reactive Pulsar auto-configuration. `ReactivePulsarTemplate` and `ReactivePulsarClient` beans are no longer available. The app starts but fails on first use.

What You'll See

```
// Boot 4.0 startup or runtime failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'org.springframework.pulsar.reactive.core.ReactivePulsarTemplate' available

// Or auto-wiring failure:
Field reactivePulsarTemplate required a bean of type
  'ReactivePulsarTemplate' that could not be found.
```

What Changed

`spring-boot-starter-pulsar-reactive` and the corresponding auto-configuration module were removed from Spring Boot 4.0. The auto-configuration for `ReactivePulsarClient`, `ReactivePulsarTemplate`, and `ReactiveMessageListenerContainer` no longer exists in Boot's auto-configure artifact.

Why

Boot 4.0's modular restructuring dropped auto-configurations for reactive integration layers with limited adoption relative to their maintenance cost. The imperative Spring Pulsar integration remains fully supported.

The Fix

Replace reactive auto-config with explicit bean definitions
Before

```
<!-- Boot 3.5: auto-configured reactive Pulsar client -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-pulsar-reactive</artifac
</dependency>
```

After

```
<!-- Boot 4.0: use imperative starter or configure reactive
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-pulsar</artifactId>
</dependency>
```

How To Fix

Switch to the imperative Pulsar integration or configure reactive beans manually.

If the reactive API is required, declare the reactive Pulsar beans explicitly in a `@Configuration` class. If reactive messaging is not critical, switch to the imperative `spring-boot-starter-pulsar` which remains auto-configured in Boot 4.0.

Scope Check

Grep for `ReactivePulsarTemplate`, `ReactivePulsarClient`, and `spring-boot-starter-pulsar-reactive` across your source and build files.

Watch Out

- The imperative `PulsarTemplate` and `PulsarClient` auto-configuration in `spring-boot-starter-pulsar` is unaffected by this change.

Verify

Reactive Pulsar producers and consumers initialise correctly after removing the dependency on Boot's auto-configuration

Further Info

The Spring Pulsar reactive library itself survives; only Boot's auto-configuration went away, so the beans must be declared by hand.

Links

- Spring Boot 4.0 Migration Guide
- Spring Pulsar Documentation

Spring Boot Spock Integration Removed

Tier 2 - Won't Start Impact: L

Spring Boot 4.0 requires Groovy 5; Spock does not support it yet. Spock-based `@SpringBootTest` specifications stop running.

What You'll See

```
// Spock specification in a Boot 4.0 project:
@SpringBootTest
class MyServiceSpec extends Specification {
    @Autowired MyService service

    def "should do something"() {
        expect: service.doThing() == "result"
    }
}

// Fails at compile or runtime — Spock and Groovy 5 are incompatible.
// Exact error depends on Groovy/Spock versions on the classpath.
```

What Changed

Spring Boot 4.0 upgraded its Groovy baseline to Groovy 5. Spock Framework (as of this writing) requires Groovy 4.x or earlier, so Spring Boot removed the Spock integration.

Why

The Groovy 5 upgrade aligned Boot with the current Groovy ecosystem. Spock's Groovy 5 support was not ready in time for the Boot 4.0 release, making the integration impossible to ship.

The Fix

> Spock specification → JUnit 5 migration
> Before
>
> ```groovy
> // MyServiceSpec.groovy
> @SpringBootTest
> class MyServiceSpec extends Specification {
>     @Autowired MyService service
>
>     def "should return result"() {
> ```

```
        expect: service.doThing() == "result"
    }
}
```

After

```
// MyServiceTest.java
@SpringBootTest
class MyServiceTest {
    @Autowired MyService service;

    @Test
    void shouldReturnResult() {
        assertThat(service.doThing()).isEqualTo("result");
    }
}
```

How To Fix

Migrate Spock specifications to JUnit 5.

Rewrite Spock-based Spring integration tests using JUnit 5 and AssertJ. The `given/when/then` blocks map to JUnit 5 methods and AssertJ assertions. Data-driven tests (`where:` blocks) translate to `@ParameterizedTest` with `@MethodSource` or `@CsvSource`.

Wait for Spock's Groovy 5 support if migration is impractical.

If the volume of Spock tests makes immediate migration impractical, track the Spock project's Groovy 5 roadmap and defer the Boot 4.0 upgrade for those modules.

Scope Check

Find all `.groovy` test files that extend `Specification` and carry `@SpringBootTest`, `@SpringRunner`, or any Spring test slice annotation. These are the tests that break.

Watch Out

- Pure unit Spock tests, those that never load a Spring context, still work as long as Groovy compilation itself succeeds. The breakage is in tests that depend on Spring Boot's test infrastructure.

Verify

Spock specifications that use @SpringBootTest or @SpringRunner pass after migrating to JUnit 5 or waiting for Spock's Groovy 5 support

Further Info

The removal covers the `spock-spring` module and Boot's `@SpringBootTest` support for Spock specifications. Failures appear at Groovy compile time or at runtime before any test method executes.

Links

- Spring Boot 4.0 Migration Guide
- Spock Framework GitHub

Spring Session Hazelcast Auto-Configuration Removed

Tier 2 - Won't Start Impact: M

Spring Boot 4.0 dropped built-in Hazelcast session auto-configuration. The app compiles but fails at startup with `NoSuchBeanDefinitionException`.

What You'll See

```
// Boot 4.0 startup failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'org.springframework.session.SessionRepository' available

// Or context load failure mentioning missing HazelcastIndexedSessionRepository
```

What Changed

Spring Boot's auto-configuration for Hazelcast-backed Spring Session was removed from `spring-boot-autoconfigure`. The classes `HazelcastSessionConfiguration` and `HazelcastSessionProperties` no longer exist in the Spring Boot artifact. The Hazelcast team took ownership of the integration.

Why

Boot 4.0 handed several Spring Session store integrations to the teams closest to the technology. Hazelcast and MongoDB now maintain their own, keeping Boot lean and letting each integration track its store's release cycle.

The Fix

pom.xml — add Hazelcast team's Spring Session integration
Before

```
<!-- Boot 3.5: built-in, no extra dependency needed beyond s
<dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-hazelcast</artifactId>
</dependency>
```

After

```
<!-- Boot 4.0: use Hazelcast team's own integration artifact
<dependency>
    <groupId>com.hazelcast</groupId>
    <artifactId>hazelcast-spring-session</artifactId>
    <version><!-- check Hazelcast release --></version>
</dependency>
```

How To Fix

Migrate to the Hazelcast-maintained Spring Session integration.

Replace `spring-session-hazelcast` with the artifact published by the Hazelcast team and configure it from Hazelcast's own documentation.

Scope Check

Check `spring.session.store-type=hazelcast` in your properties. Grep for `HazelcastIndexedSessionRepository` and `HazelcastSessionConfiguration` in your source.

Watch Out

- Property keys and bean names may differ in the Hazelcast-maintained integration. Verify them against Hazelcast's own documentation rather than the Spring Boot migration guide.

Verify

Hazelcast-backed session store initialises correctly after migrating to the Hazelcast-team-maintained integration

Further Info

In Boot 3.5, `spring-session-hazelcast` on the classpath plus `spring.session.store-type=hazelcast` was enough: Boot wired the session store for you. That wiring is what disappeared.

Links

- Spring Boot 4.0 Migration Guide
- Hazelcast Spring Session Documentation

Spring Session MongoDB Auto-Configuration Removed

Tier 2 - Won't Start Impact: M

Spring Boot 4.0 dropped built-in MongoDB session auto-configuration. The app compiles but fails at startup with `NoSuchBeanDefinitionException`.

What You'll See

```
// Boot 4.0 startup failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'org.springframework.session.SessionRepository' available

// Or context load failure mentioning missing MongoIndexedSessionRepository
```

What Changed

Spring Boot's auto-configuration for MongoDB-backed Spring Session was removed from `spring-boot-autoconfigure`. The `MongoSessionConfiguration` and related auto-configuration classes no longer exist in the Spring Boot artifact. Spring Data MongoDB took ownership of the integration.

Why

Boot 4.0 handed several Spring Session store integrations to the teams closest to the technology. MongoDB and Hazelcast now maintain their own, keeping Boot lean and letting each integration track its store's release cycle.

The Fix

pom.xml — switch to Spring Data MongoDB's own session support
Before

```
<!-- Boot 3.5: built-in auto-config via spring-session-data-
<dependency>
    <groupId>org.springframework.session</groupId>
    <artifactId>spring-session-data-mongodb</artifactId>
</dependency>
```

After

```
<!-- Boot 4.0: session support is part of spring-boot-starte
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId
</dependency>
<!-- Refer to Spring Data MongoDB docs for explicit session
```

How To Fix

Migrate to the Spring Data MongoDB session integration.

The MongoDB team's Spring Session support is bundled with Spring Data MongoDB. Add `spring-boot-starter-data-mongodb` and configure the session repository according to Spring Data MongoDB's documentation. The `spring.session.store-type=mongodb` property alone is no longer sufficient.

Scope Check

Check `spring.session.store-type=mongodb` in your properties. Grep for `MongoIndexedSessionRepository` and `MongoSessionConfiguration` in your source.

Watch Out

- MongoDB session property keys changed: `spring.session.mongodb.` *became* *`spring.session.data.mongodb.`*. This is a separate but related break covered in the session property renames card.

Verify

MongoDB-backed session store initialises correctly after migrating to the MongoDB-team-maintained integration

Further Info

In Boot 3.5, `spring-session-data-mongodb` on the classpath plus `spring.session.store-type=mongodb` was enough: Boot wired the session repository for you. That wiring is what disappeared; the integration now ships as part of Spring Data MongoDB.

Links

- Spring Boot 4.0 Migration Guide

@SpringBootTest No Longer Auto-Configures MockMvc

Tier 2 - Won't Start Impact: S

`@SpringBootTest` no longer auto-configures `MockMvc` in Boot 4.0. Add `@AutoConfigureMockMvc` to your test class or the `MockMvc` bean won't be available.

What You'll See

```
@SpringBootTest
class MyControllerTest {
    @Autowired
    MockMvc mockMvc;  // null in Boot 4.0

    @Test
    void test() throws Exception {
        mockMvc.perform(get("/api/hello"))  // NullPointerException
               .andExpect(status().isOk());
    }
}

// Boot 4.0 failure:
org.springframework.beans.factory.NoSuchBeanDefinitionException:
  No qualifying bean of type
  'org.springframework.test.web.servlet.MockMvc' available
```

What Changed

MockMvc auto-configuration was decoupled from `@SpringBootTest`. The MOCK webEnvironment still creates a mock servlet context, but the `MockMvc` bean must now be opted into with `@AutoConfigureMockMvc`.

Why

The implicit bean was a hidden dependency: tests got MockMvc without declaring they needed it. Opt-in matches Boot 4.0's explicit-over-implicit theme and keeps the test context leaner when MockMvc isn't needed.

The Fix

Add @AutoConfigureMockMvc
Before

```
@SpringBootTest
class MyControllerTest {
    @Autowired
    MockMvc mockMvc;
}
```

After

```
@SpringBootTest
@AutoConfigureMockMvc
class MyControllerTest {
    @Autowired
    MockMvc mockMvc;
}
```

@WebMvcTest is unaffected — MockMvc is its primary purpose
Added

```
// @WebMvcTest already includes MockMvc auto-configuration.
// Only @SpringBootTest is affected.
```

How To Fix

Add @AutoConfigureMockMvc to the test class.

This is a one-annotation fix. Add `@AutoConfigureMockMvc` alongside `@SpringBootTest` on any test class that autowires `MockMvc`.

Scope Check

Search for `@SpringBootTest` test classes that also autowire `MockMvc` (look for `@Autowired` fields or constructor parameters of type `MockMvc`). Each needs `@AutoConfigureMockMvc` added. `@WebMvcTest` tests are unaffected.

Watch Out

- The same change applies to `WebTestClient` and `TestRestTemplate`. Boot 4.0 also no longer auto-configures these in `@SpringBootTest`: add `@AutoConfigureRestTestClient` or `@AutoConfigureTestRestTemplate` respectively.

Verify

MockMvc is available in @SpringBootTest tests after adding @AutoConfigureMockMvc

Further Info

Per the migration guide, a Boot 3.5 @SpringBootTest with the default MOCK webEnvironment supplied MockMvc without any extra annotation; tests relying on that fail with NoSuchBeanDefinitionException on 4.0.

Links

- spring-break module: springboottest-mockmvc-autoconfig
- Spring Boot 4.0 Migration Guide

Tier 3

Tier 3 - Behaves Differently

16 cards

DevTools Live Reload Disabled by Default

Tier 3 - Behaves Differently Impact: S

Boot 4.0 disables DevTools Live Reload by default: no automatic browser refresh after code changes. Set `spring.devtools.livereload.enabled=true` in your dev profile to restore it.

What You'll See

```
# Boot 3.5: live reload works automatically with devtools on classpath.
# Boot 4.0: live reload disabled. Browser does not refresh.
# No error: the server restarts but the browser stays on the old page.
```

What Changed

The `spring.devtools.livereload.enabled` property now defaults to `false`. The Live Reload server is not started unless this property is explicitly set to `true`.

Why

Live Reload opens a port (35729) on the developer's machine and needs a browser extension. Starting it unasked surprised developers who did not know about or want it. Opt-in gives explicit control over which DevTools features are active.

The Fix

application-dev.properties or application.properties
Before

```
# Boot 3.5: live reload on by default, no property needed
```

After

```
# Boot 4.0: must opt in
spring.devtools.livereload.enabled=true
```

How To Fix

Add spring.devtools.livereload.enabled=true to your dev profile.

Add the property to `application-dev.properties` or your development-time configuration. Keep it out of production profiles: DevTools should not be on the production classpath anyway, and scoping the property to dev makes the intent clear.

Scope Check

Check `application.properties` and `application-dev.properties` for any existing DevTools configuration.

Watch Out

- The DevTools automatic restart (triggered by classpath changes) is separate from Live Reload and unaffected. The app still restarts; the browser just does not receive the push notification to refresh.

Verify

Browser refreshes automatically after code changes when spring.devtools.livereload.enabled=true is set in development profile

Further Info

In Boot 3.5, adding `spring-boot-devtools` to the classpath automatically started a Live Reload server on port 35729, which pushed browser refresh signals after application restarts.

Links

- Spring Boot 4.0 Migration Guide
- Spring Boot DevTools Documentation

Hibernate Native Query Date Return Types Changed

Tier 3 - Behaves Differently Impact: M

Native SQL queries now return `java.time.LocalDate` instead of `java.sql.Date`. Code casting to the old types throws ClassCastException or silently loses time components.

What You'll See

```
// Native query — same code, different result type
Object result = em.createNativeQuery("SELECT created_date FROM orders WHERE id = 1")
                  .getSingleResult();

// Before (Spring Boot 3.5)
result.getClass() → java.sql.Date

// After (Spring Boot 4.0)
result.getClass() → java.time.LocalDate

// ClassCastException at runtime
java.lang.ClassCastException:
  java.time.LocalDate cannot be cast to java.sql.Date

// Or subtler: time component silently lost
java.sql.Timestamp ts = (java.sql.Timestamp) result;  // was Timestamp
// Now: java.time.LocalDateTime — no getTime() method
```

What Changed

Hibernate 7 (used by Spring Boot 4.0) changed the default Java types returned for SQL `DATE`, `TIME`, and `TIMESTAMP` columns in native queries. `java.sql.Date` became `java.time.LocalDate`, `java.sql.Time` became `java.time.LocalTime`, and `java.sql.Timestamp` became `java.time.LocalDateTime`.

Why

The `java.sql` date/time classes are legacy wrappers around `java.util.Date` with known timezone and truncation bugs. Hibernate aligned its defaults with `java.time`: immutable, thread-safe, the modern standard.

The Fix

> Fix native query result handling
> Before

```
java.sql.Date date = (java.sql.Date) row[2];
long millis = date.getTime();
```

After

```
LocalDate date = (LocalDate) row[2];
long millis = date.atStartOfDay(ZoneOffset.UTC)
                  .toInstant().toEpochMilli();
```

Typed native query
Before

```
@Query(value = "SELECT created_date FROM orders", nativeQuer
List<java.sql.Date> findAllDates();
```

After

```
@Query(value = "SELECT created_date FROM orders", nativeQuer
List<java.time.LocalDate> findAllDates();
```

Timestamp results
Before

```
java.sql.Timestamp ts = (java.sql.Timestamp) row[3];
```

After

```
LocalDateTime ts = (LocalDateTime) row[3];
```

How To Fix

Update result type casts to java.time (recommended).

Replace all `java.sql.Date` casts with `java.time.LocalDate`, `java.sql.Time` with `java.time.LocalTime`, and `java.sql.Timestamp` with `java.time.LocalDateTime`.

Force old types with explicit addScalar().

For legacy code that can't be updated, use Hibernate's `addScalar("col", StandardBasicTypes.DATE)` to force the old `java.sql` return types on specific queries.

Scope Check

Search for `nativeQuery = true`, `createNativeQuery`, and any cast to `java.sql.Date`, `java.sql.Time`, or `java.sql.Timestamp` in query result handling code.

Watch Out

- This only affects native queries that return raw `Object[]` rows. JPQL queries with entity mappings or `@ColumnResult` type mappings are unaffected because the type is explicit.
- `java.time.LocalDate` has no time component. If your code relied on `java.sql.Date.getTime()` returning midnight epoch millis, the replacement requires an explicit timezone conversion.

Verify

SELECT queries return correct date/time types (check ResultSet)

Further Info

Driven by Hibernate 7.0, upstream of Spring Boot 4.0, and documented in its migration guide. See also: hibernate-dialect-removal.

Links

- spring-break module: hibernate-native-datetime
- Spring Boot 4.0 Migration Guide

Jackson Date Serialisation Flip

Tier 3 - Behaves Differently Impact: M

Jackson 3.0 defaults to ISO-8601 date strings instead of numeric timestamps. Every API response containing a date silently changes shape.

What You'll See

```
// API response — before (Spring Boot 3.5)
{"created": 1714089600000}

// API response — after (Spring Boot 4.0)
{"created": "2025-04-26T00:00:00Z"}

// Front-end JavaScript blows up
TypeError: date.getTime is not a function
// or: contract test fails
Expected: <1714089600000> (Long)
  Actual: <"2025-04-26T00:00:00Z"> (String)
```

What Changed

Jackson 3.0 changed the default value of `SerializationFeature.WRITE_DATES_AS_TIMESTAMPS` from `true` to `false`. Dates, times, and durations now serialise as ISO-8601 strings by default instead of numeric epoch milliseconds.

Why

ISO-8601 strings are human-readable, timezone-aware, and the de facto standard in JSON APIs. The old numeric default caused constant confusion about whether the value was seconds or milliseconds, and lost timezone information entirely.

The Fix

application.properties — restore old behaviour
Before

```
# (no explicit setting needed in Boot 3.5)
```

After

```
spring.jackson.serialization.write-dates-as-timestamps=true
```

or adopt ISO-8601 and fix the client
Before

```
const ts = new Date(response.created);  // was a number
```

After

```
const ts = new Date(response.created);  // now parses ISO st
```

ObjectMapper explicit configuration
Before

```
// Jackson 2 default: timestamps enabled
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
```

After

```
// Jackson 3 default: ISO strings
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());
// only if you need the old numeric format:
mapper.enable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS
```

How To Fix

Opt in to the new ISO-8601 default (recommended).

Update your API clients and contract tests to expect ISO-8601 strings. This is the better long-term format. Coordinate the change with front-end teams so they parse the string, not a number.

Restore numeric timestamps.

Add `spring.jackson.serialization.write-dates-as-timestamps=true` to `application.properties`. This gives you the old behaviour while you plan the migration.

Scope Check

Search for any DTO field of type `java.util.Date`, `Instant`, `LocalDate`, `LocalDateTime`, `ZonedDateTime`, or `Duration`. Every one of those fields will serialise differently after the upgrade. Check API consumers, contract tests, and front-end parsing code.

Watch Out

- This change is invisible at compile time and at startup. The app runs perfectly until a client tries to parse the date field as a number. Without contract tests, you won't catch this before production.
- The `@JsonFormat` annotation on individual fields still overrides the global default. Annotated dates are unaffected; the unannotated ones flip.

Verify

JSON responses show dates in expected format (check API output)

Further Info

Driven by Jackson 3.0, upstream of Spring Boot 4.0, and announced in its release notes. See also: jackson-dates-timestamps, jackson-property-inclusion.

Links

- spring-break module: jackson-date-serialisation
- Jackson 3 migration guide
- Jackson 3 in Spring (blog)

Jackson Locale Format Change (BCP 47)

Tier 3 - Behaves Differently Impact: S

Jackson 3.0 serialises `Locale` using IETF BCP 47 format (zh-CN) instead of Java's underscore format (zh_CN). Locale-matching logic silently breaks.

What You'll See

```
// API response — before (Spring Boot 3.5)
{"userLocale": "zh_CN"}

// API response — after (Spring Boot 4.0)
{"userLocale": "zh-CN"}

// Downstream lookup fails
Map<String, String> labels = i18nMap.get(user.getLocale());
// returns null — map keys use "zh_CN", response now sends "zh-CN"

// Test failure
org.opentest4j.AssertionFailedError:
Expected: "zh_CN"
  Actual: "zh-CN"
```

What Changed

Jackson 3.0 switched `Locale` serialisation from Java's `toString()` format (`zh_CN`) to the IETF BCP 47 language tag format (`zh-CN`). Underscores become hyphens and the structure follows RFC 5646.

Why

BCP 47 is the web standard used by HTTP `Accept-Language` headers, HTML `lang` attributes, and JavaScript's `Intl` API. Aligning Jackson's output with the rest of the web ecosystem removes a constant source of mismatch bugs.

The Fix

> Custom serialiser to restore old format
> Before
>
> ```
> // No custom serialiser needed in Boot 3.5
> ```
>
> After

```
@JsonSerialize(using = JavaLocaleSerializer.class)
private Locale userLocale;
```

Or update all consumers to use BCP 47
Before

```
String localeKey = user.getLocale(); // "zh_CN"
```

After

```
String localeKey = Locale.forLanguageTag(user.getLocale()).t
```

Fix i18n lookup maps
Before

```
i18nMap.put("zh_CN", chineseLabels);
```

After

```
i18nMap.put("zh-CN", chineseLabels);
```

How To Fix

Adopt BCP 47 throughout (recommended).

Update your i18n maps, database locale columns, and downstream consumers to use hyphenated BCP 47 tags. This aligns with HTTP headers and browser APIs. Run a data migration for persisted locale strings.

Write a custom Locale serialiser.

Register a custom Jackson serialiser that calls `locale.toString()` instead of `locale.toLanguageTag()`. This preserves the old underscore format while you plan the broader migration.

Scope Check

Search for any DTO or entity field of type `java.util.Locale` that gets serialised to JSON. Also check database columns, message queues, and i18n resource bundle keys that use the underscore format. Every one of those is a potential mismatch.

Watch Out

- If you store locale strings in a database and compare them with equality checks, the format flip causes mismatches that surface as missing data, not errors. A `SELECT WHERE locale = 'zh_CN'` query won't match the new `zh-CN` value sent by the API.
- BCP 47 has subtag rules beyond swapping underscores for hyphens. Some locales have three components (e.g. `zh_Hant_TW` becomes `zh-Hant-TW`). Use `Locale.forLanguageTag()` for correct round-tripping.

Verify

Locale-sensitive fields render correctly in all target locales

Further Info

Driven by Jackson 3.0, upstream of Spring Boot 4.0. See also: jackson-date-format, jackson-group-id.

Links

- spring-break module: jackson-locale-format
- Jackson 3 migration guide
- IETF BCP 47 (RFC 5646)

Jackson 3 Auto-Discovers All Modules on Classpath

Tier 3 - Behaves Differently Impact: S

Jackson 3 registers every module it finds on the classpath. Modules that arrive as transitive dependencies can silently change your JSON output. Disable with `spring.jackson.find-and-add-modules=false`.

What You'll See

```
// Boot 3.5: only well-known modules registered. Output is predictable.
{"amount": 150.00, "currency": "EUR"}

// Boot 4.0: a transitive Jackson module on the classpath changes
// BigDecimal serialization:
{"amount": "150.00", "currency": "EUR"}

// No error, no warning. The JSON schema changed.
```

What Changed

Jackson 3 uses the Java ServiceLoader mechanism to find and register all `com.fasterxml.jackson.databind.Module` implementations on the classpath. Spring Boot no longer controls which modules are registered: every module present is activated.

Why

Automatic discovery lets library authors ship Jackson integration without requiring explicit registration. Spring Boot aligned with Jackson 3's convention.

The Fix

application.properties — disable auto-discovery to restore Boot 3.5 behaviour

Before

```
# Boot 3.5: no property needed, only well-known modules regi
```

After

```
# Boot 4.0: disable auto-discovery to prevent unexpected mod
spring.jackson.find-and-add-modules=false
```

How To Fix

Audit classpath modules and disable auto-discovery if needed.

Run `mvn dependency:tree` and look for any artifacts ending in `-module` or containing `jackson`. For each one, verify its effect on your serialization output. If unexpected modules are activating, set `spring.jackson.find-and-add-modules=false` and register only the modules you need explicitly.

Explicitly register required modules.

After disabling auto-discovery, register required modules via a `Jackson2ObjectMapperBuilderCustomizer` (now renamed `JsonMapperBuilderCustomizer`) bean that calls `modules(new JavaTimeModule(), ...)`.

Scope Check

Run your application's serialization tests after upgrading. Compare JSON output field by field for any type that uses custom serialization or has complex types (dates, BigDecimal, Optional, collections). Grep for `ObjectMapper` customization code to understand what was previously registered manually.

Watch Out

- This interacts with the Jackson 2 compatibility shim (`spring-boot-jackson2`). If you have both Jackson 2 and Jackson 3 modules on the classpath, both may be auto-discovered and conflict.
- Test suites that stub or mock the `ObjectMapper` will not catch this: the change only manifests with a real, auto-configured `ObjectMapper` bean.

Verify

JSON serialization output is identical before and after the upgrade, or unexpected module behaviour is controlled with spring.jackson.find-and-add-modules

Further Info

In Boot 3.5 (Jackson 2), Spring Boot registered a fixed set of well-known modules: Java time, parameter names, and a handful of others. Everything else needed explicit registration. A module a library brought in for its own internal use can now alter your own `ObjectMapper` output.

Links

- Spring Boot 4.0 Migration Guide

spring.jackson.default-property-inclusion Silently Ignored

Tier 3 - Behaves Differently Impact: M

The `spring.jackson.default-property-inclusion` property no longer configures Jackson 3's ObjectMapper. Null fields reappear in your API responses without warning.

What You'll See

```
# application.properties (still present, no startup warning)
spring.jackson.default-property-inclusion=non_null

// API response — before (Spring Boot 3.5)
{"name": "Alice", "age": 30}

// API response — after (Spring Boot 4.0)
{"name": "Alice", "age": 30, "middleName": null, "nickname": null}

// Contract test failure
org.opentest4j.AssertionFailedError:
Expected JSON keys: [name, age]
  Actual JSON keys: [name, age, middleName, nickname]
```

What Changed

Spring Boot 4.0's auto-configuration for Jackson 3 no longer reads the `spring.jackson.default-property-inclusion` property. The property sits in your config file doing nothing: no deprecation warning, no startup error. The ObjectMapper reverts to Jackson's default inclusion policy: `ALWAYS` (include everything, including nulls).

Why

Jackson 3 moved inclusion configuration from a single global enum to a granular per-type system. The old Spring Boot property didn't map cleanly to the new API, so the binding was removed pending a redesign.

The Fix

application.properties — old (silently ignored)
Before

```
spring.jackson.default-property-inclusion=non_null
```

After

```
# Removed - configure via ObjectMapper bean instead
```

ObjectMapper customiser bean
Before

```
# (relied on spring.jackson.default-property-inclusion)
```

After

```
@Bean
public Jackson2ObjectMapperBuilderCustomizer nonNullInclusio
    return builder -> builder
        .serializationInclusion(JsonInclude.Include.NON_NULL
}
```

Or per-class annotation
Before

```
public class UserDto {
```

After

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public class UserDto {
```

How To Fix

Register an ObjectMapper customiser bean (recommended).

Create a `Jackson2ObjectMapperBuilderCustomizer` bean that calls `serializationInclusion(NON_NULL)`. This replaces the property-based configuration and works with Jackson 3.

Annotate DTOs individually.

Add `@JsonInclude(JsonInclude.Include.NON_NULL)` to each DTO class. More verbose, but gives you per-class control and makes the contract explicit in the code.

Scope Check

Check whether your `application.properties` or `application.yml` sets `spring.jackson.default-property-inclusion`. If it does, that setting is now dead.

Watch Out

- The only signal is larger JSON payloads with unexpected null fields. Without contract tests, that is easy to miss.
- If your front-end checks for key presence (e.g. `if ("middleName" in user)`) instead of checking for null values, the extra null fields change its branching logic.

Verify

JSON output excludes null fields where expected (compare before/after)

Further Info

Driven by the combination of Jackson 3.0 and Spring Boot 4.0's auto-configuration changes. See also: jackson-dates-timestamps, jackson-date-format.

Links

- Spring-Break Demo
- Spring Boot 4.0 Migration Guide
- Jackson 3 in Spring (blog)

Liveness and Readiness Probes Enabled by Default

Tier 3 - Behaves Differently Impact: S

Boot 4.0 enables liveness and readiness probes everywhere, Kubernetes or not. The `/actuator/health` response changes shape. Downstream parsers and security rules may break.

What You'll See

```
// Boot 3.5 (outside Kubernetes): /actuator/health response
{"status":"UP"}

// Boot 4.0: /actuator/health response includes probe groups
{
  "status": "UP",
  "groups": ["liveness", "readiness"]
}

// Consumer code that checked for exact JSON equality or did not
// expect the "groups" key now fails.
```

What Changed

The `management.endpoint.health.probes.enabled` property now defaults to `true` instead of being driven by Kubernetes detection. Both `/actuator/health/liveness` and `/actuator/health/readiness` are always exposed, and the main health response always lists them in the `groups` field.

Why

Probes are useful outside Kubernetes: load balancers, service meshes, and health-check scripts benefit from explicit liveness and readiness semantics. Making them universally available removes the need for environment-specific configuration.

The Fix

application.properties — disable probes if not wanted

Before

```
# Boot 3.5: probes off by default outside Kubernetes
# management.endpoint.health.probes.enabled=true  # required
```

After

```
# Boot 4.0: probes on by default; disable explicitly if not
management.endpoint.health.probes.enabled=false
```

How To Fix

Disable probes if your deployment does not use them.

Set `management.endpoint.health.probes.enabled=false` in `application.properties` to restore Boot 3.5 behaviour for non-Kubernetes deployments. Alternatively, update downstream health consumers to expect the `groups` field.

Update security rules if probe endpoints were unintentionally exposed.

Review your actuator security configuration. If `/actuator/health/liveness` and `/actuator/health/readiness` were not expected to be publicly accessible, add them to your security exclusions or require authentication.

Scope Check

Search for code or tests that parse the `/actuator/health` response body and check for exact JSON structure. Also review Actuator security configurations for endpoint exposure rules. Check monitoring integrations (Datadog, Prometheus, PagerDuty health checks) that consume the health endpoint.

Watch Out

- Application liveness state can be changed programmatically via `ApplicationAvailability`. If your code already does this, the probes are now always reachable: confirm the state transitions are correct before going to production.

Verify

Health endpoint responds correctly and liveness/readiness groups are present or absent as expected after reviewing probe configuration

Further Info

Boot 3.5 activated the probes only when it detected a Kubernetes environment (via environment variables) or when `management.endpoint.health.probes.enabled=true` was set explicitly. Boot 4.0 drops the detection, which can expose probe endpoints to networks that never expected them.

Links

- spring-break module: health-probes-default-on
- Spring Boot 4.0 Migration Guide
- Spring Boot Liveness and Readiness Probes

Logback Default Charset Changed to UTF-8

Tier 3 - Behaves Differently Impact: S

Boot 4.0 forces Logback file appenders to UTF-8. Non-ASCII characters appear garbled to tools expecting the platform's default encoding. No error: just wrong characters.

What You'll See

```
# Boot 3.5 (platform encoding = Windows-1252):
2025-01-01 INFO  Order confirmed: €150.00 for Müller

# Boot 4.0 (UTF-8 forced, consumed by tool expecting Windows-1252):
2025-01-01 INFO  Order confirmed: â,¬150.00 for MÃ¼ller
# Or if the terminal/tool can't decode UTF-8 sequences:
2025-01-01 INFO  Order confirmed: ?150.00 for M?ller
```

What Changed

Spring Boot's default Logback configuration changed the charset for file appenders to UTF-8. Console appenders now use `Console#charset()` (the JVM's console charset, Java 17+) rather than the platform default. The change affects the default Logback XML provided by Spring Boot; custom Logback configurations that already specify an explicit charset are unaffected.

Why

UTF-8 is the universal standard for text encoding. Platform-default encoding produced different bytes on different operating systems, making cross-platform log analysis unreliable.

The Fix

> logback-spring.xml — if you need to override the default
> Before
>
> ```
> <!-- Boot 3.5: no charset specified, platform default used f
> <appender name="FILE" class="ch.qos.logback.core.FileAppende
>     <file>app.log</file>
>     <encoder>
>         <pattern>%d{yyyy-MM-dd} %-5level %msg%n</pattern>
>     </encoder>
> </appender>
> ```

After

```
<!-- Boot 4.0: UTF-8 is default; specify charset explicitly
<appender name="FILE" class="ch.qos.logback.core.FileAppende
    <file>app.log</file>
    <encoder>
        <charset>UTF-8</charset>
        <pattern>%d{yyyy-MM-dd} %-5level %msg%n</pattern>
    </encoder>
</appender>
```

How To Fix

Update log consumers to expect UTF-8.

If log files are consumed by external tools, log aggregators, or scripts, ensure those consumers are configured to read UTF-8. This is the correct long-term fix.

Restore platform encoding if needed.

If UTF-8 is not acceptable for your environment, add an explicit `<charset>` to your file appenders in `logback-spring.xml`. This is a short-term workaround; prefer updating consumers to UTF-8.

Scope Check

Check `logback-spring.xml` and `logback.xml` for file appenders that do not specify a `<charset>`. Also audit downstream tools that specify the input encoding for log files: Splunk forwarders, Fluentd parsers, Elastic Filebeat configs.

Watch Out

- This only affects file appenders. Console output uses the JVM console charset (usually controlled by the terminal), not forced to UTF-8.
- Custom Logback configurations that already declare `<charset>UTF-8</charset>` explicitly are unaffected.

Verify

Log output contains correctly encoded characters (especially non-ASCII) in both console and file appenders after the charset change

Further Info

Boot 3.5 used the platform's default encoding: often Cp1252 or GBK on Windows, sometimes ISO-8859-1 on Linux. Consumers built around those encodings will misread the new UTF-8 bytes in international text, currency symbols, and emoji.

Links

- Spring Boot 4.0 Migration Guide

MongoDB Configuration Properties Renamed

Tier 3 - Behaves Differently Impact: S ⚙ OpenRewrite

`spring.data.mongodb.` *properties are silently ignored in Boot 4.0. The new prefix is* `spring.mongodb.`. The app starts but connects with defaults: wrong host, no auth, wrong database.

What You'll See

```
# application.properties (Boot 3.5 style, ignored on 4.0):
spring.data.mongodb.uri=mongodb://user:pass@prod-host:27017/mydb
spring.data.mongodb.database=mydb

# Boot 4.0: app starts, connects to localhost:27017 with no auth.
# No error: wrong database, wrong host. Data operations succeed
# against the wrong instance.
```

What Changed

All MongoDB connection and configuration properties moved from the `spring.data.mongodb` namespace to `spring.mongodb`. Examples: `spring.data.mongodb.uri` → `spring.mongodb.uri`, `spring.data.mongodb.database` → `spring.mongodb.database`. Actuator management properties changed from `management.health.mongo.` *to* `management.health.mongodb.`.

Why

MongoDB auto-configuration moved into a dedicated persistence module, and the old prefix was a legacy of living inside Spring Data's namespace. The rename matches Boot's own namespace conventions.

The Fix

application.properties
Before

```
spring.data.mongodb.uri=mongodb://user:pass@host:27017/mydb
spring.data.mongodb.database=mydb
spring.data.mongodb.auto-index-creation=true
```

After

```
spring.mongodb.uri=mongodb://user:pass@host:27017/mydb
spring.mongodb.database=mydb
spring.mongodb.auto-index-creation=true
```

Actuator health properties
Before

```
management.health.mongo.enabled=false
```

After

```
management.health.mongodb.enabled=false
```

How To Fix

*Rename all spring.data.mongodb. properties to spring.mongodb.***

Do a project-wide search-and-replace: `spring.data.mongodb.` → `spring.mongodb.`. Check all `application.properties`, `application.yml`, and any profile-specific variants. OpenRewrite has a migration recipe that handles this automatically.

Scope Check

Grep for `spring.data.mongodb` across all property files, YAML files, and `@Value` annotations. Also check `management.health.mongo`.

Watch Out

- The app starts, connects with driver defaults, and runs normally against the wrong instance. Confirm the property is being read by testing with a URI that would obviously fail, such as a non-existent host.
- Spring Session MongoDB property keys changed separately: `spring.session.mongodb.` → *`spring.session.data.mongodb.`*. Check the spring-session-property-renames card.

Verify

Application connects to MongoDB and all configuration (URI, auth, pool size, etc.) applies correctly after renaming properties

Further Info

Every property under the old prefix moved, pool size and auth settings included.

Links

- spring-break module: mongodb-property-renames
- Spring Boot 4.0 Migration Guide

MongoDB UUID and BigDecimal Representations No Longer Defaulted

Tier 3 - Behaves Differently Impact: M

Boot 4.0 removes the default UUID and BigDecimal BSON representations. Documents written under Boot 3.5 defaults may deserialise wrongly or fail. Configure the representations explicitly.

What You'll See

```
// Boot 3.5: UUID stored as STANDARD binary subtype 4
// Boot 4.0 default: UUID stored as JAVA_LEGACY binary subtype 3

// Reading a Boot 3.5 document in a Boot 4.0 app:
org.bson.codecs.configuration.CodecConfigurationException:
  No codec found for UUID with representation JAVA_LEGACY

// Or: UUID field deserialises to null or a wrong value
```

What Changed

The `spring.mongodb.representation.uuid` and `spring.data.mongodb.representation.big-decimal` properties must now be configured explicitly. Spring Boot 4.0 no longer injects defaults for these representations. For compatibility with existing data, declare the same values Boot 3.5 set automatically.

Why

Boot's defaults overrode the MongoDB driver defaults, surprising teams using MongoDB outside Spring Boot or migrating data between tools. Explicit configuration puts the application in control of the data format.

The Fix

application.properties — restore Boot 3.5 representations

Before

```
# Boot 3.5: these were set automatically (no configuration n
```

After

```
# Boot 4.0: must be explicit to match what Boot 3.5 used
spring.mongodb.representation.uuid=standard
# For BigDecimal - check what representation your Boot 3.5 d
# spring.data.mongodb.representation.big-decimal=decimal128
```

How To Fix

Declare the representations that match your existing data.

Set `spring.mongodb.representation.uuid` to `standard` if your documents were written with Boot 3.5 defaults. For BigDecimal, check which BSON type your existing documents use (inspect with mongosh) and configure `spring.data.mongodb.representation.big-decimal` accordingly.

Migrate existing data if changing representations.

To adopt different representations going forward, write a migration script to convert existing documents first. Changing representation without migrating data makes old documents unreadable.

Scope Check

Grep for `UUID` fields in MongoDB document classes (`@Document`). Also find any `BigDecimal` or `BigInteger` fields. These are at risk. Run a read test against your production data snapshot before deploying.

Watch Out

- Wrong-representation reads return valid-looking, wrong UUIDs. No exception, no log line.
- Test against a real data snapshot. Unit tests create fresh documents and never catch representation mismatches.

Verify

UUID and BigDecimal fields round-trip correctly and existing documents with the old representation are still readable after explicit configuration

Further Info

Boot 3.5 set UUID representation to `STANDARD` and BigDecimal to a specific codec. With no Boot default, the representation falls back to whatever the MongoDB driver or Spring Data

MongoDB defaults to. New documents may use a different binary subtype, breaking code that inspects raw BSON.

Links

- Spring Boot 4.0 Migration Guide
- MongoDB UUID Representations

@NullMarked Default — IDE/Compiler Null Safety

Tier 3 - Behaves Differently Impact: M

Spring Framework 7 applies `@NullMarked` to every package. IDEs and null-checking tools now flag your existing null-passing code as warnings or errors.

What You'll See

```
// Your existing code — unchanged, worked fine in Boot 3.5
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
String name = ctx.getEnvironment().getProperty("app.name");
int length = name.length();  // ← IDE warning: name may be null

// IntelliJ inspection results after upgrade
WARNING: Method invocation 'length' may produce NullPointerException
  at MyService.java:42
WARNING: Passing 'null' argument to parameter annotated as @NonNull
  at MyConfig.java:18

// If using -Werror or NullAway/ErrorProne
$ mvn compile
[ERROR] MyService.java:[42,24] error: [NullAway] dereferencing expression
  name, which is @Nullable
[ERROR] MyConfig.java:[18,35] error: [NullAway] passing @Nullable parameter
  where @NonNull is required
```

What Changed

Spring Framework 7 added `@NullMarked` (from JSpecify) at the package level across the entire framework. This means all method parameters and return types are assumed `@NonNull` by default unless explicitly annotated `@Nullable`. IDEs, static analysis tools, and compile-time null checkers now see Spring's null contracts and flag violations.

Why

Formal null contracts let tools catch `NullPointerException` bugs at compile time instead of runtime. Kotlin interop also improves: Kotlin's type system can now correctly infer nullability from Spring APIs.

The Fix

> Handle nullable return from getProperty()
> Before

```
String name = env.getProperty("app.name");
return name.toUpperCase();
```

After

```
String name = env.getProperty("app.name");
if (name == null) {
    throw new IllegalStateException("app.name not configured
}
return name.toUpperCase();
```

Or use the non-null variant
Before

```
String name = env.getProperty("app.name");
```

After

```
String name = env.getRequiredProperty("app.name");
```

Suppress warnings during migration
Before

```
// (no null warnings in Boot 3.5)
```

After

```
// Temporarily suppress in build.gradle
tasks.withType(JavaCompile) {
    options.compilerArgs += ['-Xlint:-nullness']
}
```

How To Fix

Fix null handling in your code (recommended).

Follow the IDE warnings and add null checks, use `Optional`, or switch to non-null API variants like `getRequiredProperty()`. These are real NPE bugs that were always there; the framework now surfaces them.

Suppress during migration.

If the volume of warnings is too high to fix at once, suppress null-checking warnings at the compiler level and create a backlog to address them incrementally. Don't leave the suppression in place permanently.

Scope Check

Open your project in IntelliJ or VS Code with Java support and look at the new inspection warnings. Any code that passes `null` to a Spring API parameter, or uses a Spring API return value without a null check, will be flagged. The volume depends on how much Spring API surface your code touches directly.

Watch Out

- If your CI pipeline uses `–Werror`, NullAway, ErrorProne, or Checker Framework, the new null annotations turn warnings into build failures. The framework's null contracts are now visible to your tools; unchanged code becomes a broken build.
- Kotlin projects are affected differently. Kotlin's compiler uses these annotations to infer platform types. A Spring method that returned `String!` (platform type, nullable unknown) now returns `String` (non-null) or `String?` (nullable). Kotlin code that didn't handle nullability may now fail to compile.

Verify

No new NullPointerException at boundaries where nulls were OK before

Further Info

Runtime behaviour is unchanged: the impact lands entirely in compilers, IDEs, and static analysis tools.

Links

- Spring Boot 4.0 Migration Guide
- JSpecify @NullMarked

Controller/View Spans Silently Disabled

Tier 3 - Behaves Differently Impact: M

Spring Boot 4.0 disables automatic controller and view rendering spans by default. Observability dashboards show gaps where traces used to appear; the app itself works fine.

What You'll See

```
// Grafana / Jaeger trace — before (Spring Boot 3.5)
── HTTP GET /api/orders ──────────────────────────
  └── controller: OrderController.getOrders  [12ms]
      └── view: orders/list                  [3ms]
          └── jdbc: SELECT * FROM orders     [8ms]

// Grafana / Jaeger trace — after (Spring Boot 4.0)
── HTTP GET /api/orders ──────────────────────────
  └── jdbc: SELECT * FROM orders             [8ms]
// controller and view spans MISSING

// Dashboard alert
"No data" for panel "Controller Response Time p99"
// SLO breach: "99th percentile latency unknown"
```

What Changed

Spring Boot 4.0 disabled the automatic instrumentation for Spring MVC controller method spans and view rendering spans by default. The properties `management.observations.http.server.requests.enabled` and related `spring.mvc.observation` settings changed their defaults. The traces still record HTTP and JDBC spans, but the controller-level granularity is gone.

Why

Controller spans added an extra span per request and duplicated information already in the HTTP server span. The new default cuts overhead and storage costs, with opt-in for teams that need the detail.

The Fix

application.properties — re-enable controller spans
Before

```
# (default in Boot 3.5: controller spans enabled)
```

After

```
management.tracing.enabled=true
spring.mvc.observation.auto-configuration.enabled=true
```

Or enable via ObservationRegistry customiser
Before

```
# (auto-instrumentation was on by default)
```

After

```
@Bean
public ObservationRegistryCustomizer<ObservationRegistry> se
    return registry -> registry.observationConfig()
        .observationHandler(new DefaultMeterObservationHandl
}
```

Micrometer tracing config
Before

```
# (no explicit config needed in Boot 3.5)
```

After

```
management.observations.http.server.requests.enabled=true
management.observations.http.client.requests.enabled=true
```

How To Fix

Re-enable controller observation spans.

Add `spring.mvc.observation.auto-configuration.enabled=true` and `management.observations.http.server.requests.enabled=true` to your `application.properties`. This restores the controller-level spans in your traces.

Update dashboards to use HTTP spans.

If the overhead matters, update your Grafana/Datadog dashboards to use the HTTP server span (`http.server.requests`) instead of the controller span. This span is still present and includes the handler information as attributes.

Scope Check

Check your observability dashboards, SLO definitions, and alerting rules. Any panel or alert that queries for controller-level span names (e.g. `controller.*`) or view rendering spans will show "No data" after the upgrade. Also check custom `SpanExporter` or `ObservationHandler` beans that filter by span name.

Watch Out

- Logs and functionality look normal. The only symptom is missing data in your observability platform, often unnoticed until an incident needs the traces that are gone.
- SLO alerts based on controller span latency percentiles now fire as "no data", which some alerting systems treat as "OK". Your SLO monitoring silently disappears.

Verify

All Grafana/Jaeger panels show data — no 'No data' gaps

Further Info

Part of Boot 4.0's Micrometer Tracing defaults overhaul. See also: controller-spans-disabled, aspectj-observed.

Links

- Spring Boot 4.0 Migration Guide

Path Matching Engine: AntPathMatcher → PathPattern

Tier 3 - Behaves Differently Impact: M

Spring Security now uses `PathPattern` instead of `AntPathMatcher`. Some wildcard patterns match differently, causing silent authorisation gaps or false denials.

What You'll See

```
// Security configuration — unchanged
http.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/**/admin").hasRole("ADMIN")
);

// Before (AntPathMatcher — Spring Boot 3.5)
GET /api/v1/admin         → 403 (matched, requires ADMIN)
GET /api/v1/users/admin   → 403 (matched, ** crosses segments)

// After (PathPattern — Spring Boot 4.0)
GET /api/v1/admin         → 403 (still matched)
GET /api/v1/users/admin   → 200 (NOT matched — pattern behaves differently)

// Security test failure
Expected: request denied (403)
  Actual: request allowed (200)
// Potential security hole: endpoint exposed without authorisation
```

What Changed

Spring Boot 4.0 switched the default URL path matching engine from `AntPathMatcher` to `PathPatternParser` for both MVC request mapping and Security request matchers. While most patterns work identically, edge cases around ** in the middle of patterns, trailing slashes, and encoded path segments differ.

Why

`PathPatternParser` is significantly faster (pre-parsed at startup, no runtime string manipulation), handles path variables and matrix parameters correctly, and is the default in Spring WebFlux. Unifying on one engine simplifies the framework.

The Fix

> Fix: use PathPattern syntax for multi-segment match
> Before

```
.requestMatchers("/api/**/admin").hasRole("ADMIN")
```

After

```
.requestMatchers("/api/{*path}/admin").hasRole("ADMIN")
```

Trailing slash no longer matches by default
Before

```
.requestMatchers("/api/users").authenticated()
```

After

```
.requestMatchers("/api/users", "/api/users/").authenticated(
```

Fall back to AntPathMatcher if needed
Before

```
// using default PathPatternParser
```

After

```
@Bean
public WebSecurityCustomizer legacyMatching() {
    return web -> web.httpFirewall(new StrictHttpFirewall())
}
// In security config:
.requestMatchers(new AntPathRequestMatcher("/api/**/admin"))
    .hasRole("ADMIN")
```

How To Fix

Audit all path patterns in security config.

Review every `requestMatchers()` pattern for uses of `**` in the middle (not at the end), trailing slashes, and URL-encoded segments. Test each pattern with PathPattern semantics. Use `{*varName}` for capturing multi-segment path variables.

Use AntPathRequestMatcher for legacy patterns.

For patterns that can't be rewritten, wrap them in `new AntPathRequestMatcher(pattern)` to use the old matching engine on a per-rule basis.

Scope Check

Check every `.requestMatchers()` call in your security configuration and every `@RequestMapping` pattern in your controllers. Patterns with **in the middle, trailing slashes, or regex segments are the most likely to break. Simple patterns like `/api/users/`** (at the end) are generally safe.

Watch Out

- A pattern that no longer matches is a security hole: paths that `/api/**/admin` used to protect become unprotected. A Tier 3 change with Tier 0 security implications.
- `PathPattern` does not support **in the middle of a pattern (e.g. `/api/``/admin`)**. It only supports **at the end (e.g. `/api/`)**. Use `{*path}` for variable-length segment capture in the middle.

Verify

All API endpoints respond (especially wildcard/regex patterns)

Further Info

PathPatternParser has been WebFlux's default since Spring 5.0; Boot 4.0 brings MVC and Security into line. See also: security-dsl-rewrite, auth-default-deny.

Links

- Spring-Break Demo
- Spring Security 7 migration guide
- Spring Boot 4.0 Migration Guide

maxAttempts Now Means Retries, Not Total Attempts

Tier 3 - Behaves Differently Impact: M

`@Retryable(maxAttempts = 3)` now performs 3 retries (4 total calls) instead of 3 total attempts. Every retry site silently gains an extra call.

What You'll See

```
// Before (Spring Boot 3.5): 3 total attempts
INFO  Calling payment service (attempt 1/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/3)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/3)
ERROR Payment failed after 3 attempts

// After (Spring Boot 4.0): 1 initial + 3 retries = 4 total
INFO  Calling payment service (attempt 1/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 2/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 3/4)
WARN  Payment failed, retrying...
INFO  Calling payment service (attempt 4/4)
ERROR Payment failed after 4 attempts

// Integration test failure
Expected invocation count: 3
  Actual invocation count: 4
```

What Changed

The `maxAttempts` parameter in `@Retryable` changed semantics. In Spring Retry 1.x (Boot 3.5) it meant total attempts including the initial call. In Boot 4.0 it means retries after the initial call. `maxAttempts = 3` now makes 4 calls.

Why

The old naming was misleading: "max attempts" of 3 producing only 2 retries was a constant source of off-by-one bugs. The parameter name now matches what it counts, the number of retries.

The Fix

Fix: reduce maxAttempts by 1 to preserve old behaviour
Before

```
@Retryable(maxAttempts = 3)
```

After

```
@Retryable(maxAttempts = 2)  // 1 initial + 2 retries = 3 to
```

Or use the new explicit parameter
Before

```
@Retryable(maxAttempts = 3)
```

After

```
@Retryable(retries = 2)  // clearer: 2 retries after initial
```

RetryTemplate configuration
Before

```
RetryTemplate template = RetryTemplate.builder()
    .maxAttempts(3)
    .build();
```

After

```
RetryTemplate template = RetryTemplate.builder()
    .maxAttempts(2)  // was 3 — now means retries, not total
    .build();
```

How To Fix

Subtract 1 from every maxAttempts value.

Search for all `@Retryable` annotations and `RetryTemplate` configurations. Reduce each `maxAttempts` value by 1 to preserve the original total attempt count.

Audit retry budgets.

The extra attempt may be acceptable or even desirable. Review each retry site and decide whether the new count is correct. Document the intended behaviour either way.

Scope Check

Search for `@Retryable`, `RetryTemplate`, and any `maxAttempts` configuration in properties files. Every instance now makes one more call than before. For services with tight SLAs or rate-limited downstream APIs, this can cause cascading failures.

Watch Out

- The extra retry is invisible in logs unless you count carefully. Exponential backoff stretches the delay: `maxAttempts = 5` with 2-second backoff now takes 62 seconds to fail instead of 30.
- If your downstream service has rate limiting, the extra call per failure scenario may push you over the limit. Multiply the extra attempt by your error rate to estimate the impact.

Verify

Retried operations use correct backoff timing and max attempts

Further Info

Driven by Spring Retry 2.x (upstream of Spring Boot 4.0). Spring Retry also left the Boot BOM in 4.0, so it needs an explicit version. See also: spring-retry-bom, retryable-transaction-order.

Links

- spring-break module: retry-semantics-change
- Spring Framework Resilience docs

@Retryable + @Transactional AOP Ordering Change

Tier 3 - Behaves Differently Impact: M

The default AOP advice ordering flipped: retry now wraps outside the transaction. A retried method gets a fresh transaction each attempt instead of retrying inside the same one.

What You'll See

```
// Before (Spring Boot 3.5) — retry inside transaction
DEBUG Opening transaction
DEBUG  Attempt 1: inserting order
WARN   Optimistic lock failure, retrying...
DEBUG  Attempt 2: inserting order
DEBUG  Attempt 2: success
DEBUG Committing transaction (1 commit total)

// After (Spring Boot 4.0) — retry outside transaction
DEBUG Opening transaction #1
DEBUG  Attempt 1: inserting order
WARN   Optimistic lock failure
DEBUG Rolling back transaction #1
DEBUG Opening transaction #2
DEBUG  Attempt 2: inserting order
DEBUG  Attempt 2: success
DEBUG Committing transaction #2

// Symptom: partial writes visible between retries
// Symptom: retry sees stale data from previous transaction
```

What Changed

The default ordering of Spring AOP advice changed so that `@Retryable` has higher precedence (runs first) than `@Transactional`. The retry logic now wraps the transaction, so each retry attempt runs in its own transaction.

Why

Retrying outside the transaction is safer: a failed transaction rolls back fully before the next attempt, avoiding stale persistence context and dirty session state. The old default caused subtle Hibernate session corruption when retrying inside a failed transaction.

The Fix

> Force retry inside transaction (old behaviour)

Before

```
@Retryable(maxAttempts = 3)
@Transactional
public void placeOrder(Order order) {
```

After

```
@Retryable(maxAttempts = 3)
@Transactional
@Order(Ordered.HIGHEST_PRECEDENCE)  // force tx to wrap retr
public void placeOrder(Order order) {
```

Or set global ordering in configuration
Before

```
# (default ordering in Boot 3.5: tx outside, retry inside)
```

After

```
@EnableRetry(order = Ordered.LOWEST_PRECEDENCE)
// Ensures retry runs inside the transaction
```

Best practice: separate the concerns
Before

```
@Retryable @Transactional
public void placeOrder(Order order) { ... }
```

After

```
// Outer bean: handles retry
@Retryable(maxAttempts = 3)
public void placeOrderWithRetry(Order order) {
    orderService.placeOrder(order);
}
// Inner bean: handles transaction
@Transactional
public void placeOrder(Order order) { ... }
```

How To Fix

Separate retry and transaction into different beans (recommended).

Move `@Retryable` to an outer service and `@Transactional` to an inner service. This makes the ordering explicit, avoids AOP precedence surprises, and works the same regardless of framework defaults.

Set explicit advice ordering.

Use `@EnableRetry(order = ...)` and `@EnableTransactionManagement(order = ...)` to explicitly control which advice runs first. Document the intended order.

Scope Check

Search for any method annotated with both `@Retryable` and `@Transactional`. Each one has silently changed its transactional behaviour. Also check for `@Retryable` methods that call `@Transactional` methods on the same bean (self-invocation doesn't go through the proxy).

Watch Out

- If your retry logic depends on reading data written earlier in the same transaction, that data is now rolled back before the retry. The second attempt starts with a clean slate, which may cause duplicate inserts or missing context.
- Optimistic locking exceptions (`OptimisticLockException`) are now handled differently. Previously the retry happened inside the same persistence context. Now each retry gets a fresh EntityManager, which is usually better but changes the behaviour.

Verify

Retried transactional methods execute retries outside the transaction

Further Info

Driven by a Spring Framework 7.0 change in default AOP advice ordering. See also: retry-semantics, spring-retry-bom.

Links

- Spring-Break Demo
- Spring Boot 4.0 Migration Guide

Spring Session Property Prefixes Renamed

Tier 3 - Behaves Differently Impact: S

`spring.session.redis.` *and* *`spring.session.mongodb.`* are silently ignored in Boot 4.0. The new prefixes are `spring.session.data.redis.` *and* *`spring.session.data.mongodb.`*.

What You'll See

```
# application.properties (Boot 3.5 style, ignored on 4.0):
spring.session.redis.namespace=myapp
spring.session.redis.flush-mode=on-save
spring.session.mongodb.collection-name=sessions

# Boot 4.0: app starts, sessions stored under default namespace.
# No error. Session isolation between apps sharing the same Redis
# instance is broken.
```

What Changed

Spring Session's Redis backend properties moved from `spring.session.redis.` *to* *`spring.session.data.redis.`*. MongoDB backend properties moved from `spring.session.mongodb.` *to* *`spring.session.data.mongodb.`*. This mirrors the MongoDB connection property rename pattern (`spring.data.mongodb` → `spring.mongodb`).

Why

The `data` infix ties each session store's configuration to its Spring Data module namespace, making the relationship explicit.

The Fix

Redis session properties

Before

```
spring.session.redis.namespace=myapp:session
spring.session.redis.flush-mode=on-save
spring.session.redis.save-mode=on-set-attribute
spring.session.redis.cleanup-cron=0 * * * * *
```

After

```
spring.session.data.redis.namespace=myapp:session
spring.session.data.redis.flush-mode=on-save
spring.session.data.redis.save-mode=on-set-attribute
spring.session.data.redis.cleanup-cron=0 * * * * *
```

MongoDB session properties
Before

```
spring.session.mongodb.collection-name=sessions
```

After

```
spring.session.data.mongodb.collection-name=sessions
```

How To Fix

*Rename spring.session.redis. to spring.session.data.redis.***

Do a project-wide search-and-replace: `spring.session.redis.` → `spring.session.data.redis.` and `spring.session.mongodb.` → `spring.session.data.mongodb.`. Check all property files, YAML files, and profile variants.

Scope Check

Grep for `spring.session.redis` and `spring.session.mongodb` across all property and YAML files.

Watch Out

- The session store type property (`spring.session.store-type`) is unchanged. Only the store-specific configuration sub-keys are affected.
- Nothing fails at startup, which makes this hard to catch in testing. A wrong namespace in Redis means sessions are stored but never found: users appear logged out immediately after login. Confirm the fix by checking session keys exist in Redis under the expected namespace.

Verify

Session store connects correctly with the renamed property prefixes — confirm sessions persist to Redis or MongoDB with the new configuration

Further Info

Old-prefix properties still parse, so the app starts cleanly. Sessions fall back to defaults (wrong namespace, wrong collection, wrong TTL) or the auto-configuration picks the first available store type.

Links

- spring-break module: spring-session-property-renames
- Spring Boot 4.0 Migration Guide